

PYTHON MINESWEEPER

Name: Ciaran O'Hara

Candidate number: 3100

Centre name: King Edward VI 6th Form College

Centre number: 31190

Qualification: OCR H446/3

Date: 13/07/2026

26/6/2024

CONTENTS

Project Analysis	6
Introduction	6
Problem Identification	6
Examples	6
Inputs, processes and outputs	7
Stakeholders.....	7
Interview	7
Conclusion.....	8
Research.....	8
Existing solutions.....	8
Project versus existing solutions.....	10
Features and Limitations.....	10
Essential features	10
Why they are essential.....	11
Extra/Stretch features.....	11
Limitations.....	11
Requirements.....	12
Hardware requirements.....	12
Software requirements	12
Storage requirements	13
Success Criteria	13
Program Design.....	15
Decomposition	15
Intialise/ Creating the main grid	15
Calculating the numbers	16
Clearing the empty tiles	18
Create Other Lists.....	21
Win/ Lose Conditions	21
Structure (sections of code).....	22
Algorithms.....	25
Interface	25
Key Data Structures and Variables.....	26
Validation	27
Validation Methods.....	27
Test Design	28

Prototype One – Text-Based Minesweeper.....	29
Introduction	29
Classes, Functions/ Procedures & Data Structures.....	29
'Tile' Class.....	29
Initialise	31
CalcNumber.....	33
Choice.....	36
Printboard	37
Fill	39
Checkwin	42
Testing.....	43
Test Plan.....	43
Test Results (Table)	44
Test Results (Images).....	45
Fixed Errors.....	47
Fixed Errors (Images).....	47
Conclusion.....	47
Prototype Two – Minesweeper in PyGame	49
Introduction	49
Program Structure.....	49
Main Data & Where It Is Stored	49
Display Order.....	49
Classes, Functions/ Procedures & Data Structures.....	50
'Tile' Class.....	50
GridImages	50
creategrid.....	52
createimagegrid	53
minedistance.....	54
CheckWin	56
ClearEmpty.....	57
firstgo	57
The Game Loop	58
Before the loop starts	58
The Event Loop.....	59
Display.....	60
Tile Interaction	61
End of the Loop	63

Testing	63
Test plan	64
Test Results (Table)	65
Test Results (Images, if applicable)	65
Fixed Errors.....	68
Fixed Errors (Images).....	71
Next Version	71
Conclusion	72
Final Version – Python Minesweeper	73
Introduction	73
Program Structure.....	73
Main Data & Where It Is Stored	73
Display Order.....	74
Classes, Functions/ Procedures & Data Structures.....	75
'Tile Class'	75
'Notch Class'	76
GridImages	76
createSlider	77
activeNotchDetection	78
creategrid.....	79
createimagegrid	80
minedistance.....	80
CheckWin	81
ClearEmpty.....	81
firstgo	81
displayGrid.....	82
resetGrid.....	82
actAdd	83
actRemove.....	83
playForfeit	83
buttonState	84
hiScores	85
The Game Loop	87
Before the loops starts.....	87
The Event Loop.....	88
Display	89
Playing Loop	91

End of the loop	91
Testing	92
Test Plan	93
Test Results (Table)	94
Test Results (Images, if applicable)	95
Fixed Errors.....	99
Fixed Errors (Images).....	99
Final Program Images.....	102
Conclusion	105
Program Evaluation.....	107
Success Criteria	107
User Testing.....	108
Usability.....	108
Appearance	109
Playing Experience	109
Errors/Limitations	109
Changes They Want To See.....	110
Overall Opinion	110
Reflection/ Next Steps.....	110
Overall Conclusion.....	111

PROJECT ANALYSIS

INTRODUCTION

For this programming project, I am going to create my own version of the game Minesweeper. It will follow the fundamental rules of the game, such as the moves the player can make, and relocating the mine if the player lands on one on their first go.

Minesweeper is a logical video game released in 1989. The goal of the game is to flag all the mines hidden in a grid of tiles. The locations of the mines are determined by numbers on the tiles nearby to the mines (one tile away from the mine) corresponding to how many mines are around that tile. The player can then 'flag' the mines once they think they have found one. The player wins if all the mines have been found, or if the only tiles remaining are mines. The player loses if they hit a mine. The only exception to this is when the player hits a mine on their first go, in which the mine will be relocated somewhere else on the grid and the numbers recalculated.

PROBLEM IDENTIFICATION

The game relies on mathematical calculations and data comparisons, which are essential for calculating the number of mines around a certain point, and area filling, for the entire grid.

EXAMPLES

- One of the most important aspects of Minesweeper is the numbers on the grid, which tell the player how many mines are. Without it, the game would be unplayable. I have created a loop which will repeat the same set of mathematical calculations on every point on the grid, also using logical comparisons to avoid performing the calculations on tiles which would cause the program to crash.

The function relies on maths to manipulate coordinates of the tiles and to keep tally of mines around that tile.

- Another aspect is the automatic clearing of all touching empty tiles when one is cleared by the user. This uses an area filling algorithm, which uses mathematical calculations to find the coordinates of nearby tiles and then logical comparisons to check they fit the criteria to be cleared. There are an extra set of calculations and comparisons to add one extra 'ring' around the area, to include the numbers to make the game playable.

The function relies on maths to manipulate the coordinates of the tiles and logic to ensure that for a point to be cleared, it meets all the criteria required.

Both examples use numerous mathematical and logical calculations, which is what computer excel in solving. Therefore, this means that the problem is solvable by computational methods as the program relies on mathematical and logical calculations and working out.

INPUTS, PROCESSES AND OUTPUTS

The problem can be defined by the inputs, processes, and outputs below:

- Inputs – The program takes the command word and a set of coordinates through a text input. The program will translate these into actions which will be performed in the grid. For the version which will use a GUI, the inputs will be mouse inputs by clicking on the grid. The location where it is clicked will translate to the coordinates of the pixels on the screen.
- Processes – Once the inputs have been entered, the program will use mathematical and logical calculations and comparisons to process the data which will change the tiles on the grid.
- Outputs – Once the processes have finished, the grid is changed to the new values.

STAKEHOLDERS

INTERVIEW

During a set of two interviews with potential players, I asked a series of questions about the game:

- Do you think that the game is a good way to 'kill' time?

The interviewee stated that it is a fantastic way to kill time and that "Minesweeper is one of those games that is on everything", referring to the fact that the game requires so little system requirements. They continued to add that its quick to play, so it's a good game to play if you're waiting for the bus or train.

The second interviewee said that it is a good a way to solve boredom, good for developing problem solving skills, and a good game for strategic minds.

- What are your thoughts on the game, and would you enjoy it?

Both interviewees said that they would enjoy the game.

- What are your thoughts on a hexagonal grid as an extra feature?

The first interviewee said that it would be more challenging and harder to play, but that they would enjoy the challenge. They also said that because of the added difficulty of a hexagonal grid, that it would take longer to complete the grid, meaning that the hexagonal grid would not be ideal for those wanting a short game.

The second interviewee said that it would help diversify the game and help it to stand out

- Do you have any suggestions for the game?

The first interviewee recommended to add a tutorial or explanation of how to play minesweeper for those who will be new to playing the game.

The second interviewee suggested special features like a moving mine that slowly follows the player's mouse, and if it touches the mouse, then the player loses the game. They also suggested special bonus squares which could move mines around on the grid or recreate a section of the grid, making the player lose their progress in that area and need to solve a new section where the old one was.

CONCLUSION

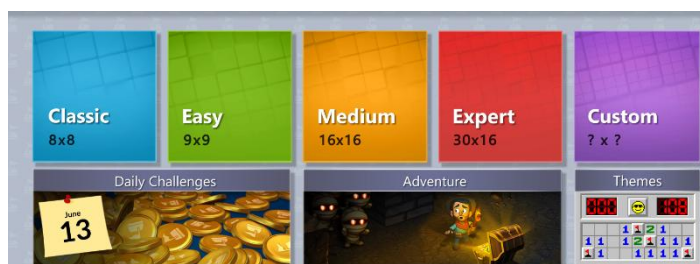
In conclusion, the likely users of this program will be people who like logical challenge or 'brain game' and those who need something quick to play whilst waiting for the bus, train, etc. It will cover a large area of people, not bound by nationality or culture.

RESEARCH

EXISTING SOLUTIONS

Microsoft Minesweeper:

Microsoft Minesweeper is the first minesweeper game ever made, and has been updated and expanded upon ever since. It features the original version with all the regular rules. However, the app game also features daily challenges and an adventure mode, which is a level-based game, where the player needs to dig through level (following minesweeper rules) to find loot and the key to get to the next level. The player has multiple lives and can find extra lives. The game also includes sound effects, which occur when the player hits a mine.

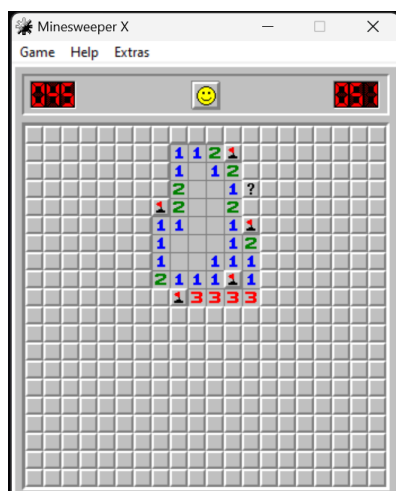


However, there are downsides to the game. The game requires an internet connection, and it will also display video and banners. There is also a paid 'premium' subscription where it will not show ads and will grant access to past daily challenges. The player will earn more XP as well and will get free

'continues', which will let the player continue playing even if they hit a mine.

Minesweeper X:

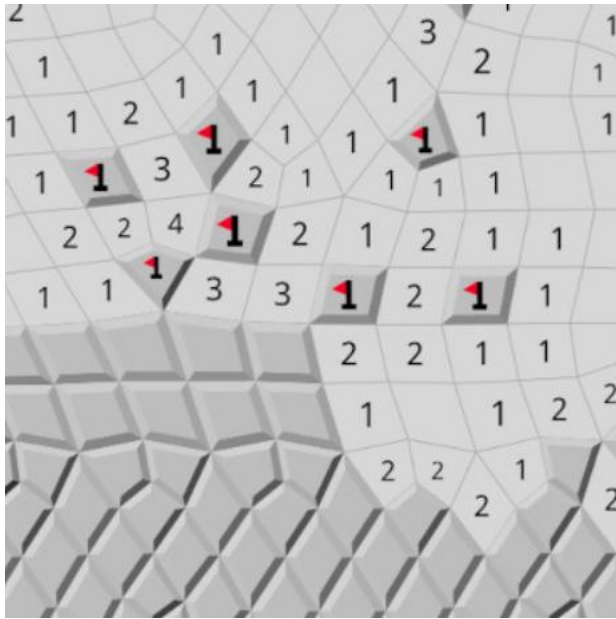
Minesweeper X is the most accurate clone of the original Microsoft Minesweeper. The game only features the classic version of minesweeper, and its videos are accepted for the world rankings of the game. It has unique features which include automatically saving videos of best times or if they're under a certain time. The ability to make your own custom skin for the grid is also available. The grid sizes are available in the different difficulties: beginner, intermediate, expert and custom, where you can choose the size of the grid yourself. However, the interface for changing the grid is a simple dropdown and text input, which may be difficult to find and use for some players.



Unlike Microsoft Minesweeper, there are no adverts and no paid content. Everything in the game is completely free.

Minesweeper Twist:

Minesweeper Twist is another of many unique styles of minesweeper games. The game follows the regular rules of minesweeper however, the grid is made of irregular polygons, and each tile can have between 6 and 10 neighbours, making it more challenging and confusing.



Some possible downsides may be that it is quite confusing to play, so could affect the amount of people playing the game, as only people who have already played minesweeper will understand how the game plays, and they too could find it too confusing.

PROJECT VERSUS EXISTING SOLUTIONS

The program will have the following advantages over other programs:

- A score system, to be able to play against people and compare previous scores.
- Open source, for people to be able to edit and change.

FEATURES AND LIMITATIONS

ESSENTIAL FEATURES

- The calculations to work out the numbers required for figuring out the location of the mines.
- Relocation of the mine to somewhere else on the grid if it is hit on the player's first go.
- The automatic clearing of empty tiles.
- The option to flag, clear or 'question mark' a tile.
- If a mine is hit, the player loses the game

WHY THEY ARE ESSENTIAL

Features	Reason for importance
Calculations the work out the number of mines nearby a tile.	This is essential for the game, as it will show the numbers on the grid which will allow the player to locate the mines and flag them.
The relocation of a mine if hit on the first go.	It will prevent the player from immediately losing the game as it would be unfair.
The automatic clearing of empty tiles.	The would be playable without this but it would take a very long time to clear the grid and would not be enjoyable.
The option to flag, question or clear a tile.	This feature is also essential for the game. Without it, the player could not make any progress, it would just be a grid and nothing else could be done with it.
If a mine is hit, the player loses the game.	Another very essential feature. Without it, the whole point of the game is lost, to clear the grid without hitting a mine.

EXTRA/STRETCH FEATURES

In addition to the essential features for the game to run, I will also include some extra features to make the game more interesting:

- A variable grid, so the user can change the size of the grid they'll be clearing.
- A score which will be displayed at the end of the game (either winning or losing). The score will be worked out by multiplying the number of mines correctly flagged by a multiplier based off how long it takes for the player to get to the end of the game.
- Saving the top 5 high scores to an external file which will be displayed in a high scores tab in the menu.
- A hexagonal grid¹, which will be played in the same way as the regular grid but will have 6 neighbouring tiles instead of the 8 a regular square grid will have.

LIMITATIONS

The program has the following limitations:

¹ <https://minesweepergame.com/download/game.php?id=26>

- Limited grid size: As the window size of the game will be unchangeable, and the grid will be static, the grid will have a maximum height and width to make sure the player can read the numbers clearly and press the tiles without the risk of pressing a different accidentally.
- Saved data: The high scores will be saved to an external text file, and any other file type will not be able to be read by the program.
- Grid skin: The grid skin will be set to the default and the user will not be able to change to a custom one.
- Sound effects: The game will not include sound, so will be silent

These limitations may be solved in future versions of the game.

REQUIREMENTS

For the game to be able to be played, a computer is required. This will mean that the computer will need to match the following requirements for the game to run well.

HARDWARE REQUIREMENTS

Since minesweeper was first created in 1989 for the computers back then. The hardware requirements are very small, as it would most likely be able to run on any modern smart device or computer.

These devices will include:

- Desktop computers and laptops
- Touchscreen smart phones
- Touchscreen tablets

SOFTWARE REQUIREMENTS

The software requirements for this game require any python IDE capable of running PyGame, and the user may also need to install PyGame using the command prompt.

This requires the device to have an operating system capable of running Python, and installing PyGame either through a command prompt, or other means. So, the game will be playable on computer operating systems such as Windows, Linux, Google and MacOS. It may also be possible on Android smartphone devices, such as Samsung, as there are many options for customisation. However, it may not be possible on Apple's iOS for

their iPhones, as the operating system is more restrictive, which is used to protect users from viruses and other malicious software.

The computer does not need to be connected to the internet to play.

STORAGE REQUIREMENTS

The program itself will most likely be less than 100KB. There will be data which will be stored in the hard drive of the computer. This is the text file for the top 10 high scores, and will most likely be less than 1KB of data needing to be stored

To make sure that the program and text file will have enough space, the entire program will require 1MB of storage.

SUCCESS CRITERIA

Below is the set of criteria which will determine whether the program is a success or not.

Requirement	Criteria	Justification
Mine-Distance calculations	The numbers next to each mine are calculated quickly (within 2 seconds) and correctly.	In order for the game to be playable.
Quick startup	The program starts up within 5 seconds.	To ensure the game is quick and efficient.
Quick processing	The program processes user inputs and changes the grid where fit within 1 second.	To ensure the game is quick and efficient.
Win and Lose conditions	Win: The program ends if all mines have been flagged or there are only mined tiles left. Lose: The program ends if a mined tile is hit, except for the first go, in which the mine will be relocated, and distances recalculated.	In order for the game to be playable.
Grid creation	The grid is successfully created to the desired height and width which is chosen by the user, within a set range.	In order for the game to be playable.

Mine locations	Using the grid and a random number generator, the grid is successfully filled with mines using the random number generator to change the likelihood that a mine is put there.	In order for the game to be playable.
Display grid	The grid is displayed correctly and is in proportion with the screen. The entire grid fits in the screen and isn't being cut off.	In order for the game to be playable.
High scores	High scores are saved onto an external text file. The score and 3 letter initials are written onto the file. Only the top 5 scores are written.	Extra features to make the program more unique and interesting.

PROGRAM DESIGN

The following sections will describe what the final version of the program should look like, both code layout and user interface.

[Speak more about usability and variables.]

DECOMPOSITION

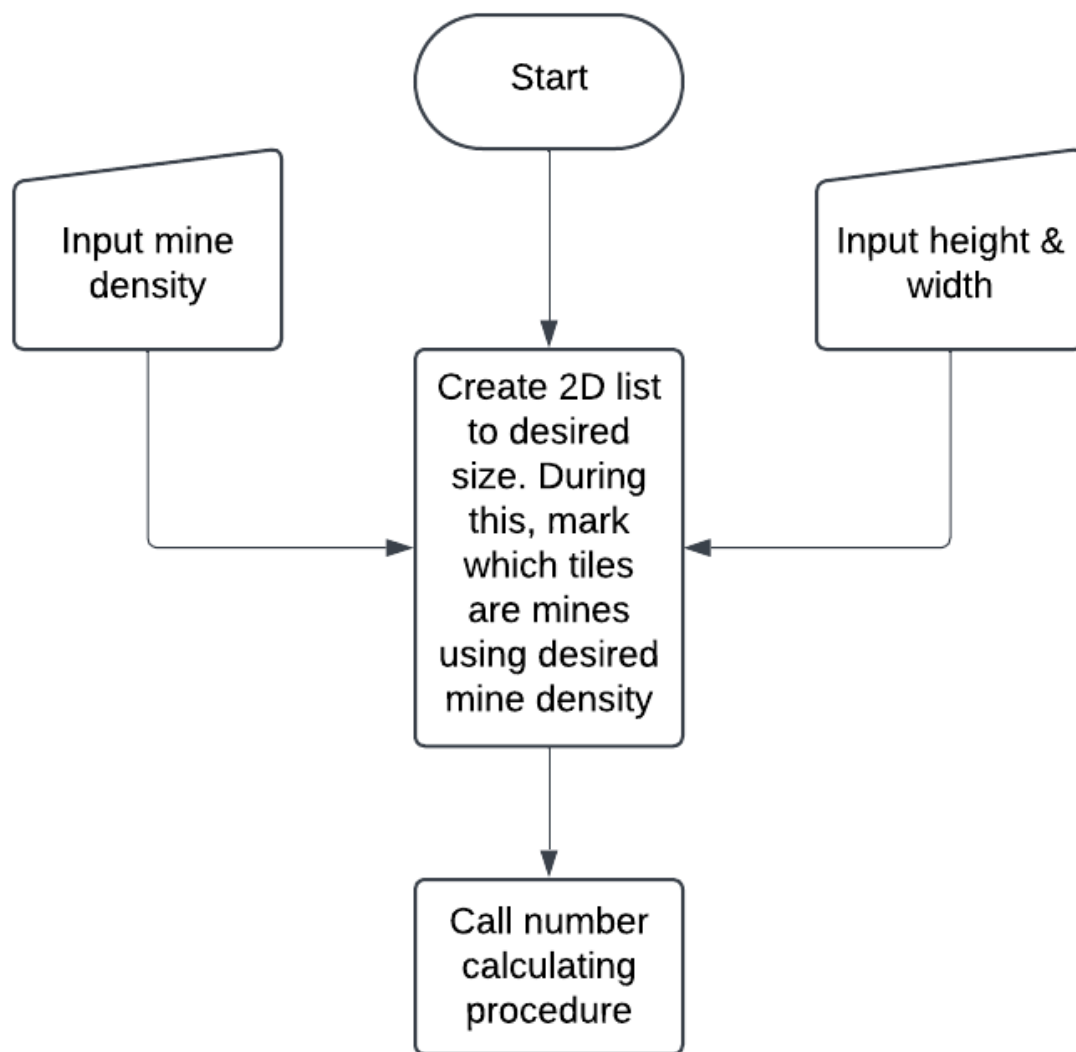
The main parts of the program will be split up into functions and procedures, with each one serving a different purpose. This will help to keep the program organized and make sure that a certain function serves a certain role.

The main functions will be the ones that handle the tiles' data and process it to create the grid and other item crucial for the game to work. The main ones are:

- Creating the main grid
- Calculating the tile numbers
- Clearing more than one empty tile at one (area filling algorithm)
- Checking if the player has won

INITIALISE/ CREATING THE MAIN GRID

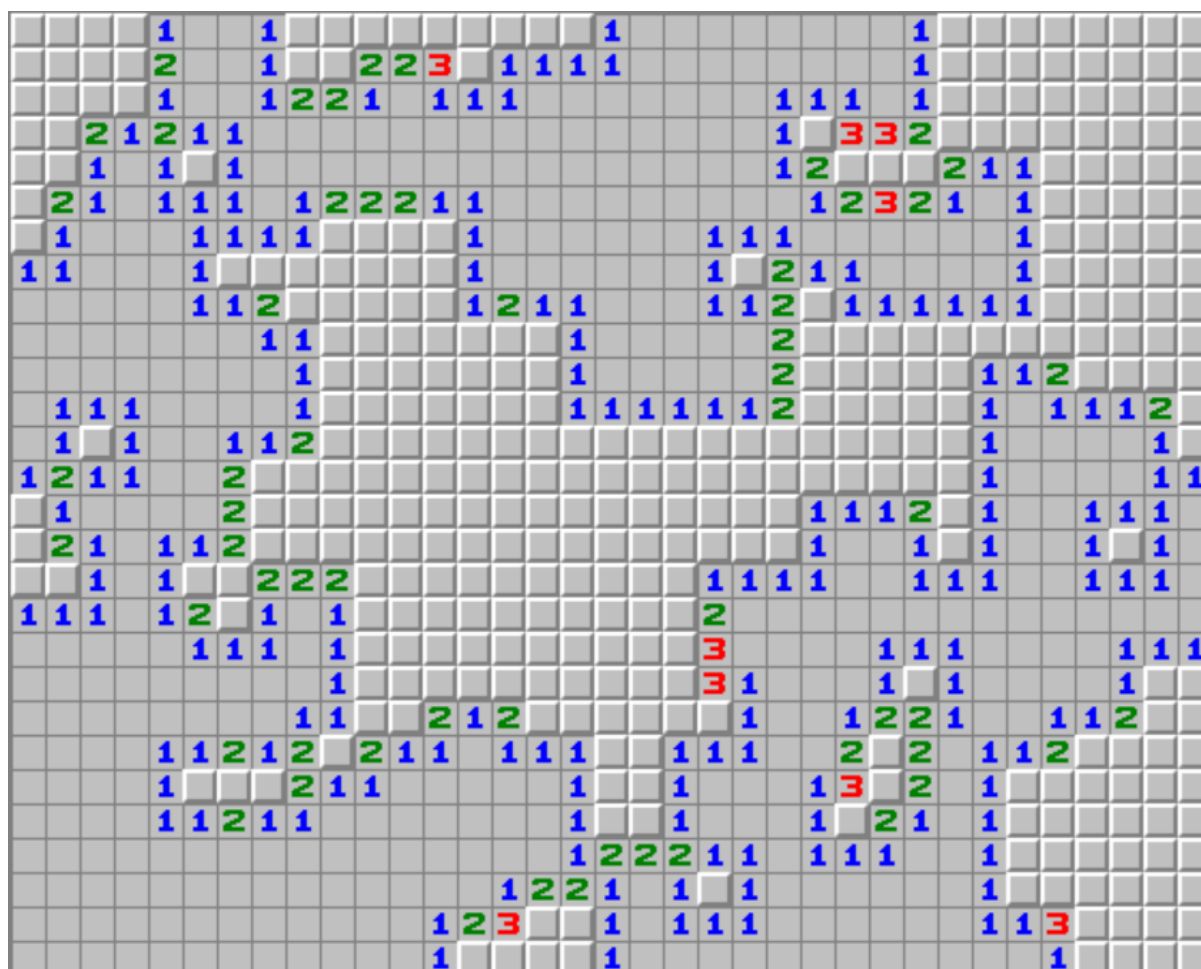
This procedure will create the main list which will store the data about the tiles that the rest of the program will need. It will also be displayed to the screen in the form of a grid, which the player will rely on to play the game.



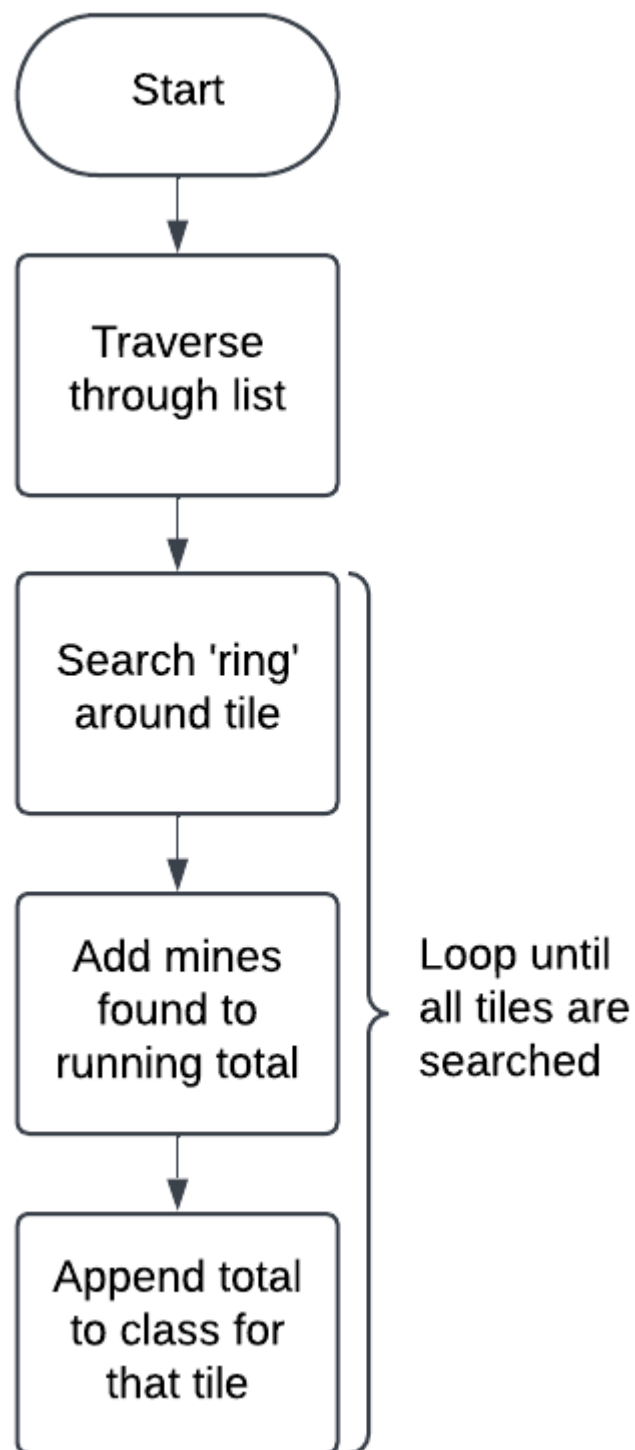
The function will work by creating a list to the selected height and width, it will then use a random number generator to choose whether the tile is a mine or not, this information will then be saved to the class at the relevant list index. For my first prototype, I will not include a class, and instead just append a symbol in its place. This is because it will be simpler to code, as the focus for the first prototype will be on proving I am able to program the rules of Minesweeper, and not on efficiency.

CALCULATING THE NUMBERS

This function will focus on creating the numbers you see on the board. An example is shown below using Minesweeper X.

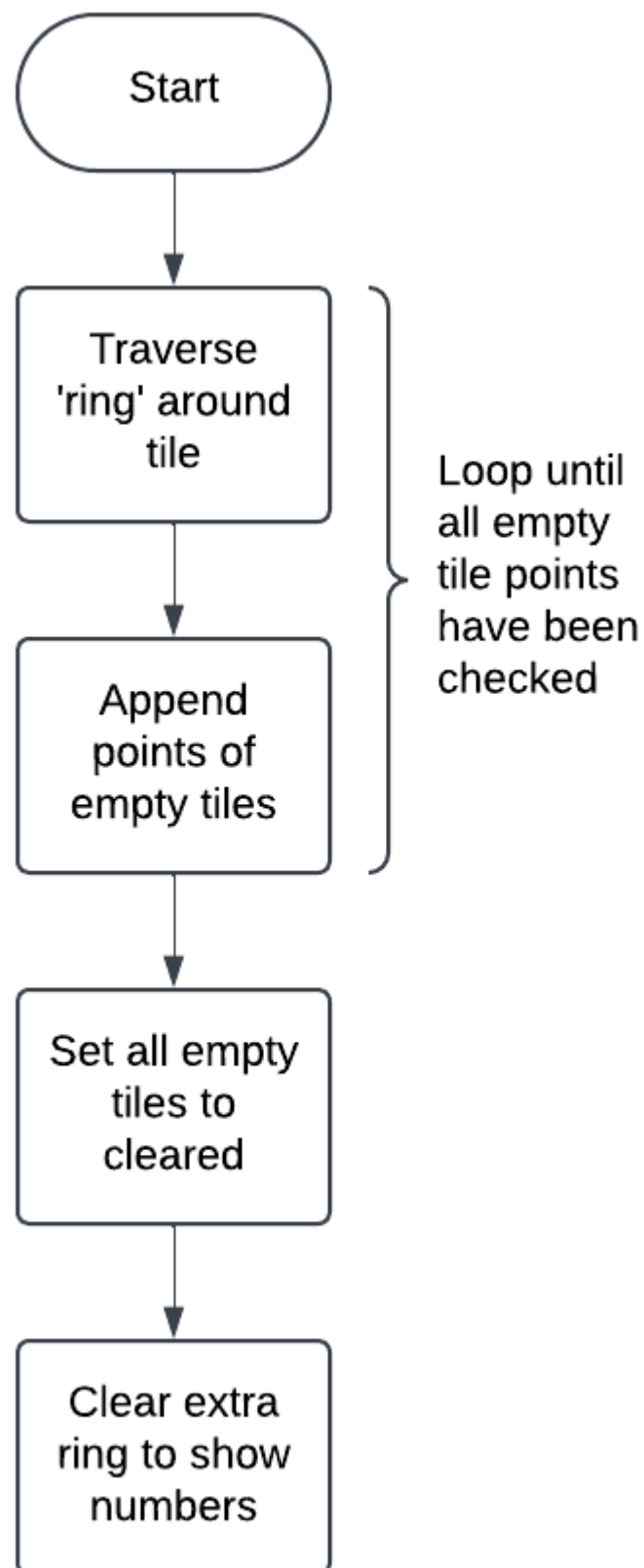


The numbers will be calculated by traversing the 8 square 'ring' around each tile, and then assigning the total number of mines in that ring to that tile. In the case of edge and corner tiles, there will be a check whether the index is equal to the length of the list, or is at an index of -1, which would mean it's out of range and would crash the program or cause a logic error (the numbers would be miscalculated). Below is the diagram for how the function will look/ be laid out.



There is a rule in Minesweeper where if you uncover an empty tile, then all adjacent empty tiles are cleared at the same time.

How I will do this is similar to the number calculating in the sense of looking at the 'ring' of tiles around the focused one. The coordinates of all empty tiles found in the ring will be appended to a list, and this will repeat for each point in the list until there are no more points to focus. And lastly an extra 'ring' will be cleared around the space, showing the first layer of numbers onto the screen.

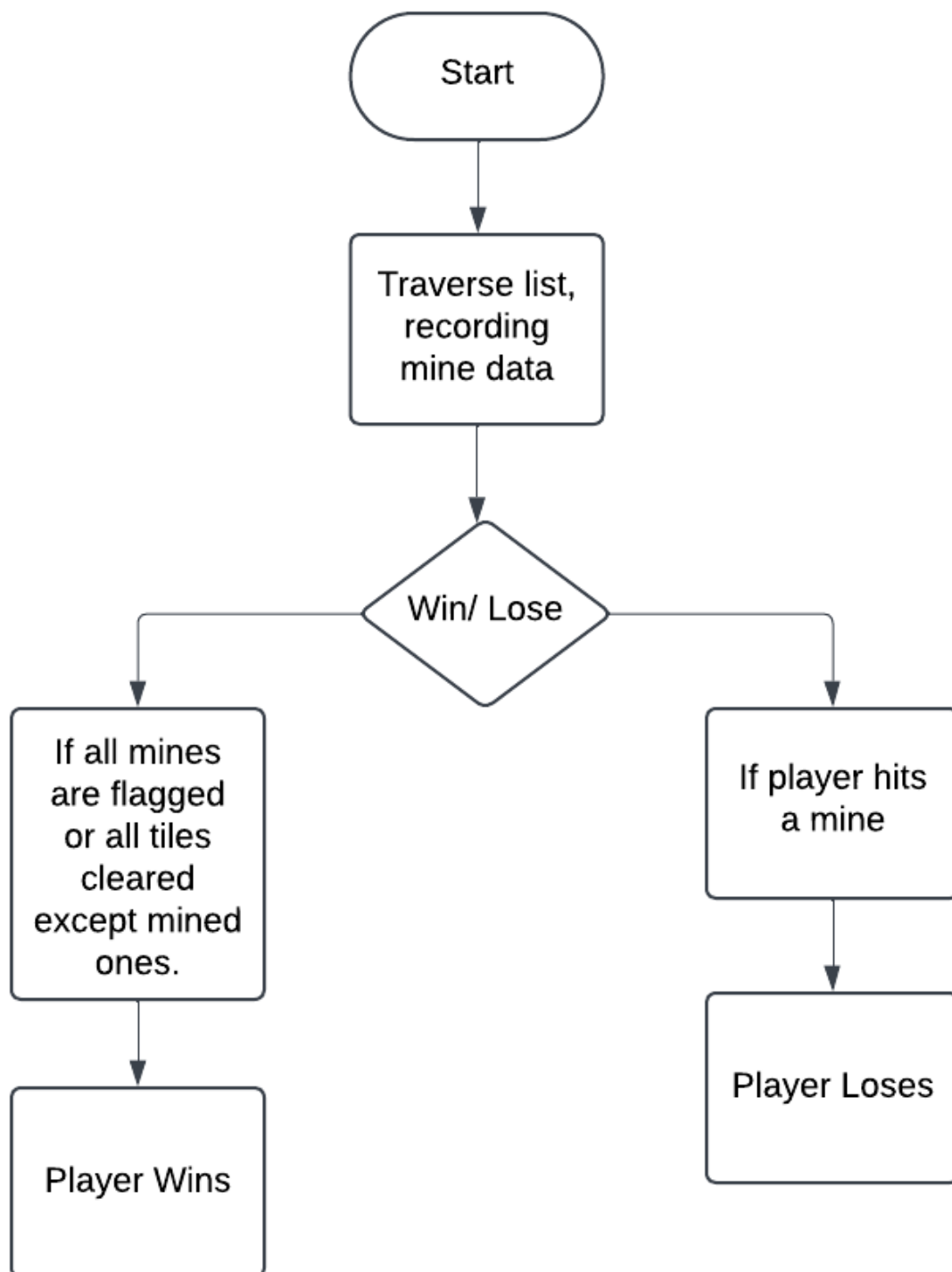


CREATE OTHER LISTS

This function creates the other lists needed for the game to run and matches the size of the main list. The code for this will be very similar to the main list creation, being made to the same size as that one. The only difference being that it does not call the function which calculates the mine numbers. For this reason, there is no flow chart being shown below.

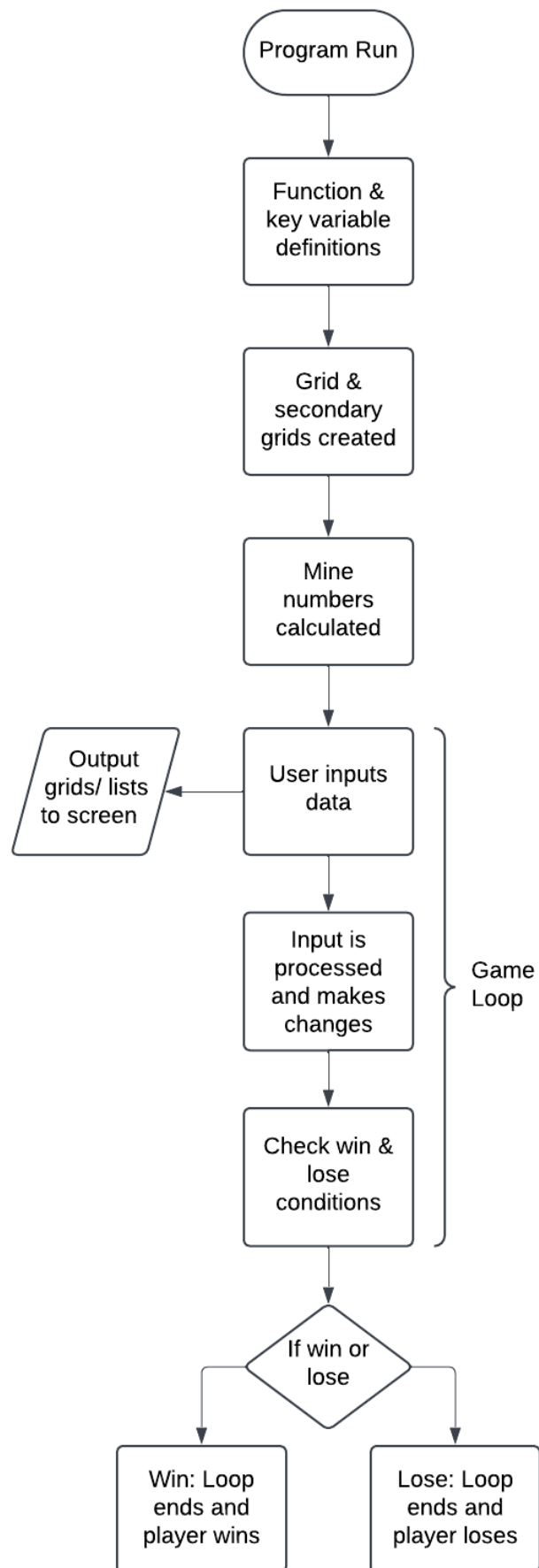
WIN/ LOSE CONDITIONS

The last function will be to check whether the user has won or lost the game. The conditions for winning will be if the user has flagged all mined tiles or has cleared all tiles except for ones that are mined. The lose condition will be if the user clears any mined tile except if it's the user's first go, in which case the mine will be relocated and numbers recalculated, allowing the user to continue. Below is the flow chart for this.



The program will be made up of functions and procedures. This is to keep the game loop as small as possible, and to organise the code so that code that does a certain thing is separated from the code that does something else.

The diagram below shows the structure of the program, note that the game loop is the only thing that outputs data to the screen.



ALGORITHMS

[flow charts for functions in there are in their respective sections]

INTERFACE

Below are the visualisations of what the screen will look like in the final version. There are two main screens which will be stored in a stack to switch between each screen depending on the stage the game is in.

The first visualisation is the menu. I have designed it in this way for it to be simple and effective and being easy to use. The right is where the user can edit the grid by changing the size and the mine density, with also displaying a high score. The rest of the screen is a sort of title screen where the game name will be displayed.

Python
Minesweeper

Width

Height

1 in x
chance to
be mine

High Score: ----

PLAY

The one below is the screen to be displayed whilst the game is being played. The right still serves as the menu but instead displays the setting that the board is set to and including a timer from when the game was started. There is a 'Forfeit' button in place of the 'Play' button to end the game. Once the game has ended, the screen will revert to the main menu in preparation for the next game.



The player will use both their keyboard and mouse to interact with the game. the keyboard will just be for editing the grid settings, and the mouse will use the left, middle and right clicks for clearing, questioning and flagging tiles. The interface will only use the left click, and anything else done by the user will not be processed, which also means the user can't crash the program by giving the wrong input, making it easier to use and more user-friendly.

The interface will make it easy for the user to edit the grid size and mine density. It's designed in a way where it's easy to use and understand what data needs to be entered where, which means the user won't need prior knowledge or tutorials to be able to work the interface.

KEY DATA STRUCTURES AND VARIABLES

Data	Type	Purpose
Tile	Class	To store all the information about an individual tile including:

		• Number • If it's a mine or not • The image
Objects	List	To store the 'Tile' class, matching the list coordinates with where the tile would be on the grid.
Win	Variable (flag)	Flag that becomes true when the player wins.
Lose	Variable (flag)	Flag that becomes true when the player loses.
fps	Variable	The number that controls the frames per second of the game loop.
GridImages	Dictionary	Store all the file addresses of all the images that will be used in the program.
imagegrid	List	Stores file paths for the images of each tile
FQgrid	List	Same as imagegrid but for the flags and questions
firstGo	Variable (flag)	Flag that shows whether the player is on their first go or not
left	Variable (flag)	Flag that becomes true when the left mouse button is pressed. Is reset at the end of each loop
right	Variable (flag)	Flag that becomes true when the right mouse button is pressed. Is reset at the end of each loop
middle	Variable (flag)	Flag that becomes true when the scroll wheel is pressed. Is reset at the end of each loop
down	Variable (flag)	Flag that becomes true is the left click has been held down for more than one loop
activeScreens	List (stack)	A stack used to store which screen is the active one (at the top of the stack). e.g menu, playing, game over
gameEnd	Variable (flag)	A flag that is true when the player either wins or loses. Its used to break out of the game loop
gameLoop	Variable (flag)	A flag that allows the game loop to run when it is true
reset	Variable (flag)	A flag that tells the program to reset the grid when it is true (a new game is created and old one is discarded)

VALIDATION

There are two main areas where the user will need to enter data, and these will need to be validated to prevent the program crashing due to the wrong input being provided. These points are:

- The menu, where the user will enter the desired grid size, and mine density.
- During the game being played, where the user will have the option to clear, flag, or question a tile. There is also the forfeit button that user can press to end the current game.

VALIDATION METHODS

For the text-based version (prototype 1), the data will be entered into the python shell. This means that for the grid sizes, the input will need to be checked it is an integer, and within a certain range. For the commands to clear, flag and question, there will need to be multiple checks to ensure the data is correct. These are:

- The command word is one of the three available.
- The command word is at the start. If not, then the code to split the command line up will not work, and the program will crash.
- The second part of the command (the coordinates) is in the format of a letter followed by an integer (e.g. a4).
- The first part of coordinate is in range for the height of the grid. This will be done by a list of the alphabet and then indexing the letter to get its number. This means the grid can only be a maximum size of 26x26.
- The second part of the coordinate is also in range for the height of height of the grid.

As the final version will have a GUI, there are different inputs which will all require validation to prevent the program from crashing while it is running. These are:

- The desired width is within the maximum and minimum values and is an integer.
- The desired height is within the maximum and minimum values and is an integer.
- The input for the '1 in x' chance of a mine appearing is greater than 0 and is an integer.
- Mouse button presses only do anything within the grid and menu area. Left clicks are only accepted in the menu, whereas left, right and middle clicks are allowed on the grid

The reason for validating the inputs is to prevent the user from accidentally, or purposely, crashing the program. This is to ensure that the program can remain user friendly and is not seen as unreliable by users.

TEST DESIGN

There are multiple points where the program can crash, so thorough testing will be required to make sure that all if not the most common errors and bugs are fixed. Below is a table showing a basic idea of the kinds of tests that will be required to make the sure the program is in a good working order.

Test	Description	Inputs	Type
Entering the wrong data into the height and width inputs in the menu.	Enter both non-integer and integer above and below the range of the allowed inputs.	Mouse – left click (hold)	Erroneous
Entering the wrong data into the mine density input.	Enter both non-integer and integer values below 1.	Same as above	Erroneous

'Spam' the mouse buttons on the grid.	Pressing every button on the mouse very quickly and randomly on the grid.	Mouse – any button	Boundary/Erroneous
Pressing more than one mouse button at once.	Tests whether the program will choose one or the other	Mouse – any 2 buttons	Erroneous
Press and holding any button	Tests what will happen if the user holds down the button instead of just clicking it	Mouse – any button (hold)	Boundary/Erroneous
Using the mouse to clear the grid	The player is able to use their mouse and the three buttons (left, right and middle) to clear the covered tiles and flag the mines	Mouse – all 3 buttons	Normal
The program detects a win or loss	The program will check whether the player has won or lost the game using predefined conditions. If the player clears all of the non-mined tiles or has flagged all mined tiles, then the player has won. And, if the player hits a mined tile (attempts to clear it), then they will lose.	No input, program functions work this out	Normal

PROTOTYPE ONE – TEXT-BASED MINESWEEPER

INTRODUCTION

The main goal of the first prototype was to serve as a proof of concept to show I can code the rules of minesweeper, and to also set a baseline of functions which I can use and adapt on for prototype 2.

All of the important code is created in functions and procedures, making all of the code modular, making it easier to tweak and aid me in the second prototype as I can use code from prototype 1.

The game loop is very short as all of the main code is called by functions and procedures.

CLASSES, FUNCTIONS/ PROCEDURES & DATA STRUCTURES

'TILE' CLASS

I started the prototype by creating the class for the tiles, so I can store multiple pieces of data about each tile in one place.

```
class Tile():  
    def __init__(self, mine, flagged, uncovered, symbol):  
  
        self.mine = mine  
        self.flagged = flagged  
        self.uncovered = uncovered  
        self.symbol = symbol
```

The class has the attributes:

- Mine – to tell the program that this tile is a mine tile, which will cause the player to lose if 'dug up'.
- Flagged – to tell the program that the tile has been flagged by the player because they think it is a mine.
- Uncovered – to tell the program that the tile has been 'dug up' so to display this tile on the next cycle.
- Symbol – the number of the tile (if it is not a mine), which relates to how many mines are around it. This is also needed by the program so it can display the number for the tile.

INITIALISE

The purpose of this procedure is to set up the three lists used to handle the data about all the tiles and to allow the player to see the grid. These are:

- **Board**
This list holds all of the symbols and numbers to be displayed onto the screen, this doesn't tell the computer where the mines and other data is located, this is solely for the player to look at, to be able to work out where the mines are and beat the game
- **Hidden**
This list is similar to 'Board', but it is made up of all crosses, '+', which represents a tile which hasn't been 'dug up' yet. When the grid is interacted with, the crosses on the screen will change to different characters depending on what is at the same coordinate for 'Board' and also if the tile is flagged or a question mark is placed.
- **Tiles**
This list holds all of the data about the tiles the program needs to run the game. The list is made up of instances of the class 'Tiles' which represent one tile each. Storing the data this way means that no more lists are needed to reference between tiles and that the coordinates of each tile are the same coordinates for the same tile in all of the other lists, making getting data about a certain tile efficient.

All of the lists are of the same size, so the coordinates in one list can be used in another, this means that one set of coordinated relates to the same tile in all three lists, which makes the lists easier to handle and makes the program less complicated to program.

```
161 def Initialise():
162     for i in range(10): #Initialise board
163         templist = []
164         temptiles = []
165         temphidden = []
166
167         for j in range(10):
168
169             num = r.randint(1, 5)
170             templist.append('*') if num == 1 else templist.append(' ')
171
172             temphidden.append('+')
173
174             temptiles.append(Tile(True, False, False, '>')) if num == 1 else temptiles.append(Tile(False, False, False, '0'))
175
176         board.append(templist)
177         tiles.append(temptiles)
178         hidden.append(temphidden)
179     ~~~
```

The procedure starts off with running a 'for' loop 10 times (the number which the 'i' and 'j' loop run for relates to how large the grid is, so the size of the grid can be altered by changing those values). Three temporary lists are created and then the 'j' loop runs for 10 times as well. A random number generator is then used to get a random number from 1 to 5 (this is the chance in which a tile is a mine, changing the second number changes how many mines there will be on the board). After this line, an 'if' statement is used to check what number the variable 'num' is after being assigned one from random. If 'num' is a 1 then a '*' symbol, which stands for a mine, is appended to 'templist'. If it is any other number, then a space is appended instead, which stands for an empty tile.

After this on line 172, a cross is appended to 'temphidden', which stands for a covered tile.

The 'if' statement on line 174 checks if 'num' is equal to 1 and if so, then a class instance is appended to 'temptiles' with the data:

- Mine = True
- Flagged = False
- Uncovered = False
- Symbol = '>'

And if 'num' is not equal to 1, then the same happens but instead with:

- Mine = False
- Flagged = False
- Uncovered = False
- Symbol = '0'

The temporary lists are then appended to their corresponding lists:

- 'templist' to 'board'
- 'temptiles' to 'tiles'
- 'temphidden' to 'hidden'

This then repeats for the duration of the lists and then the procedure will end, with all lists now containing all the data about the tiles, excluding the numbering as that is calculated in a different procedure.

CALCNUMBER

A Minesweeper grid not only contains the mines, but also contains lots of numbers, which are the numbers of mines around a certain tile, so if a tile has the number '2', that means there are 2 mines in the eight squares around the tile.

So, in order to calculate these numbers, I created a procedure called 'CalcNumber'. The procedure works by traversing the list until a tile is found which doesn't contain a mine, then the eight squares around it are checked, any mines are added to a running total, and then the total is added to the coordinates of the tile on the list 'Board'.

```
181 def CalcNumber(i):
182     for i in range(len(board)): #Checking for nearby mines
183
184         mineno = 0
185         temp = []
186
187         #Checks if there are mines are around the tile, if so, adds a one to a running total to add to the class attribute
188         for j in range(len(board[i])):
189
190             if tiles[i][j].mine == False:
191
192                 y = i
193                 x = j
194                 mineno = 0
195         return mineno
```

This is the start of the procedure, which includes the 'for' loops to start traversing the list. The running total called 'mineno' is set to zero and a temporary list, 'temp' is created. There is also an 'if' statement which will run the code below if the tile it is on isn't a mine:

```
196         if y-1 != -1: #Up
197             if tiles[y-1][x].mine == True:
198                 mineno += 1
199         else:
200             pass
201
202         if y+1 != len(board): #Down
203             if tiles[y+1][x].mine == True:
204                 mineno += 1
205         else:
206             pass
207
208         if x-1 != -1: #Left
209             if tiles[y][x-1].mine == True:
210                 mineno += 1
211         else:
212             pass
213
214         if x+1 != len(board[i]): #Right
215             if tiles[y][x+1].mine == True:
216                 mineno += 1
217         else:
218             pass
219
220         if x-1 != -1 and y-1 != -1: #Top left
221             if tiles[y-1][x-1].mine == True:
222                 mineno += 1
223         else:
224             pass
225
226         if x+1 != len(board[i]) and y-1 != -1: #Top right
227             if tiles[y-1][x+1].mine == True:
228                 mineno += 1
229         else:
230             pass
231
232         if x-1 != -1 and y+1 != len(board): #Bottom left
233             if tiles[y+1][x-1].mine == True:
234                 mineno += 1
235         else:
236             pass
237
238         if x+1 != len(board[i]) and y+1 != len(board): #Bottom right
239             if tiles[y+1][x+1].mine == True:
240                 mineno += 1
241         else:
242             pass
243
244
```

Here is the code which checks the eight tiles around the empty one, each set of two 'if' statements are for a different position and does this by manipulating the list coordinates. The outer 'if' of each one is to check that if the manipulated coordinates are going to end up being longer than the list, which will crash the program if it wasn't checked, or if the value is going to end up being -1, which wouldn't crash the program but would make the number wrong as it will include the mines from the opposite end of the grid in the running total. The second 'if' checks if the coordinate is a mined tile, and so, 1 is added to the running total.

```
245         if mineno > 0:
246             temp.append(mineno)
247         else:
248             temp.append(' ')
249
250     else:
251         temp.append('*')
252
253
254     board[i] = temp
255
```

Then after the tiles are checked and the total is added to, a final 'if' statement checks if the running total is more than zero. If it is, then the number is appended to 'temp', if it's not, then a space is appended instead. Then the 'else' which is part of the first 'if' at the top of the procedure, appends a '*' if the tile is a mine.

Then finally, 'temp' is appended to the list 'Board'.

CHOICE

In order to have a way for the player to interact with the grid, and to detect a loss, I created another function named 'Choice'.

```
67 def Choice():
68
69     print()
70
71     global firstdig
72
73     coords = input('-')
74     coords = coords.upper()
75     index = coords.index(' ')
76     command = coords[0:index]
77
78     try: #Splits command into parts
79         coords = coords[index+1:]
80     except:
81         return (None, None)
82
83     coords = list(coords)
84
85     x = coords[0] #Splits into x and y
86     try:
87         x = letters.index(x)
88     except:
89         return (None, None)
90
91     try:
92         temp = ''
93         for i in range(1, len(coords)):
94             temp += str(coords[i])
95         temp = int(temp)
96         y = temp
97     except:
98         return (None, None)
```

The function starts by making the variable 'firstdig' a global variable which removes the need for the value to be returned. Then, the player is able to input a command in the format of a command word followed by the coordinate. For example, 'dig f5' and 'flag a0'. Then the input is changed to all uppercase, so the player can't crash the program by using the wrong case. The space in between the command word and coordinate is then indexed and the command is sliced from the input and saved to its own variable called 'command'. After this, on line 78, the original input is sliced so only the set of coordinates remain and is saved to the same variable that the original input was on. This is done using exception handling, so if the user has forgotten to enter a coordinate, the function ends, returning '(None, None)', so the program doesn't crash, it will just repeat the function when the loop goes round again. After this block is run, 'coords' is changed into a list. The second exception handling block is for splitting the coordinates into their individual 'x' and 'y' coordinates. The first character in 'coords' is saved to a variable called 'x' (for the x coordinate). The character is then indexed in the list 'letters' to check if it is a letter of the alphabet, and if so, the numerical index is saved over the other value in the variable 'x'. The third and final exception handling block is used to get the 'y' values. A 'for' loop is set up, starting at 1 (to skip out the letter) and ending at the length of the list. It will then run for that amount of time, adding the string version of the list item to a temporary variable. After the list is run, the temporary variable is turned into an integer and saved to the variable 'y' (for the y coordinate). If the player didn't give any more coordinates after the letter, or there are not numbers in the space where they should be, the 'except' is run, returning '(None, None)', preventing a crash.

The rest of the function is to process the command word and coordinates, which are below:

```
100     if 'DIG' in command:
101         if board[y][x] == ' ':
102             Fill(x, y)
103         else:
104             hidden[y][x] = board[y][x]
105
106     if tiles[y][x].mine == True and firstdig == False:
107         return True
108
109     #If mine is hit on first go
110     elif tiles[y][x].mine == True and firstdig == True:
111         tiles[y][x].mine = False
112         board[y][x] = ' '
113         firstdig = False
114
115
116     for i in range(len(board)):
117         Break = False
118
119         for j in range(len(board[i])):
120
121             if tiles[i][j].mine == True or (j != x and i != y):
122                 pass
123
124             else:
125                 if tiles[i][j].mine == False:
126                     tiles[i][j].mine = True
127                     board[i][j] = '*'
128                     CalcNumber()
129                     Break = True
130                     break
131
132             if Break == True:
133                 break
134
135         hidden[y][x] = board[y][x]
136
137     else:
138         pass
139
140     firstdig = False
141
```

First is the block that runs if the command word is 'DIG', which will clear a tile. If the tile selected by the player is empty, then the procedure 'Fill' is called, which will clear all adjacent empty tiles, and one more layer after that, to reveal the numbers so the player can keep playing. Next, on line 106, the tile is checked if it is a mine, and if 'firstdig' is False, meaning that this isn't the player's first go. If it is a mine, then the function ends, returning a 'True' value. The statement after this checks if the tile is a mine, but instead, checks if 'firstdig' is True. In Minesweeper, there is a rule that if the player hits a mine on their first go, then the mine is relocated to somewhere else on the board. So, if these conditions are met, then the current tile's mine attribute is set to False, and its board appearance is set to a space, and 'firstdig' is set to False. A set of 2 'for' loops are then run which will traverse the list, and as soon as a non-mined tile is found, that tile becomes a mine, and the original mine is turned into a regular tile. The function 'CalcNumber()' is then run to recalculate the numbers on the grid. Then finally, 'firstdig' is set to false as the player will no longer be on their first go.

PRINTBOARD

```
def Printboard(array): #Prints out mine board
    string = ' '
    for i in range(len(array)):
        string += ' ' + letters[i]

    print(string)

    for i in range(len(array)):
        if i < 10:
            string = '0' + str(i)
        else:
            string = str(i)
        for j in range(len(array[i])):
            string += ' ' + str(array[i][j])
        print(string)
```

I then created a procedure for printing the board/ grid to the shell. It starts by creating a top row of letters, which will be used to help the player find the coordinates of a certain tile. It works by traversing through a list of the letters of the alphabet called 'letters' for the length of the mine grid, and adding a space and then the corresponding letter to a string. After the loop has run, the string is printed to the shell before a second 'for' loop is run.

The first loop checks if the counter is less than 10, and if it is then it is converted to a string and a zero is added before it. This is for the vertical coordinates, the zero is because they need to be the same size for single digit numbers and double-digit ones (e.g. 00, 01, 02 ... 09, 10, 11...).

After this, the grid list is traversed. Using the 'i' and 'j' counters, the list item for the corresponding 'x' and 'y' coordinate is found. The number string from earlier is added before this, and then all of the list items on the 'y' level are added to it. It is then printed to the shell to display the grid, so it looks like this:

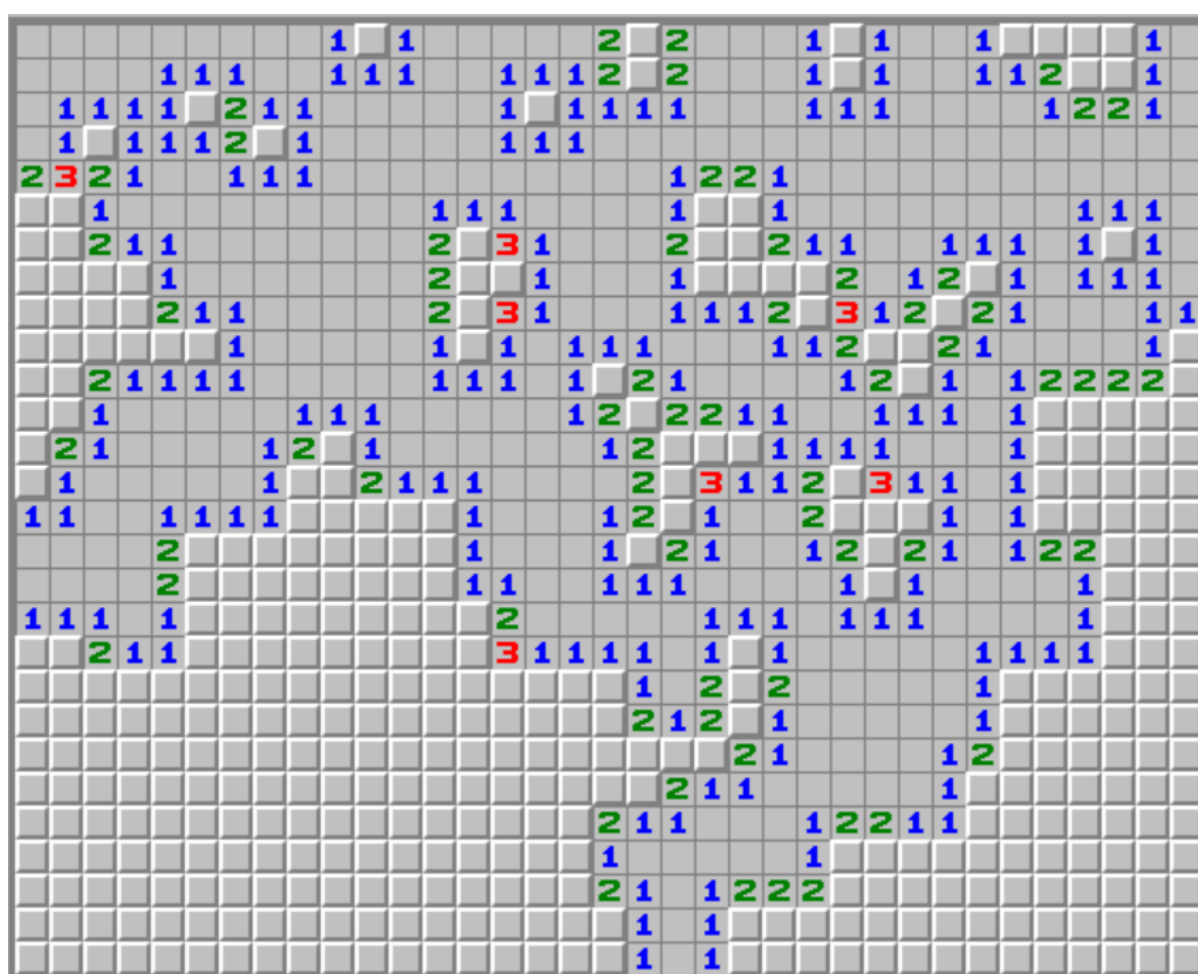
```
  A B C D E F G H I J
00 + + + + + + + + +
01 + + + + + + + + +
02 + + + + + + + + +
03 + + + + + + + + +
04 + + + + + + + + +
05 + + + + + + + + +
06 + + + + + + + + +
07 + + + + + + + + +
08 + + + + + + + + +
09 + + + + + + + + +
```

FILL

In Minesweeper, if the player clears an empty tile, a non-mined tile with no numbers in it, then all adjacent empty tiles are also cleared. These tiles are cleared in all directions:

- Up
- Down
- Left
- Right
- Top and bottom corners

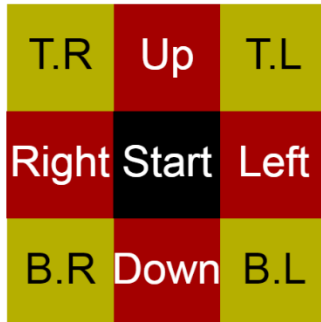
The tiles are cleared in such a way so that a 'ring' of numbers is also cleared, as the player can't find the mines without the numbers. An example of this is shown below using Minesweeper X:



Here you can see the empty tiles and the 'rings' of numbers. All of these empty tiles are touching each other, so they have cleared in one press. Mines can never be in this ring, as there always need to be numbers around one, so the numbers would be shown instead of the mine.

So, in order to for the player to do this on my game, I created an area filling algorithm, to find all empty tiles and then be able to clear them all on one go. The algorithm works by moving outwards from the starting position in a ring, checking that the points it has expanded to are empty tiles, and then expanding in another

ring from the points which are empty tiles. The coordinates of the empty tiles which are yet to be expanded from are appended to a list called 'ActivePoints'. And the coordinates of the empty tiles which have been expanded from are appended to a list called 'CompletedPoints'. This repeats until 'ActivePoints' is empty. Then as a final expansion. All points are expanded from to add the ring of numbers. (All points are expanded from rather than just the outer ones because it would be complicated to code and would just do the exact same thing, so there's no point in doing it)



Here is a simple visualisation of the area filling algorithm. It shows the start point, and how it expands outwards in all directions. The red squares show the up, down, left and right directions, and the yellow/gold squares represent the top and bottom corners. All of these squares that were empty tiles would be appended to 'ActivePoints' and would be expanded from like this until the list is empty.

```
def Fill(x, y):
    ActivePoints = []
    CompletedPoints = []

    Y = y
    X = x

    ActivePoints.append([Y, X])

    # FILLS IN EMPTY TILES
    while len(ActivePoints) != 0:
        node = ActivePoints[0]

        y = node[0]
        x = node[1] #Up, down, left, right

        if y + 1 != len(board) and board[y + 1][x] == ' ' and ([y + 1, x] not in CompletedPoints and [y + 1, x] not in ActivePoints):
            ActivePoints.append([y + 1, x])

        if y - 1 != -1 and board[y - 1][x] == ' ' and ([y - 1, x] not in CompletedPoints and [y - 1, x] not in ActivePoints):
            ActivePoints.append([y - 1, x])

        if x + 1 != len(board[y]) and board[y][x + 1] == ' ' and ([y, x + 1] not in CompletedPoints and [y, x + 1] not in ActivePoints):
            ActivePoints.append([y, x + 1])

        if x - 1 != -1 and board[y][x - 1] == ' ' and ([y, x - 1] not in CompletedPoints and [y, x - 1] not in ActivePoints):
            ActivePoints.append([y, x - 1])
```

The algorithm is programmed as a procedure. The algorithm begins by creating the 'ActivePoints' and 'CompletedPoints' lists. Then an 'X' and 'Y' variable store the parameters which are the coordinates of the tile the player has 'dug'. Then the starting tile is appended to 'ActivePoints' and a 'while' loop is run on the conditions that it is run while 'ActivePoints' is not empty. The tile at the front of 'ActivePoints' is copied to a variable called 'node' and then the coordinate data in the node is saved to an 'x' and 'y' variable. Then there is a block of four 'if' statements, these ones expand the area in the four directions: up, down, left, right. The statement has a long Boolean expression, but it checks: whether the tile above the starting one is actually a tile on the grid, and if it is an empty tile, and if it is not in 'CompletedPoints' and also not in 'ActivePoints'. If all of these conditions are true, then the coordinates of the selected tile are appended to 'ActivePoints'. All of the following 'if' statements use the same conditions, but the position changes from up to down to left to right.


```
#Corners#
if y + 1 != len(board) and x - 1 != -1 and board[y + 1][x - 1] == ' ' and ([y + 1, x - 1] not in CompletedPoints and [y + 1, x - 1] not in ActivePoints):
    ActivePoints.append([y + 1, x - 1])
if y + 1 != len(board) and x + 1 != len(board[y]) and board[y + 1][x + 1] == ' ' and ([y + 1, x + 1] not in CompletedPoints and [y + 1, x + 1] not in ActivePoints):
    ActivePoints.append([y + 1, x + 1])
if y - 1 != -1 and x - 1 != -1 and board[y - 1][x - 1] == ' ' and ([y - 1, x - 1] not in CompletedPoints and [y - 1, x - 1] not in ActivePoints):
    ActivePoints.append([y - 1, x - 1])
if y - 1 != -1 and x + 1 != len(board[y]) and board[y - 1][x + 1] == ' ' and ([y - 1, x + 1] not in CompletedPoints and [y - 1, x + 1] not in ActivePoints):
    ActivePoints.append([y - 1, x + 1])

CompletedPoints.append(ActivePoints[0])
del ActivePoints[0]
```

After this block of code has run, another block of four 'if' statements are run, but these ones expand it diagonally, to get the top and bottom corners of the ring. The Boolean expressions are very similar, but these ones have an extra condition that checks if the left or right of the tile is actually in the grid, and an empty tile. To better understand the expansion, there is a diagram below.

After this block has run. The starting point is appended to 'CompletedPoints' and deleted from 'ActivePoints'. This way, on the next loop, a different point will be the starting point. The loop will then continue this until all active points have been completed, which will mean that 'ActivePoints' is empty, stopping the loop.

```
#          FILLS IN TILES ADJACENT TO THE EMPTY TILES

temp = []

for i in range(len(CompletedPoints)):
    coord = CompletedPoints[i]
    x = coord[1]
    y = coord[0]          #Up, down, left, right#

    if y + 1 != len(board) and board[y + 1][x] != ' ' and [y + 1, x] not in temp:
        temp.append([y + 1, x])

    if y - 1 != -1 and board[y - 1][x] != ' ' and [y - 1, x] not in temp:
        temp.append([y - 1, x])

    if x + 1 != len(board[y]) and board[y][x + 1] != ' ' and [y, x + 1] not in temp:
        temp.append([y, x + 1])

    if x - 1 != -1 and board[y][x - 1] != ' ' and [y, x - 1] not in temp:
        temp.append([y, x - 1])

    #Corners#

    if y + 1 != len(board) and x - 1 != -1 and board[y + 1][x - 1] != ' ' and [y + 1, x - 1] not in temp:
        temp.append([y + 1, x - 1])

    if y + 1 != len(board) and x + 1 != len(board[y]) and board[y + 1][x + 1] != ' ' and [y + 1, x + 1] not in temp:
        temp.append([y + 1, x + 1])

    if y - 1 != -1 and x - 1 != -1 and board[y - 1][x - 1] != ' ' and [y - 1, x - 1] not in temp:
        temp.append([y - 1, x - 1])

    if y - 1 != -1 and x + 1 != len(board[y]) and board[y - 1][x + 1] != ' ' and [y - 1, x + 1] not in temp:
        temp.append([y - 1, x + 1])

#          REPLACES VALUES TO FILL IT IN

for i in range(len(temp)):
    CompletedPoints.append(temp[i])

temp = CompletedPoints

while len(temp) != 0:
    node = temp[0]

    y = node[0]
    x = node[1]

    hidden[y][x] = board[y][x]

    del temp[0]
```

After the 'while' loop has run, temporary list is created and a 'for' loop is run which runs for the length of 'CompletedPoints'. In the loop there are again, two blocks of 4 'if' statements. These blocks are also very similar to the ones in the 'while' loop and the top are for up, down, left and right, with the bottom one for the top and bottom corners. But the main difference is that this loop adds an extra ring around the area, so that the numbers are visible, the reason for which being explained above. The conditions are: if whatever direction it is in is a grid point, and the tile is not going to be an empty one, and the tile is not already in the temporary list. If these are all true, then the coordinates of the new tile are appended to the temporary list. This then repeats until the 'for' loop ends, and then the temporary list is appended to 'CompletedPoints' after it has finished.

Then, a final loop is run, which changes the characters in 'hidden' to the ones in 'board' for the coordinates of all the tiles cleared during the area filling algorithm being run.

So, the grid will go from this (left) to this (right), in one command:

```

      A B C D E F G H I J
00 + + + + + + + + +
01 + + + + + + + + +
02 + + + + + + + + +
03 + + + + + + + + +
04 + + + + + + + + +
05 + + + + + + + + +
06 + + + + + + + + +
07 + + + + + + + + +
08 + + + + + + + + +
09 + + + + + + + + +

```

```

      A B C D E F G H I J
00 + + + + 1      1 + +
01 + + + + 2 1    2 + +
02 + + + + + 1    2 + +
03 1 1 1 1 1 1    1 4 +
04      2 +
05      1 +
06      1 1 1    1 +
07 1 1      1 + 1 1 2 +
08 + 1 1 1 3 + + + + +
09 + + + + + + + + +

```

CHECKWIN

The final function is a very small one called 'Checkwin', which is made of a 'for' loop that counts how many mines have been flagged, and an 'if' statement that checks if all mines have been flagged. If so, the function returns True. This breaks the game loop and the player wins the game.

TESTING

For this prototype, all of the testing will involve either the text input from the player, or how the characters interact with each other.

TEST PLAN

Test no.	Test description	Test type	Data	Expected result
1a	Dig: DIG command clears a tile	Normal	'dig a5'	Clears successfully
1b	Dig: DIG command on empty tile clears all adjacent tiles, and an extra layer to include the numbers	Normal	'dig' on any empty tile	Clears all successfully
1c	Dig: DIG command works on a corner	Boundary	'dig a0'	Clears successfully
1d	Dig: DIG a flagged tile	Boundary	'dig' on any flagged tile	Nothing happens
2a	Grid: Program prevents user from entering an incorrect command word	Erroneous	'cheese a5'	Print 'Input Error' and nothing happens besides that
2b	Grid: Program prevents user from entering incorrect x coordinate	Erroneous	'dig z5'	Print 'Input Error' and nothing happens besides that
2c	Grid: Program prevents user from entering an incorrect y coordinate	Erroneous	'dig a43'	Print 'Input Error' and nothing happens besides that
2d	Grid: Program prevents user entering a negative y coordinate	Erroneous	'dig a-1'	Print 'Input Error' and nothing happens besides that
3a	Mine: Player loses upon hitting a mine not on the player's first go	Normal	'dig' on a mined tile after already clearing a tile	Nothing happens
3b	Mine: Mine is relocated upon hitting a mine on the player's first go	Normal	'dig' on a mined tile on the first go	Mine is relocated and game continues
4a	Flag: 'flag' an already flagged tile	Normal	'flag' on any flagged tile	Flag is removed and becomes a '+'

4b	Flag: 'flag' a cleared tile	Boundary/ Erroneous	'flag' any cleared tile	Nothing happens
4c	Flag: 'flag' a questioned tile	Normal	'flag' any questioned tile	Tile becomes a flag
5a	Question: 'question' an already questioned tile	Normal/ Boundary	'question' on any 'questioned' tile	Question mark is removed and becomes a '+'
5b	Question: 'question' a cleared tile	Boundary/ Erroneous	'question' any cleared tile	Nothing happens
5c	Question: 'question' a flagged tile	Normal	'question' any flagged tile	Tile becomes a question mark

TEST RESULTS (TABLE)

Test no.	Actual result	Action & Reason
1a	Expected result	N/A
1b	Expected result	N/A
1c	Expected result	N/A
1d	Tile is cleared	None: Will be fixed in next version & not common enough to be a major issue
2a	Expected result	N/A
2b	Program crashes	None: This type of issue will not be encountered in future versions
2c	Program crashes	None: This type of issue will not be encountered in future versions
2d	Tile on corresponding index is cleared	None: Not common enough to be a major issue
3a	Expected result	N/A
3b	Tile is cleared and game is not lost, however, the tile symbol is the mine symbol	None: Major issue but playable without fixing. It will take a while to fix, so will be addressed in the next prototype
4a	Expected result	N/A
4b	Flag icon appears on tile	Fix: Add 'if' statement to check if tile is not a cross, if it is not a cross, then the tile will not be flagged
4c	Expected result	N/A
5a	Question mark does not disappear	Fix: 'if' statement that if the tile is questioned and the command is 'QUESTION', then the tile is turned to a cross

5b	Question mark appears on tile	Fix: same as action for 4b , except it will not be questioned
5c	Expected result	N/A

TEST RESULTS (IMAGES)

Below are screenshots of the code running which shows examples of all test results:

```
-flag a0
  A B C D E F G H I J
00 > + + + + + + + +
01 + + + + + + + +
02 + + + + + + + +
03 + + + + + + + +
04 + + + + + + + +
05 + + + + + + + +
06 + + + + + + + +
07 + + + + + + + +
08 + + + + + + + +
09 + + + + + + + +
```

```
-dig a0
  A B C D E F G H I J
00
01 1 1 1 1 1 2 1 1
02 1 + 1 1 2 + + + +
03 2 + + + + + + + +
```

Tests: 1a 1b 1c 1d

```
54  A B C D E F G H I J
00 + + + + + + + +
01 + + + + + + + +
02 + + + + + + + +
03 + + + + + + + +
04 + + + + + + + +
05 + + + + + + + +
06 + + + + + + + +
07 + + + + + + + +
08 + + + + + + + +
09 + + + + + + + +
```

```
-cheese a5
Input_error
```

Tests: 2a

```
-dig z5
Traceback (most recent call last):
  File "C:\Ciaran\Python - College W
, line 379, in <module>
    lose = Choice()
  File "C:\Ciaran\Python - College W
, line 101, in Choice
    if board[y][x] == ' ':
IndexError: list index out of range
```

Tests: 2b

```
-dig a-1
  A B C D E F G H I J
00 + + + + + + + +
01 + + + + + + + +
02 + + + + + + + +
03 + + + + + + + +
04 + + + + + + + +
05 + + + + + + + +
06 + + + + + + + +
07 + + + + + + + +
08 + + + + + + + +
09 2 + + + + + + + +
```

Tests: 2d

```
-dig g5
  A B C D E F G H I J
00 + + + + + + + +
01 + + + + + + + +
02 + + + + + + + +
03 + + + + + + + +
04 + + + + + + + +
05 + + 1 1 1 2 * + + +
06 + + 1 1 1 + + +
07 + + 1 1 1 + + +
08 + + 1 2 + + +
09 2 + 1 2 + + +
Mine Hit!
```

Tests: 3a

```
-dig a0
  A B C D E F G H I J
00 * + + + + + + +
01 + + + + + + + +
02 + + + + + + + +
03 + + + + + + + +
04 + + + + + + + +
05 + + + + + + + +
06 + + + + + + + +
07 + + + + + + + +
08 + + + + + + + +
09 + + + + + + + +
```

Tests: 3b

```
-dig a43
Traceback (most recent call last):
  File "C:\Ciaran\Python - College W
, line 379, in <module>
    lose = Choice()
  File "C:\Ciaran\Python - College W
, line 101, in Choice
    if board[y][x] == ' ':
IndexError: list index out of range
```

Tests: 2c

```
-flag a0
  A B C D E F G H I J
00 + + + + + + + +
01 + + + + + + + +
02 + + + + + + + +
03 + + + + + + + +
04 + + + + + + + +
05 + + + + + + + +
06 + + + + + + + +
07 + + + + + + + +
08 + + + + + + + +
09 + + + + + + + +
```

Tests: 4a

```

  A B C D E F G H I J
00 + + + + 1 1 + + +
01 + + + + 1 1 1 2 +
02 + + + 3 1 1 +
03 + 1 1 1 1 2 +
04 + 1 1 + +
05 + 1 1 1 1
06 + 1
07 + 2 1
08 + + 2 2 1 2 1 1
09 + + + + + + 1

-flag e5
  A B C D E F G H I J
00 + + + + 1 1 + + +
01 + + + + 1 1 1 2 +
02 + + + 3 1 1 +
03 + 1 1 1 1 2 +
04 + 1 1 + +
05 + 1 > 1 1 1
06 + 1
07 + 2 1
08 + + 2 2 1 2 1 1
09 + + + + + + 1

```

Tests: 4b

```

  A B C D E F G H I J
00 + + + + 1 1 + + +
01 + + + + 1 1 1 2 +
02 + + + 3 1 1 +
03 + 1 1 1 1 2 +
04 + 1 1 + +
05 + 1 ? 1 1 1
06 + 1
07 + 2 1
08 + + 2 2 1 2 1 1
09 + + + + + + 1

-flag e5
  A B C D E F G H I J
00 + + + + 1 1 + + +
01 + + + + 1 1 1 2 +
02 + + + 3 1 1 +
03 + 1 1 1 1 2 +
04 + 1 1 + +
05 + 1 > 1 1 1
06 + 1
07 + 2 1
08 + + 2 2 1 2 1 1
09 + + + + + + 1

```

Tests: 4c

```

  A B C D E F G H I J
00 + + + + 1 1 + + +
01 + + + + 1 1 1 2 +
02 + + + 3 1 1 +
03 + 1 1 1 1 2 +
04 + 1 1 + +
05 + 1 ? 1 1 1
06 + 1
07 + 2 1
08 + + 2 2 1 2 1 1
09 + + + + + + 1

-question e5
  A B C D E F G H I J
00 + + + + 1 1 + + +
01 + + + + 1 1 1 2 +
02 + + + 3 1 1 +
03 + 1 1 1 1 2 +
04 + 1 1 + +
05 + 1 ? 1 1 1
06 + 1
07 + 2 1
08 + + 2 2 1 2 1 1
09 + + + + + + 1

```

Tests: 5a

```

  A B C D E F G H I J
00 + + + + 1 1 + + +
01 + + + + 1 1 1 2 +
02 + + + 3 1 1 +
03 + 1 1 1 1 2 +
04 + 1 1 + +
05 + 1 1 1 1
06 + 1
07 + 2 1
08 + + 2 2 1 2 1 1
09 + + + + + + 1

-question e5
  A B C D E F G H I J
00 + + + + 1 1 + + +
01 + + + + 1 1 1 2 +
02 + + + 3 1 1 +
03 + 1 1 1 1 2 +
04 + 1 1 + +
05 + 1 ? 1 1 1
06 + 1
07 + 2 1
08 + + 2 2 1 2 1 1
09 + + + + + + 1

```

Tests: 5b

```

  A B C D E F G H I J
00 + + + + 1 1 + + +
01 + + + + 1 1 1 2 +
02 + + + 3 1 1 +
03 + 1 1 1 1 2 +
04 + 1 1 + +
05 + 1 > 1 1 1
06 + 1
07 + 2 1
08 + + 2 2 1 2 1 1
09 + + + + + + 1

-question e5
  A B C D E F G H I J
00 + + + + 1 1 + + +
01 + + + + 1 1 1 2 +
02 + + + 3 1 1 +
03 + 1 1 1 1 2 +
04 + 1 1 + +
05 + 1 ? 1 1 1
06 + 1
07 + 2 1
08 + + 2 2 1 2 1 1
09 + + + + + + 1

```

Tests: 5c

FIXED ERRORS

I've chosen three errors found from testing to fix from prototype one. These are:

- Test **4b** – Flag can be placed on an empty tile when it shouldn't be.
- Test **5a** – If the question command is used on a questioned tile, the tile should go back to a cross, but nothing happens.
- Test **5b** – A question mark can be placed on an empty tile when it shouldn't be.

The fix for each problem is below:

- Test **4b** – An 'if' statement that checks if the tile the player has selected is empty, and if so, nothing happens.
- Test **5a** – An 'if' statement that checks if the tile the player has selected is empty, and if so, the tile goes back to a cross.
- Test **5b** – An 'if' statement that checks if the tile the player has selected is empty, and if so, nothing happens.

FIXED ERRORS (IMAGES)

```

A B C D E F G H I J
00 + + + + 1      1 +
01 + + + + 1      1 +
02 + + + + 3 1 1  1 1
03 + + + + + 1
04 + + + 2 2 1 1
05 + + + 1
06 + + + 1 1 1
07 + + + + + 2 1 1
08 + + + + + + 1
09 + + + + + + 1

-question g0
A B C D E F G H I J
00 + + + + 1      1 +
01 + + + + 1      1 +
02 + + + + 3 1 1  1 1
03 + + + + + 1
04 + + + 2 2 1 1
05 + + + 1
06 + + + 1 1 1
07 + + + + + 2 1 1
08 + + + + + + 1
09 + + + + + + 1
          
```

Tests: **4b 5b**

```

A B C D E F G H I J
00 + + + ? 1      1 +
01 + + + + 1      1 +
02 + + + + 3 1 1  1 1
03 + + + + + 1
04 + + + 2 2 1 1
05 + + + 1
06 + + + 1 1 1
07 + + + + + 2 1 1
08 + + + + + + 1
09 + + + + + + 1

-question d0
A B C D E F G H I J
00 + + + + 1      1 +
01 + + + + 1      1 +
02 + + + + 3 1 1  1 1
03 + + + + + 1
04 + + + 2 2 1 1
05 + + + 1
06 + + + 1 1 1
07 + + + + + 2 1 1
08 + + + + + + 1
09 + + + + + + 1
          
```

Test: **5a**

As shown by the difference between the above images and the test results, the errors have been fixed and tests now a success.

CONCLUSION

In conclusion of the development and testing of prototype one, the program is a successful proof of concept for showing the essential rules of Minesweeper can be made using Python. Although there are several errors within the code, the game is still playable and for most errors, these will not be encountered by a sensible player, with the other errors being very rare to be a common occurrence. However, all of the major issues will be addressed in the next prototype, with some not being encountered again due to game switching from text-based to Pygame. I have chosen to use Pygame as opposed to GUIZero because there is greater control over where items go on screen, and I prefer the way Pygame allows me to do this.

PROTOTYPE TWO – MINESWEEPER IN PYGAME

INTRODUCTION

For Prototype Two, I have created a new program from scratch, using the things I learnt from the first prototype, to create a version of the game using the PyGame module.

PROGRAM STRUCTURE

MAIN DATA & WHERE IT IS STORED

The program uses a lot of different pieces of data about each tile, and these are stored across multiple data structures. These are:

- **Tile**
A class this stores all the data about a tile that could be required. There is one instance of this class for every tile and are stored in **objects**.
- **objects**
A 2D list where all the instances of **Tile** are stored.
- **imagegrid**
A 2D list the same size as **objects** which stores the file path of each tiles' assigned image. Similar to the first prototype, the 2D lists are the same size so they can access the same tile across multiple lists using the same index.
- **FQgrid**
Stands for 'flag & question grid'. It is again a 2D list but stores the file paths of the images of tiles that are flagged or questioned.
- **GridImages**
The dictionary that stores the file paths for the image files for each possible tile state.

DISPLAY ORDER

As there are three different grids of images being displayed to the screen, there needs to be an order they are displayed in to ensure they are not out of order. The order is as follows:

- **Numbers**
The grid of numbers is displayed first as it's the stuff you see after clearing the top, and there's nothing under it.

- **Covered Tiles**

This is then followed by the covered tiles, which are removed when the player interacts with them.

- **Flags and Questions**

Lastly, the FQgrid goes on top, which displays the flags and questions when the player interacts with the covered tiles.

CLASSES, FUNCTIONS/ PROCEDURES & DATA STRUCTURES

'TILE' CLASS

Similar to prototype one, there is also the main class called 'Tile' which stores most of the important data about each tile. There is one instance for each tile in the grid, and these are stored in a 2D list called 'objects'.

```
class Tile():
    def __init__(self, Mine, Number, Size, Position):
        self.mine = Mine
        self.flagged = False
        self.questioned = False
        self.cleared = False
        self.number = Number #Empty
        self.size = Size
        self.position = Position

        #self.Display = pygame.Surface((Size, Size))
        self.Display = pygame.image.load(GridImages['C'])
        self.Display = pygame.transform.scale(self.Display, (Size, Size))
```

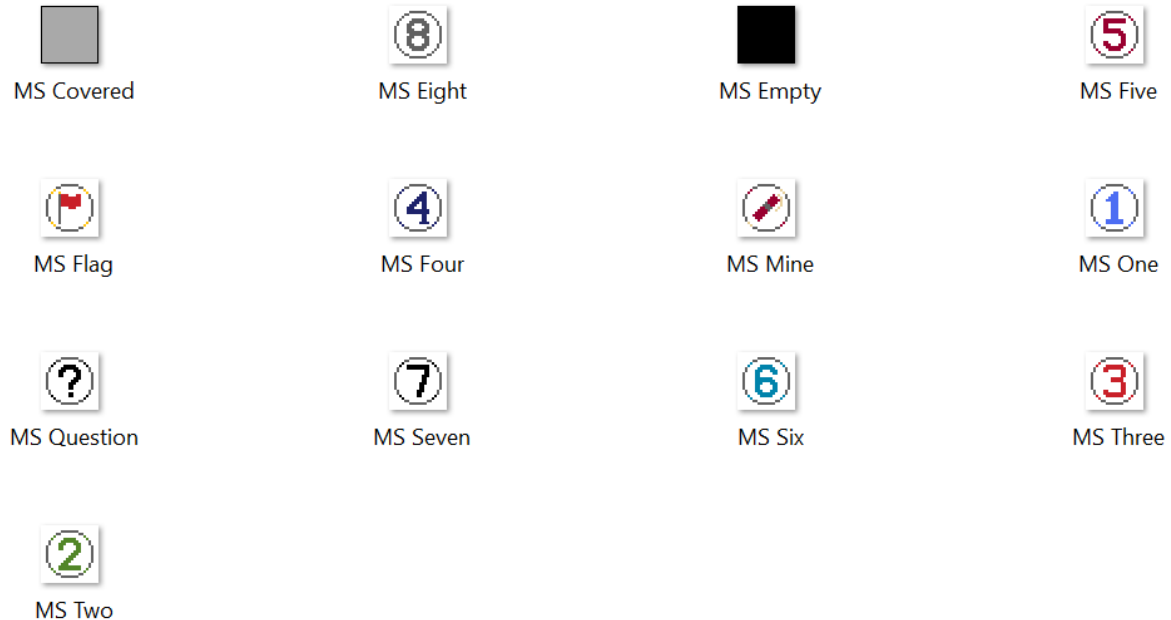
This is the code which defines the class and its attributes. There are some attributes that are defined upon a new instance being created, and some which are hardcoded into it. The new class includes some of the data from the previous one, such as 'Mine' and 'Number'. There are also 'flagged' and 'questioned', but these have a set value upon creation of an instance.

There are also a few new ones, most importantly there are 'position' and 'Display', which record the pixel coordinates of the tile and the corresponding image.

There are no methods in this class instead, the various functions and procedures call the data from the class.

GRIDIMAGES

'GridImages' is a dictionary that stores the images for the tile displays, which are stored as the 'Display' attribute in the 'Tile' class. Below are all of the images that I created using a pixel art software, which are stored in this dictionary.



It contains all of the numbers, and the flag, question mark, and mine images. 'MS Covered' is the image for when the tiles haven't been cleared yet, and 'MS Empty' is a transparent image which is for tiles that have no number. Below is the code for the dictionary.

```
GridImages = {
    1: 'Grid Images\MS One.png',
    2: 'Grid Images\MS Two.png',
    3: 'Grid Images\MS Three.png',
    4: 'Grid Images\MS Four.png',
    5: 'Grid Images\MS Five.png',
    6: 'Grid Images\MS Six.png',
    7: 'Grid Images\MS Seven.png',
    8: 'Grid Images\MS Eight.png',
    'M': 'Grid Images\MS Mine.png',
    'F': 'Grid Images\MS Flag.png',
    'Q': 'Grid Images\MS Question.png',
    'E': 'Grid Images\MS Empty.png',
    'C': 'Grid Images\MS Covered.png'
}
```

Here, you can see the label for each item, which is the path for each image. I do not need to enter the entire path, as the program and image folder are stored in the same parent folder.

CREATEGRID

Similar to 'Initialise', this procedure creates the 'objects' list, and determines which tiles are mines and not based off of a percentage chance. Below is the start of the procedure.

```
def creategrid():  
  
    global Gheight #Stands for Grid height and Grid width  
    global Gwidth  
  
    Gheight = 28 #int(input('height: ')) #MAX 28  
    Gwidth = 35 #int(input('width: ')) #MAX 35  
    chance = 12 #Percentage chance of a tile being a mine  
  
    temp = []  
    areaWidth = 890 #35  
    areaHeight = 700 #28  
    centre = (445, 350)  
    size = 25  
  
    if Gheight > 28 or Gwidth > 35: #If inputted grid size is too big  
        print('too many tiles')  
        0/0  
  
    startY = 400 - ((Gheight / 2) * size)  
    startX = 495 - ((Gwidth / 2) * size)  
  
    y = startY  
    x = startX
```

It starts with multiple variables being defined and assigned a value.

Variable Name	Purpose/ What is stored
Gheight	Stores the height of the current grid in tiles. Can be changed.
Gwidth	Stores the width of the current grid in tiles. Can be changed.
Chance	The percentage chance of a tile being a mine.
Temp	Temporary list, to help append to 'objects' or any other list.
areaWidth	The width of the playable grid in pixels. Gwidth cannot exceed 35 because of this value.
areaHeight	The height of the playable grid in pixels. Gheight cannot exceed 28 because of this value.

centre	The centre of the playable area.
size	The size of each tile (25x25 pixels). Cannot be changed.
startX	x-coordinate of the starting point of the grid. Used to create the grid and ensure that it is centred on the screen.
startY	y-coordinate of the starting point of the grid. Used to create the grid and ensure that it is centred on the screen.

This is then followed by the code below. The two 'for' loops are for creating the 2D 'objects' list. A temporary list is created, and then the second loop is run for width of the grid. A random number generator is used, and if the value is less than or equal to the chance, then a new instance of 'Tile' is created, with it being a mine, if not then the same thing happens but it isn't a mine. This is then appended to the temporary list, and then appended to 'objects'.

```

for i in range(Gheight):
    temp = []
    for j in range(Gwidth): #Creates tiles and whether theyre a mine or not, and gives them their size
        if random.randint(1, 100) <= chance: #changed from (1, chance) == 1
            temp.append(Tile(True, 0, size, (x, y)))
        else:
            temp.append(Tile(False, 0, size, (x, y)))

        x += size
    objects.append(temp)

    y += size
    x = startX

## for i in range(len(objects)):
##     for j in range(len(objects[i])):
##         objects[i][j].Display.fill('Dark Grey')

createimagegrid(Gwidth, Gheight, size) #adds the dark grey square to give the appearance of a covered tile.
minedistance() #calculates mine numbers

```

After the loops have run, 'createimagegrid' and 'minedistance' are called.

CREATEIMAGEGRID

This is a procedure to create the 'imagegrid' list.

```
def createimagegrid(w, h, s):#ONLY ADDS MINE IMAGES, NUMBERS ARE ADDED IN 'minedistance'

    for i in range(h):
        temp = []
        for j in range(w): #if tile is mine, append mine image, if not, append empty
            if objects[i][j].mine == True:
                temp.append(pygame.image.load(GridImages['M']))
            else:
                temp.append(pygame.image.load(GridImages['E']))

        imagegrid.append(temp)

    for i in range(len(imagegrid)): #Resizes all images in list
        for j in range(len(imagegrid[i])):
            imagegrid[i][j] = pygame.transform.scale(imagegrid[i][j], (25, 25))

    #Creates FQgrid, starts off with a grid of all empty images ready for flags
    for i in range(len(imagegrid)):
        temp = []

        for j in range(len(imagegrid[i])):
            temp.append(pygame.image.load(GridImages['E'])) #Loads empty image
            pygame.transform.scale(imagegrid[i][j], (25, 25)) #Scales it to size

        FQgrid.append(temp) #Adds temporary list to FQgrid
```

It is only a small procedure, as it is used to set up the list. The mine images are added to the mines, and the rest are assigned the empty image.

The first set of 'for' loops create the list and append one of the two images per index. The second set scale each image to the required 25x25 pixels.

And the third set is for the FQgrid, which is set up with all empty images scaled to the correct size.

MINEDISTANCE

This procedure is almost identical to CalcNumber from Prototype One, the main difference is that the 'tiles' and 'board' lists have been changed to 'objects' and 'imagegrid'.

```
def minedistance():

    #Sets all numbers to 0 in case its run after grid creation
    for i in range(len(objects)):

        for j in range(len(objects[i])):
            objects[i][j].number = 0

    for i in range(len(objects)):

        for j in range(len(objects[i])):

            #font = pygame.font.SysFont('Courier New', 20) #Loading Screen#
            #loadScreen = font.render('LOADING', False, (0, 0, 0))
            #screen.blit(loadScreen, (100, 100))

            x = objects[i][j].position[0]
            y = objects[i][j].position[1]

            if objects[i][j].mine == False:
                #Up and down
                if i - 1 != -1 and objects[i-1][j].mine == True:
                    objects[i][j].number += 1

                if i + 1 != len(objects) and objects[i+1][j].mine == True:
                    objects[i][j].number += 1

                #Left and right
                if j - 1 != -1 and objects[i][j-1].mine == True:
                    objects[i][j].number += 1

                if j + 1 != len(objects[i]) and objects[i][j+1].mine == True:
                    objects[i][j].number += 1

                #Top left and right
                if j - 1 != -1 and i - 1 != -1 and objects[i-1][j-1].mine == True:
                    objects[i][j].number += 1

                if j + 1 != len(objects[i]) and i - 1 != -1 and objects[i-1][j+1].mine == True:
                    objects[i][j].number += 1

                #Bottom left and right
                if j - 1 != -1 and i + 1 != len(objects) and objects[i+1][j-1].mine == True:
                    objects[i][j].number += 1

                if j + 1 != len(objects[i]) and i + 1 != len(objects) and objects[i+1][j+1].mine == True:
                    objects[i][j].number += 1
```

This section focuses on calculating the number of mines in a ring around the tile, using the number attribute of each instance as a counter for the total number of mines.

```
#Assigns number images to the tiles
for i in range(len(objects)):

    for j in range(len(objects[i])):

        #font = pygame.font.SysFont('Courier New', 20) #Loading Screen#
        #loadScreen = font.render('LOADING', False, (0, 0, 0))
        #screen.blit(loadScreen, (100, 100))

        if objects[i][j].number == 0 and objects[i][j].mine == False: #Append to temp lists and resizes
            imagegrid[i][j] = pygame.image.load(GridImages['E'])
        elif objects[i][j].number != 0 and objects[i][j].mine == False:
            imagegrid[i][j] = pygame.image.load(GridImages[objects[i][j].number]) #appends image to list

        imagegrid[i][j] = pygame.transform.scale(imagegrid[i][j], (25, 25))

        if objects[i][j].mine == True:
            objects[i][j].number = -1 #Sets all mine numbers to -1 to avoid getting cleared in area fill
```

This part is new to the procedure from Prototype One and is used to add the number images to 'imagegrid'. It consists of a loop of 'if' statements that check if the tile is a zero and not mine, then it gets an empty image applied to it. If the number isn't a zero and not a mine, then the corresponding number image is applied.

Lastly, all mine numbers are changed to -1 to avoid being cleared during the area filling algorithm in 'ClearEmpty' being run.

CHECKWIN

CheckWin is a new function made to check whether the player has won or not. In Minesweeper, there are two ways of winning, you can either flag all of the mines or clear all of the tiles which aren't mines. This procedure will check for both.

```
def CheckWin():
    mineCount = 0 #Any mine tile
    flagCount = 0 #Flagged mine tile

    #Checks if all mines are flagged
    for i in range(len(objects)):
        for j in range(len(objects[i])):
            if objects[i][j].mine == True:
                mineCount += 1

            if objects[i][j].mine == True and objects[i][j].flagged == True:
                flagCount += 1

    if mineCount == flagCount:
        return True

    mineCount = 0 #Reuse mineCount
    clrdCount = 0 #Cleared non-mine tiles

    #Checks if the only remaining uncleared tiles are mines
    for i in range(len(objects)):
        for j in range(len(objects[i])):
            if objects[i][j].mine == False and objects[i][j].cleared == True:
                clrdCount += 1

            if objects[i][j].mine == True and objects[i][j].cleared == False:
                mineCount += 1

    '''Uses grid height and width to get grid area, take away number of cleared tiles,
    if the remainder is the same as the number of mined tiles, the player wins'''

    Garea = Gheight * Gwidth #Stands for Grid area

    Garea -= clrdCount

    if Garea == mineCount:
        return True
```

It starts with 'mineCount' and 'flagCount' being created. These are then used in the following two 'for' loops which traverse the grid, counting all of the mines, and mines that are flagged. If these are equal to each other, then the function returns true, and the player wins.

The second set of loops is to check whether there are only mines left uncleared. The variable 'mineCount' is reused and a new one called 'clrdCount' (cleared count) is created. The loops count the total number of mines and cleared tiles and save them to their corresponding variables. Then, the grid area is calculated and saved to 'Garea'. The total number of cleared tiles is subtracted from the area, to give the number of covered tiles still remaining in the grid. If the total number of remaining tiles equal the number of mines in the grid, then the player has cleared all non-mined tiles and the function returns true, meaning the player has won.

CLEAREMPTY

Called 'Fill' in Prototype One, this is essentially the exact same procedure, except the data is coming from 'objects' and not 'board'. The main part of the procedure where the adjacent empty tiles are identified and changed to cleared remains almost the exact same, minus the small change mentioned.

```
for i in range(len(temp)):
    CompletedPoints.append(temp[i])

temp = CompletedPoints

while len(temp) != 0:
    node = temp[0]

    y = node[0]
    x = node[1]

    objects[y][x].cleared = True
    objects[y][x].Display = pygame.image.load(GridImages['E'])
    objects[y][x].Display = pygame.transform.scale(objects[y][x].Display, (25, 25))

    #hidden[y][x] = objects[y][x]

    del temp[0]
```

This is the end of the procedure, which uses a copy of 'CompletedPoints' (the coordinates of all adjacent empty tiles), stored in 'temp'. then there is a while loop which traverses the list. The coordinates from each item translates to a point on the grid, and therefore an instance in 'objects'. Using this, all of the tiles in 'CompletedPoints' are changed to cleared in 'objects', and their images are changed and resized to 25 pixels.

FIRSTGO

In Minesweeper, if the player hits a mine on their first go, the mine is relocated and the player can continue. I have created a procedure for this, where if the player hits a mine on their first go the mine they landed on is removed from the grid, and the numbers are recalculated.

```
def firstgo(y, x):  
  
    objects[y][x].mine = False  
  
    minedistance()
```

The procedure is very small with the tile on the grid being changed to not a mine, and 'minedistance' being called to recalculate the numbers on the grid.

THE GAME LOOP

The game loop is a while loop which processes all of the values and user inputs using the functions defined above.

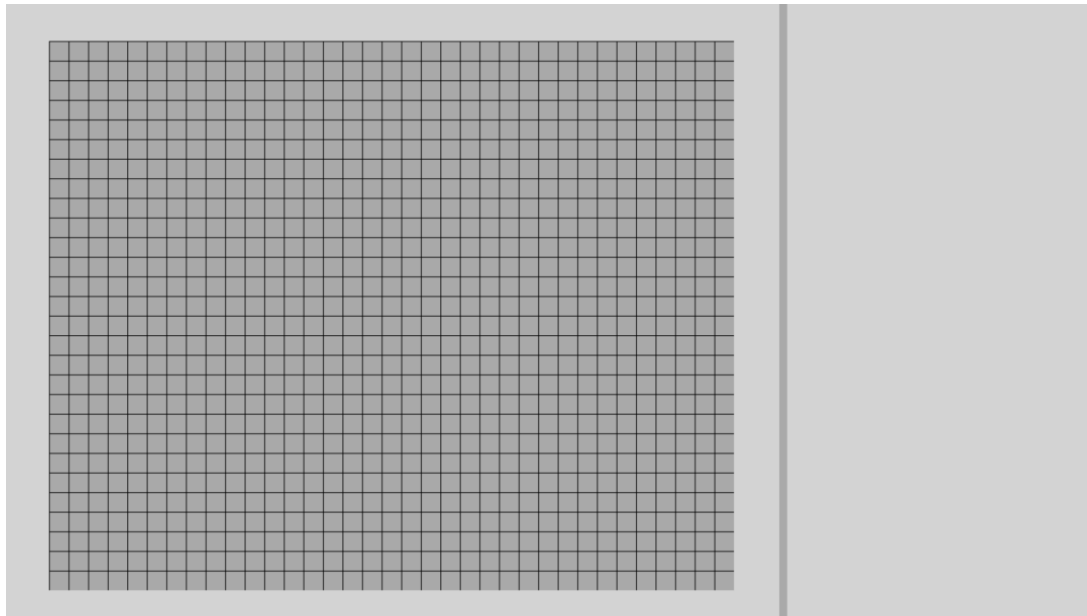
BEFORE THE LOOP STARTS

```
pos = (-1, -1)  
left = False  
right = False  
middle = False  
objects = []  
imagegrid = []  
FQgrid = [] #FQ stands for flag question (displays the images of the flag and question mark above the interactive grid)  
  
fps = 20  
  
WIN = False #If true on loop break, player has won. If False then player has lost  
LOSE = False  
gameEnd = False #Lets game run one more cycle before ending, to clear last tiles  
  
firstGo = True #For if the player hits a mine on the first go  
  
creategrid()  
  
#image = pygame.image.load('')  
#image = pygame.transform.scale(image, (100, 100))####  
  
separator = pygame.Surface((10, 800))  
separator.fill('Dark Grey')
```

Above is the code before the game loop starts. This defines all variables and flags that will be changed through while the game is running. Below is a list explaining what each one is for:

- 'pos': As the position of the mouse is only gotten when it clicks the screen, a value needs to be set before it receives one. This is what (-1, -1) is for.
- 'left', 'right' and 'middle': These three are all flags which turn true if the corresponding mouse button is pressed down. It is reset at the end of the loop before it returns to the top.
- 'objects', 'imagegrid' and 'FQgrid': This is to just create the list variables so the tile data can be added to it.
- 'fps': Variable that stores that the frames per seconds value

- 'WIN', 'LOSE' and 'gameEnd': All are flags that control whether the player has won or lost and then will break the loop to end the game. This essentially force crashes the program by making the user no longer able to interact with the window.
- 'firstGo': A flag that determines whether the user is still on their first go, and will allow the 'firstgo' function to be called if they hit a mine while this variable is 'True'.
- 'creategrid()': This creates the grid before the game actually starts.
- 'separator': The separator is the grey line that splits the window into 2 parts as seen below.



THE EVENT LOOP

This is the first thing to run once the loop has started. The event loop handles user input by detecting if the mouse button has been pressed, which triggers their corresponding flags to change to 'True'. There is also an event in which if the player pressed the 'X' on the window, the window closes.

```
#Checks for events (like close window and mouse button presses)
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
```

Here is the code to close the window, which is in the 'if' statement. It detects the event type which is 'pygame.QUIT'. If this is true, the window closes.

```
#Detects mouse buttons being pressed

#Left click
if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
    left = True
    pos = pygame.mouse.get_pos()

#Middle click
if event.type == pygame.MOUSEBUTTONDOWN and event.button == 2:
    middle = True
    pos = pygame.mouse.get_pos()

#Right click
if event.type == pygame.MOUSEBUTTONDOWN and event.button == 3:
    right = True
    pos = pygame.mouse.get_pos()
```

Above is the statements for mouse input. They are all set the same way. If a mouse button is pressed down (1 is left, 2 is middle, 3 is right), then the corresponding flag is set to true and the position of the mouse at that point is gotten and saved to 'pos'.

DISPLAY

```
#Adds separator
screen.fill('Light Grey')
screen.blit(separator, (990, 0)) #Space is 400 wide

#Display image grid
for i in range(len(imagegrid)):
    for j in range(len(imagegrid[i])):
        screen.blit(imagegrid[i][j], objects[i][j].position)

#Display interactive grid
for i in range(len(objects)):
    for j in range(len(objects[i])):
        screen.blit(objects[i][j].Display, objects[i][j].position)

#Display flag/question grid
for i in range(len(FQgrid)):
    for j in range(len(FQgrid[i])):
        screen.blit(FQgrid[i][j], objects[i][j].position)
    '''Uses pos of tiles from objects because the class is the only place that stores the pixel coordinates
    of the tiles, and theres no point in adding it anywhere else, Adds unnecessary complication'''
```

This part of the game loop focuses on displaying the images to the screen. This is made up of 3 sets of 'for' loops each with another loop inside. Each set of loops traverse their corresponding list (starting with 'imagegrid', and ending with 'FQgrid', with 'objects' in the middle) and blit (display) each index's image to the

screen. the order if these loops are important, so that the numbers are displayed first, then the covered tiles are over that, and then the flags and questions are last. All loops use the position coordinates from 'objects', as a position of one tile in any list corresponds to the same coordinate across all lists, so there is no need to add any more data to the lists is it's not needed.

TILE INTERACTION

This takes up most of the size of the game loop and is all about how the tiles react to the mouse clicks that were detected in event loop.

VARIABLE DEFINITION

```
#Check for mouse presses on interactive grid
for i in range(len(objects)):

    for j in range(len(objects[i])):
        height = objects[i][j].size
        width = objects[i][j].size

        x = objects[i][j].position[0]
        y = objects[i][j].position[1]
```

This is the start of the whole section, which defines some variables which will be used to detect if a tile has been clicked. All other screenshots of code in this section is inside of the second 'for' loop.

LEFT CLICKING

```
#If tile is left clicked
if pos[0] > x and pos[0] < x + width and pos[1] > y and pos[1] < y + height and left == True:

    if objects[i][j].flagged == False and objects[i][j].questioned == False:
        objects[i][j].Display = pygame.image.load(GridImages['E']) #Replace dark grey square with empty tile image
        objects[i][j].Display = pygame.transform.scale(objects[i][j].Display, (25, 25)) #Scale to correct size (25x25)
        objects[i][j].cleared = True

    if objects[i][j].number == 0:
        ClearEmpty(j, i)

    '''cleared = True tells class that tile has been cleared, will help with logic errors
    as a result of the code not knowing the tile has been cleared, so treats it like it hasn't been'''

    if objects[i][j].mine == True and firstGo == False and objects[i][j].flagged == False:
        LOSE = True #Player loses for hitting a mine

    if objects[i][j].mine == True and firstGo == True and objects[i][j].flagged == False:
        firstgo(i, j)
        firstGo = False

firstGo = False
```

Here is the code which handles a left on a tile. It starts with an 'if' statement that is essentially if the position of the mouse click is in the borders of the current tile in the list traversal, then the rest of the code runs. The rest of the code being if the tile is not flagged and not questioned then the covered image is replaced with a transparent image, and the attribute 'cleared' is set to true. After this it will see if the 'number' attribute is zero, which is the number for an empty tile. If so, then 'clearEmpty' runs to clear all adjacent empty tiles.

After this, there are two more 'if' statements. One for if the player hits a mine and 'firstGo' is false, which will set 'LOSE' to true and the player will lose the game. The other is if the player hits a mine, and it is their first go, which will call 'firstgo' to remove the mine the player has hit and recalculate the numbers.

After these have all run, 'firstGo' is set to false. This all repeats until all items have been looked through.

RIGHT CLICKING

```
#If tile is right clicked
if pos[0] > x and pos[0] < x + width and pos[1] > y and pos[1] < y + height and right == True:

    if objects[i][j].cleared == True: #If tile is cleared, do nothing
        pass

    elif objects[i][j].flagged == True: #If tile is already flagged, remove flag
        FQgrid[i][j] = pygame.image.load(GridImages['E'])
        FQgrid[i][j] = pygame.transform.scale(FQgrid[i][j], (25, 25))

        objects[i][j].flagged = False #Removes flagged

    else: #If tile is questioned or not flagged, change to flag
        FQgrid[i][j] = pygame.image.load(GridImages['F'])
        FQgrid[i][j] = pygame.transform.scale(FQgrid[i][j], (25, 25))

        if objects[i][j].questioned == True: #Removes questioned if necessary and adds flagged
            objects[i][j].questioned = False

        objects[i][j].flagged = True
```

The next part is if the tile is right clicked. The starting 'if' statement remains the same as for left clicking, and if this is true, then it will see if the selected tile is a cleared tile, in which it will do nothing if it is, and will pass out of the statement. If it is not cleared, then it will see if it is already flagged and will remove the flag if it is. It will also change the flagged attribute to false. If this is also false, that means the tile is either questioned or it is neither, in which case it will change the image to a flag. If it is questioned, that attribute is set to false and flagged is set to true regardless.

MIDDLE CLICKING

This piece of code works in the same way as the right clicking, except it will change it to a question if the tile is flagged or neither, and will remove it if it already questioned. It also changes the attributes, making 'flagged' to false and 'questioned' to true. Below is the code for this

```
#If tile is middle clicked
if pos[0] > x and pos[0] < x + width and pos[1] > y and pos[1] < y + height and middle == True:

    if objects[i][j].cleared == True: #If tile is cleared, do nothing
        pass

    elif objects[i][j].questioned == True: #If tile is questioned, remove question
        FQgrid[i][j] = pygame.image.load(GridImages['E'])
        FQgrid[i][j] = pygame.transform.scale(FQgrid[i][j], (25, 25))

        objects[i][j].questioned = False #Removes questioned

    else: #If tile is flagged or not questioned, change to question
        FQgrid[i][j] = pygame.image.load(GridImages['Q'])
        FQgrid[i][j] = pygame.transform.scale(FQgrid[i][j], (25, 25))

        if objects[i][j].flagged == True: #Removes flagged if necessary and adds questioned
            objects[i][j].flagged = False

        objects[i][j].questioned = True
```

END OF THE LOOP

```
WIN = CheckWin()

if gameEnd == 2:
    #Win/Lose Conditions
    if WIN:
        print('WON')
        break

    if LOSE:
        print('LOSE')
        break

#Resets mouse click status
left = False
right = False
middle = False

if WIN == True or LOSE == True:
    gameEnd += 1

#Refreshes screen
pygame.display.update()
clock.tick(fps) #fps
```

These are last lines of the game loop. It checks if the player has won or lose. If so then it starts a counter to two, which gives the program enough time to display the last move made before the game ends. Then it will break the loop. It also resets the mouse press variables and refreshes the screen.

TESTING

The following sections will show the testing process for Prototype Two. The tests will evaluate how well the program performs.

TEST PLAN

Test no.	Test description	Test type	Data	Expected result
1a	Left click any covered tile	Normal	Mouse input: Left click	Tile is cleared
1b	Right click any covered tile	Normal	Mouse input: Right click	Tile is flagged
1c	Middle click any covered tile	Normal	Mouse input: Middle click	Tile is questioned
1d	All clicks work on corners	Normal	Mouse input: All	Tiles behave how they should
1e	Rapidly press all buttons	Boundary	Mouse input: Press all buttons rapidly on one tile	tile gets either flagged, questioned or cleared
1f	Press multiple buttons at the same time	Boundary/ Erroneous	Mouse input: Left and Right click	The tile either gets cleared or flagged
2a	Player can win	Normal	Flag all mines/ clear all non-mined tiles	The game loop stops and the player wins
2b	Player can lose	Normal	Hit a mine not on the first go	The game loop stops, and the player loses
2c	'firstGo' rule works	Normal	Hit a mine on the first go	The game carries on and the cleared tile changes to either a number or empty tile
2d	ClearEmpty works	Normal	Clear empty tile	All adjacent empty tiles are cleared
3a	The program loads within 3-5 seconds	Normal	Run the program	The program loads within 3-5 seconds
3b	The 'firstGo' rule takes up to 1-2 seconds to calculate and change the grid	Normal	Same as 2c	The rule takes 1-2 seconds to complete
4a	Flag a flagged tile	Normal	Mouse input: Right click	Flag is removed from tile
4b	Question a questioned tile	Normal	Mouse input: Middle click	Question is removed from tile
4c	Flag a questioned tile	Normal	Mouse input: Right click	Tile changed to flag
4d	Question a flagged tile	Normal	Mouse input: Middle click	Tile changes to question
4e	Clear a flagged or questioned tile	Erroneous	Mouse input: Left click	Nothing happens

TEST RESULTS (TABLE)

Test no.	Actual result	Action & Reason
1a	Expected result	N/A
1b	Expected result	N/A
1c	Expected result	N/A
1d	Expected result	N/A
1e	Expected result	N/A
1f	Expected result	N/A
2a	Expected result	N/A
2b	Expected result	Fix: Although ClearEmpty works perfectly fine, testing this has uncovered a bug in the code, where if an empty tile has been flagged or questioned, then it remains once the tile has been cleared by ClearEmpty.
2c	Expected result	N/A
3a	Expected result (loads in 3s)	N/A
3b	Expected result (calculated in 2s)	N/A
4a	Expected result	N/A
4b	Expected result	N/A
4c	Expected result	N/A
4d	Expected result	N/A
4e	Expected result	N/A

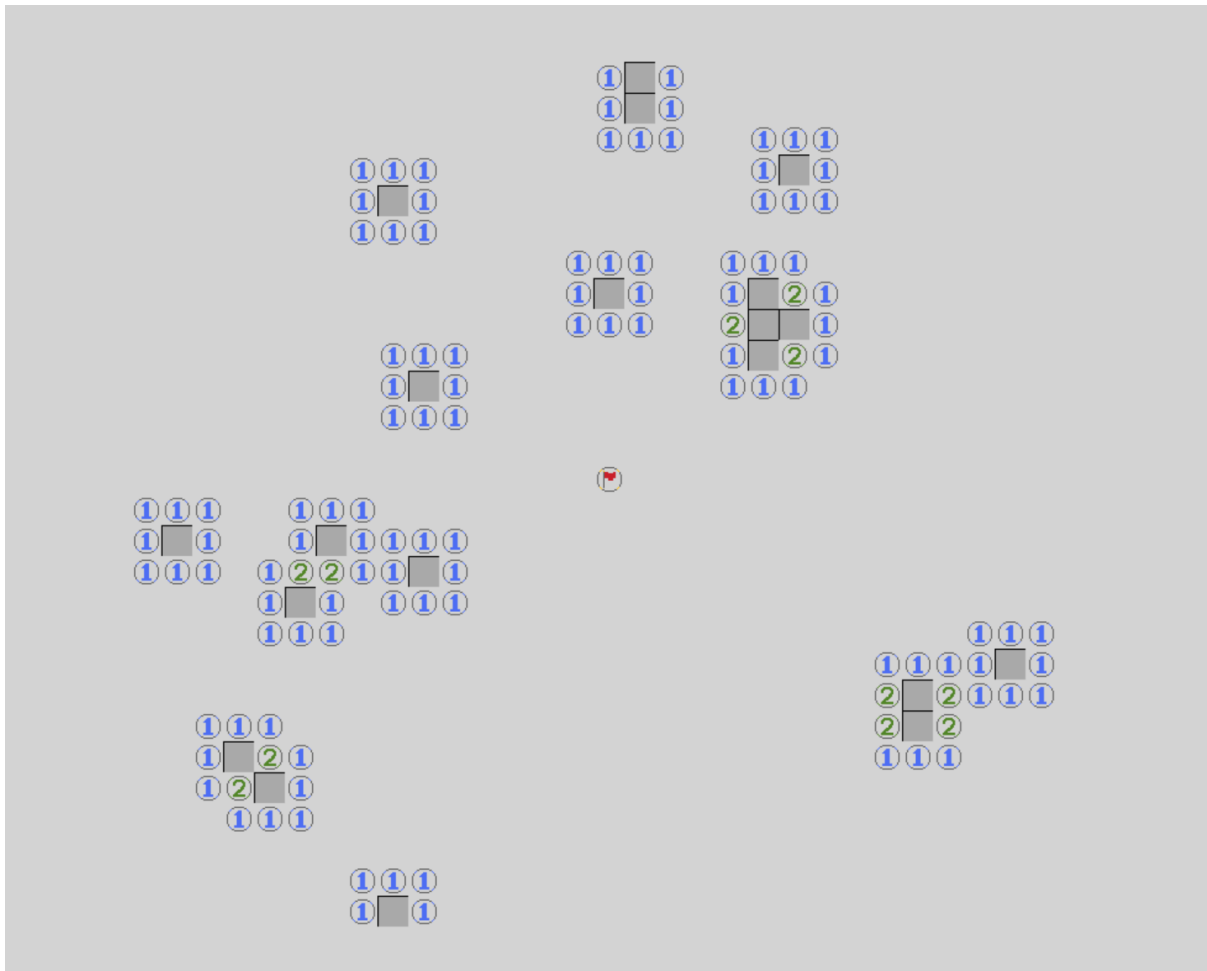
TEST RESULTS (IMAGES, IF APPLICABLE)

Below are the images for some of the tests.

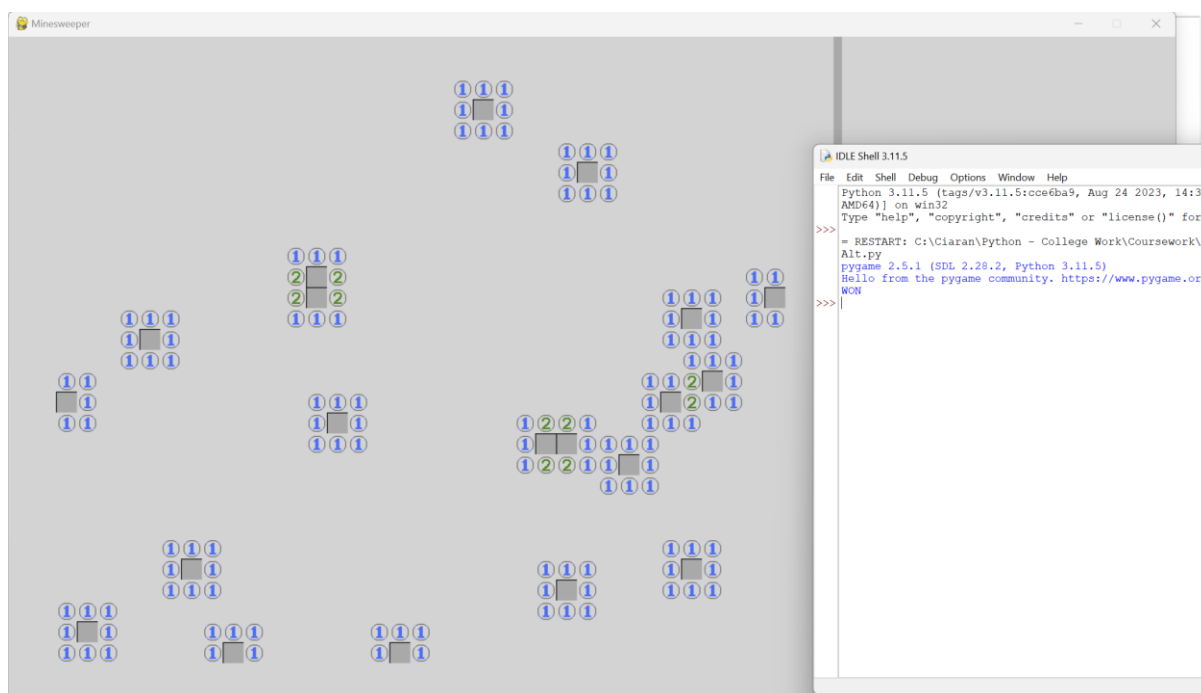
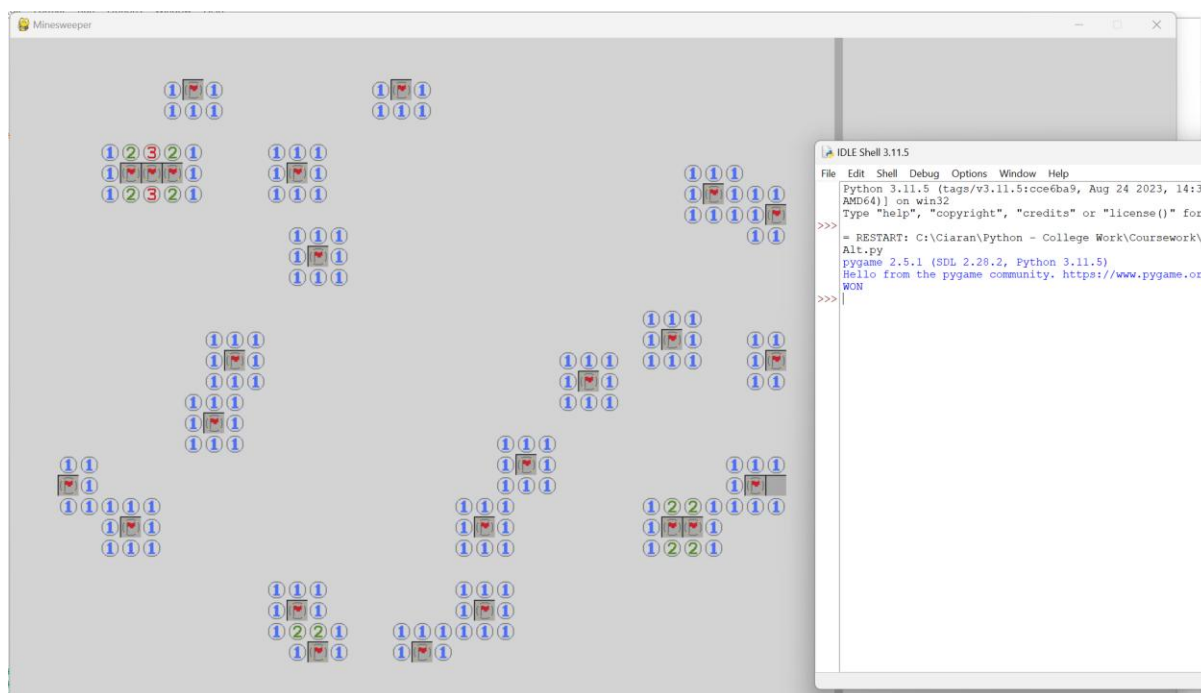
Test 1:



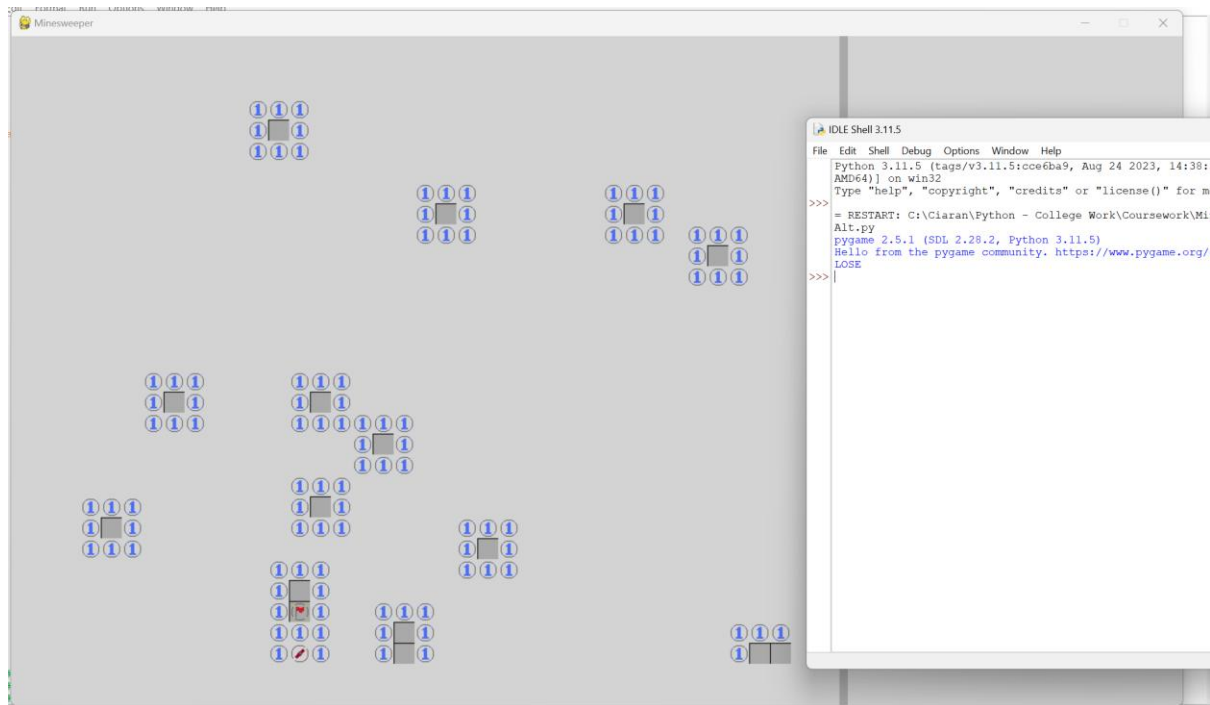
Test 1e 1f:



Test 2a 2d:



Test 2b 2d:



Test 4:



FIXED ERRORS

As mentioned in the test results for Prototype One, I have fixed the error of the 'firstgo' not working, and as shown in this prototype's results, it does work now.

From this set of results, I am going to fix the bug which occurred during test **2b**, where the flag remained after the tile under it got cleared from 'ClearEmpty'. To fix this, I am going to add a couple lines of code into 'ClearEmpty' that will change all of the images in 'FQgrid' to empty for coordinates in 'CompletedPoints'.

```
while len(temp) != 0:

    node = temp[0]

    y = node[0]
    x = node[1]

    objects[y][x].cleared = True
    objects[y][x].Display = pygame.image.load(GridImages['E'])
    objects[y][x].Display = pygame.transform.scale(objects[y][x].Display, (25, 25))

    #hidden[y][x] = objects[y][x]

    del temp[0]
```

Here is the 'while' loop at the end of the procedure that replaces the corresponding tiles' images to empty.
This is where I will put in the code.

```
while len(temp) != 0:

    node = temp[0]

    y = node[0]
    x = node[1]

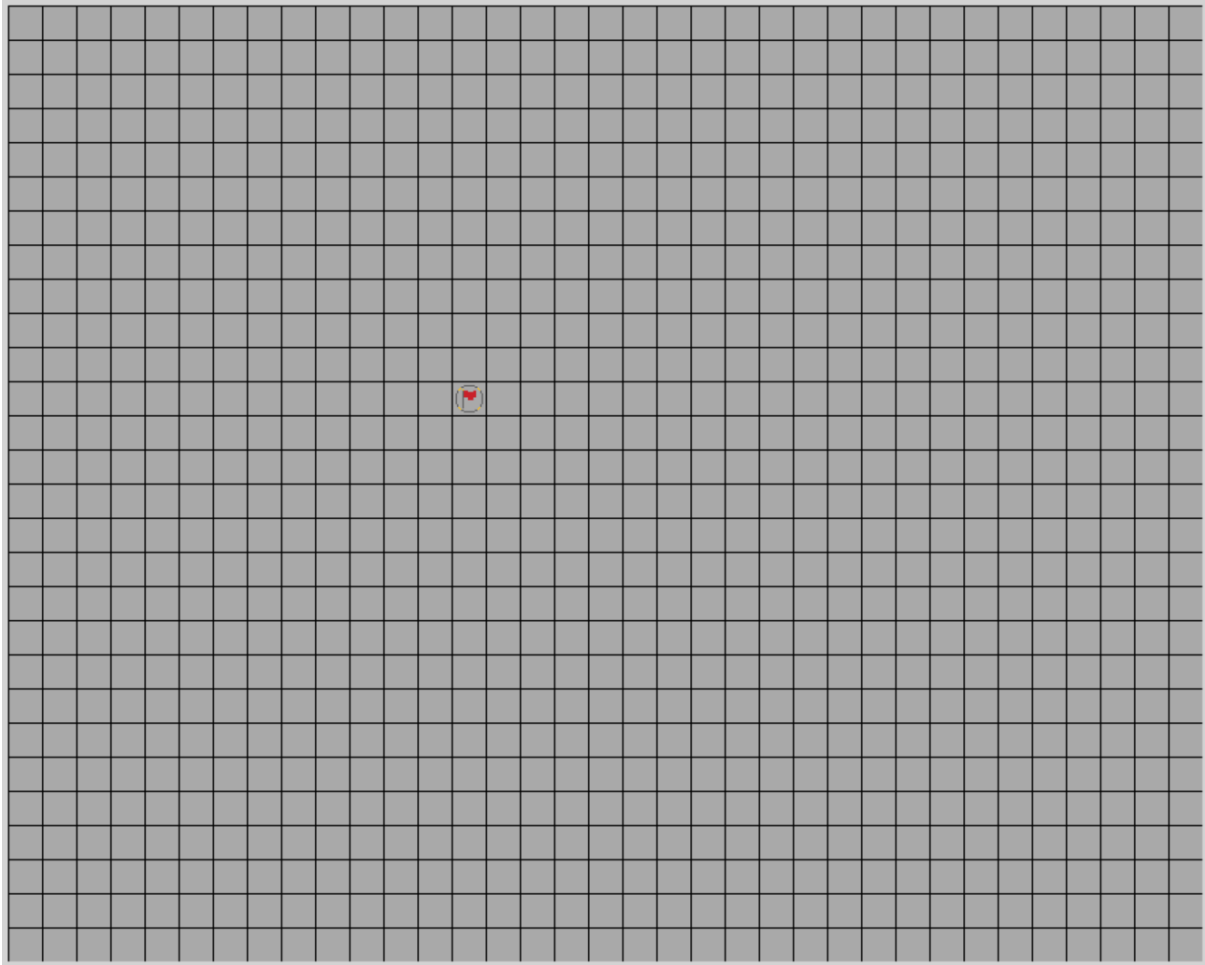
    objects[y][x].cleared = True
    objects[y][x].Display = pygame.image.load(GridImages['E'])
    objects[y][x].Display = pygame.transform.scale(objects[y][x].Display, (25, 25))

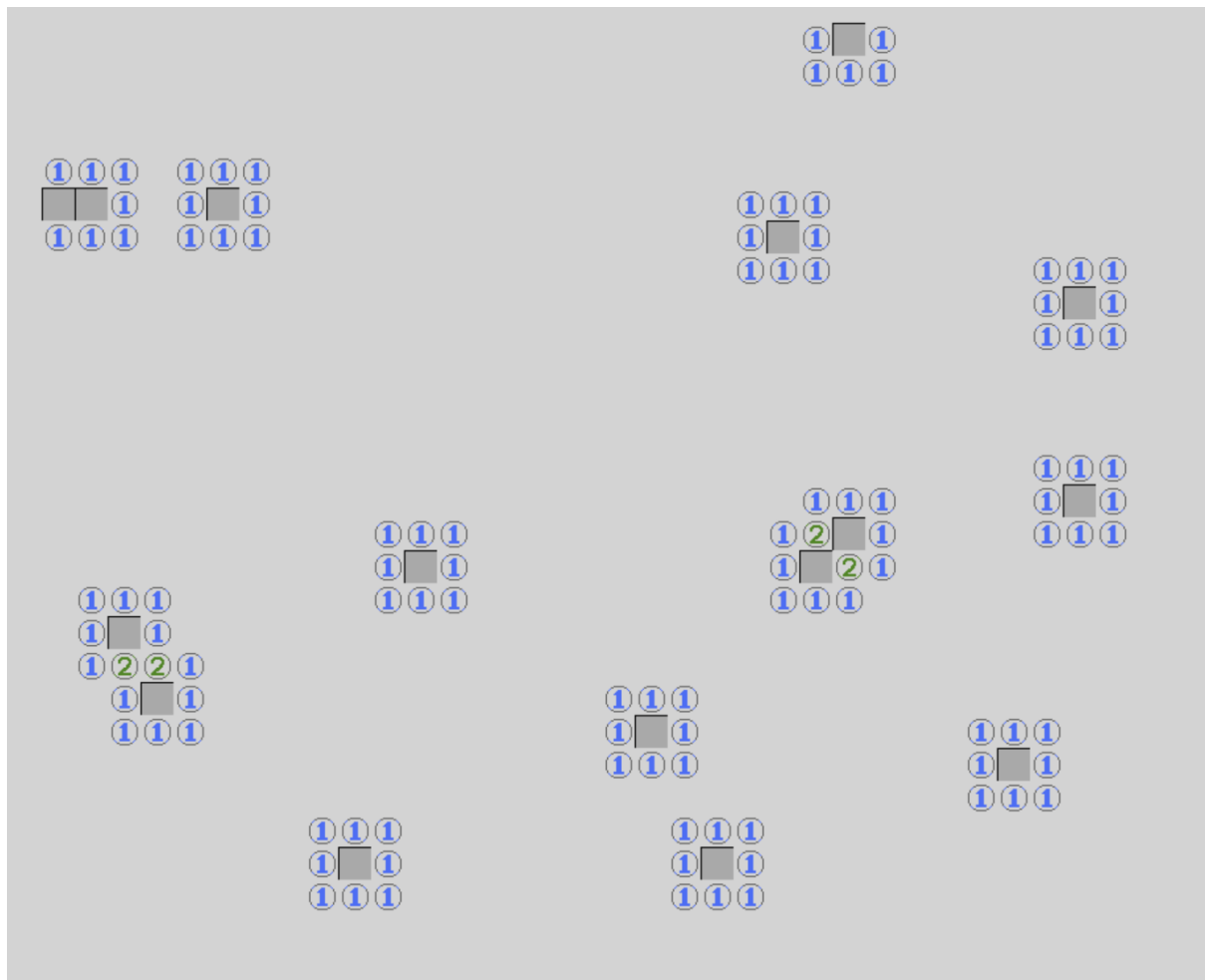
    FQgrid[y][x] = pygame.image.load(GridImages['E'])
    FQgrid[y][x] = pygame.transform.scale(FQgrid[y][x], (25, 25))

    #hidden[y][x] = objects[y][x]

    del temp[0]
```

And here is the new one, that changes all the images on 'FQgrid' for the tiles are that in 'CompletedPoints'.
And below are the images of it working.





The flag has been successfully removed. This will also work for questions as they're part of 'FQgrid' as well.

FIXED ERRORS (IMAGES)

NEXT VERSION

For the final version, I will focus on these things:

- Creating a menu screen which allows the user to change the grid settings in game, and also allow them to start again without needing to rerun the program.
- Make 'objects' the only data structure that stores anything about the tiles. So, this will remove 'imagegrid' and 'FQgrid'. All procedures and functions will need to be changed so that they use values from 'objects'. This should optimise the program, as only one list will need to be referenced instead of the current 3. 'GridImages' will stay as that's how the images are assigned to the tiles.
- Change how the images are displayed to the screen, so only the ones that would be visible/ on top are the only ones getting displayed.

Currently, all three layers of images are being displayed, these being 'imagegrid', the covered tiles and

'FQgrid'. This means that in a 35x28 tile grid, there are 2940 images being displayed on the screen, 2/3 of those not even visible to the user or are transparent images.

To optimise the process and hopefully get the program to run faster upon startup, I will add a stack attribute to the 'Tile' class. This stack will show the current later the tile is on. It will have the following layers:

- **FQ:**
This is the topmost value that can be added to the stack. This is for the flags and question marks. Note that this value will display both the **FQ** and **Covered** images due to gaps in the flag and question images.
- **Covered:**
This is middlemost value that can be added to the stack. This is for the interactive images that cover the numbers.
- **Numbers:**
This is the bottom value that will never be removed from the stack. This is for the number and mine images.

As tiles are interacted with, the values on the top of the stack will change, and the topmost value corresponds to the image that gets displayed.

CONCLUSION

To conclude the development and testing of Prototype Two, the program successfully expands off of Prototype One and by using some of the rules and procedures that I developed. It runs well and can be used to play a game of minesweeper, where the player can both win and lose. There are multiple things planned to add in the final version to make it more polished, but there will be no more changes to the gameplay. Instead, I will focus on optimisation and making it easier to use.

FINAL VERSION – PYTHON MINESWEEPER

INTRODUCTION

This is the final version of my version of Minesweeper made in Python. This version will not add any new gameplay mechanics but will instead be solely for optimising the code and creating a better UI, with the ability to restart the game without needing to rerun the program. The plan for this version is below.

- **Code Optimisation:** I am going to be to optimising and simplifying the code to make quicker to load and easier to read. I will start this by removing the lists 'imagegrid' and 'FQgrid' and saving all of its required data as attributes in 'Tile', stored on 'objects'. I'm also going to revamp how the images are displayed to the screen. As mentioned in the 'Next Version' section on Prototype 2, I'm going to use a stack to only display the topmost image of each tile, as every tile has up to 3 images associated with it at any one time. This will remove up to two thirds of images on the screen that don't need to be there.
- **Menu screen and UI:** I am also going to create a menu screen which will be the screen that appears on the screen before the player presses 'PLAY'. It is also so the player is able to change the grid size and mine density. This will be done using sliders, one for each setting (height, width, mine density). They have a default setting of a 15x15 grid and a mine density of 12%. The density slider will not go up to 100%, as anything over 40% is already incredibly difficult and bordering on impossible unless you're really lucky, so the density will be capped at either 40 or 50%.
- **High scores:** There will also be 5 high scores that will be displayed in the menu screen. They will be worked out the size of the grid, mine density, and time it is completed in. Games where the player loses will not have a score calculated. The scores will be saved to an external file so they can be saved after the user stops playing.

PROGRAM STRUCTURE

MAIN DATA & WHERE IT IS STORED

Below are the data structures that store the data which is essential for the game to work

The existing data structures have been adapted to be more concise and to remove any unnecessary lists or complication. Below is a list of the main data structures:

- **Tile**
Used from prototype two, this is a class that stores all information needed about a certain tile as attributes. There is an instance of this class for every tile, and is stored to **objects**.
- **objects**
This is a 2D list that stores all instances of the **Tile** class.

- **GridImages**
A dictionary that stores the file paths of all the possible images to be used in the program.
- **scores**
A new list for the final version, this stores the high scores from the 'Hi-Scores' file.

imagegrid and **FQgrid** have been removed from this version, as they were storing data that could be turned into an attribute in **Tile** and therefore can be made a part of **objects** instead of having two extra lists that aren't needed.

SLIDERS

There are also new data structures for the sliders, which will be on the menu screen and will be used to change the size of the grid and alter mine density. The sliders work by having set positions, or notches, along the line. When the user presses and holds the left mouse button on the hitbox of one of these notches, the last touched one becomes active, and that value is returned to be used. The data structures handling this are:

- **Notch**
This is a class that stores required information about each notch in the slider as attributes. For example, the notch's position, number value, position, and it's image icon.
- **Sliders**
There are 3 sliders: **denSlider**, **heiSlider**, **widSlider**, which are 1D lists that store a number of **Notch** classes. It is stored with the leftmost notch at index 0. All the lists are stored in the same way and are handled the exact same.

DISPLAY ORDER

To reduce the amount of images of the screen at once, and to improve the speed of the program, there are no longer any set layers of images like there was in prototype 2. Instead, the **Tile** class has been modified to include a stack as an attribute called 'imgOrder'. This stores up to 3 values which correspond to which state the tile is in.

This means that most if not all of the functions and procedures have had to be modified to use this new way of handling images. Each instance of **Tile** also now includes 4 different images: the number of the tile (or mine if it is one), the covered image, the flag image and the question image. There are also 2 methods which are 'Add' and 'Remove', which are used to increase or decrease the stack size and it's values.

The game loop then displays the image corresponding to the value at the top of the stack, meaning only one image is displayed for each tile.

CLASSES, FUNCTIONS/ PROCEDURES & DATA STRUCTURES

As most of the structures haven't changed in how they work compared to prototype 2, this section will focus on the new data structures and changes to the existing ones only.

'TILE CLASS'

```
class Tile():
    def __init__(self, Mine, Number, Size, Position):
        self.mine = Mine
        self.flagged = False
        self.questioned = False
        self.cleared = False
        self.number = Number #Empty
        self.size = (Size, Size)
        self.position = Position

        #self.Display = pygame.Surface((Size, Size))
        self.Display = pygame.image.load(GridImages['C']) #Covered image
        self.Display = pygame.transform.scale(self.Display, (Size, Size))

        self.nDisplay = pygame.image.load(GridImages['E']) #Mine/ number image
        self.nDisplay = pygame.transform.scale(self.nDisplay, (Size, Size))

        self.F = pygame.image.load(GridImages['F']) #Flag image
        self.F = pygame.transform.scale(self.F, (Size, Size))

        self.Q = pygame.image.load(GridImages['Q'])
        self.Q = pygame.transform.scale(self.Q, (Size, Size))

        self.ImgOrder = ['N', 'C'] #Max stack = ['N', 'C', 'F/Q']

    def Add(self, FQ):
        if FQ == 'F':
            self.ImgOrder.append('F')
        elif FQ == 'Q':
            self.ImgOrder.append('Q')

    def Remove(self):
        if self.ImgOrder[-1] != 'N':
            del self.ImgOrder[-1]
        else:
            pass
```

The 'Tile' class has been expanded upon to add 4 more attributes and 2 methods. The 'Add' method is responsible for increasing the stack. It uses 'if' statements to see which value to add. It takes 2 parameters with them being 'self' and 'FQ', 'FQ' is whether the tile is going to be flagged or questioned, and depending on what value this is, either 'F' or 'Q' will be appended to the stack.

'Remove' is another method which is for removing the top value from the stack. It is made of a single 'if' statement and will remove the value as long as the top value in the stack is not 'N', which is the number level. If this was removed, then the stack would be empty, and the program would crash.

'NOTCH CLASS'

```
class Notch():
    def __init__(self, position, value, jump):

        self.position = position
        self.value = value
        self.active = False
        self.icon = pygame.image.load(GridImages['SB'])
        self.iconPos = (position[0] - 13, position[1] - 23)
        self.jump = jump

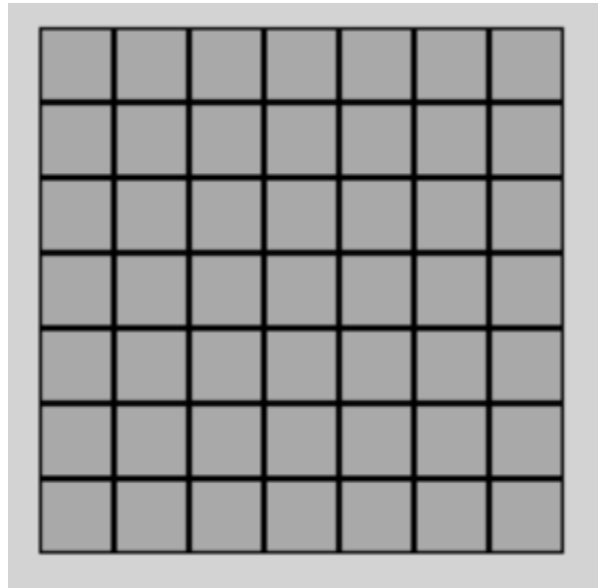
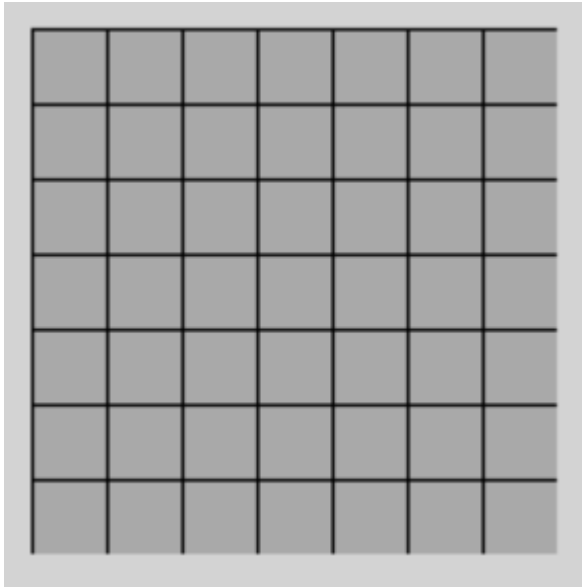
        #If active is true, icon is displayed and value is returned
```

This is the code for the 'Notch' class, it has 6 attributes and no methods

GRIDIMAGES

```
GridImages = {
    1: 'Data\Grid Images\MS One.png',
    2: 'Data\Grid Images\MS Two.png',
    3: 'Data\Grid Images\MS Three.png',
    4: 'Data\Grid Images\MS Four.png',
    5: 'Data\Grid Images\MS Five.png',
    6: 'Data\Grid Images\MS Six.png',
    7: 'Data\Grid Images\MS Seven.png',
    8: 'Data\Grid Images\MS Eight.png',
    'M': 'Data\Grid Images\MS Mine.png',
    'F': 'Data\Grid Images\MS Flag.png',
    'Q': 'Data\Grid Images\MS Question.png',
    'E': 'Data\Grid Images\MS Empty.png',
    'C': 'Data\Grid Images\MS Covered2.png', #M:
    'SB': 'Data\Grid Images\Slider Button.png' ;
}
```

This is the dictionary which stores the file paths of the images which are used in the program. I've created a new image under the label 'SB' which stands for slider button, and it is the image that is displayed on an active notch. I have also modified the image for the covered tile to make all of the sides visible. The images below show this change from the left (old) to the right (new).



CREATESLIDER

```
def createSlider(startPos, startValue, numNotches, increment, defaultVal, sliderSize):  
    lastActiveNotch = None  
  
    temp = [] #Stores list of notches  
    endValue = startValue  
    posJump = sliderSize / numNotches #Distance between each notch and number to jump each time  
  
    #Adds to end to find final value  
    for i in range(numNotches):  
        endValue += increment  
  
    posX = startPos[0]  
    posY = startPos[1]  
    distance = 0  
    count = 0  
  
    for i in range(startValue, endValue, increment):  
        temp.append(Notch((posX + distance, posY), i, posJump)) #i is the current notch's value,  
  
        if i == defaultVal:  
            temp[-1].active = True  
            lastActiveNotch = count #Sets LAN to default value's  
  
        distance += posJump  
        count += 1  
  
    return temp, lastActiveNotch
```

This is the function that creates one of the 3 sliders that are used in the menu. The function starts by defining the name and 6 parameters which are:

- **startPos**
The coordinates that the slider starts at.
- **startValue**
The number value that the slider will begin at.

- **numNotches**
The number of notches in the slider.
- **increment**
How much the number value of the notch increases from the previous notch's value.
- **defaultVal**
The number value that the slider is set upon the program first starting up.
- **sliderSize**
The length of the slider in pixels.

The function starts by defining the variables 'lastActiveNotch', 'endValue' and 'posJump' and a temporary list. A 'for' loop is then used to find the end value for the slider by adding the increment onto **startVal** for the length of **numNotches**. 'lastActiveNotch' is used for displaying the slider to screen so it can remember which notch needs to be active and displaying the image.

Then more variables are prepared for another 'for' loop which is to create the notches and append them to the list. The loop is set up so it begins at the start value, ends at the end value and the end is the increment. For every pass of the loop, a new instance of 'Notch' is appended to the list, setting its coordinates, value and distance from the next one. There is then an 'if' statement also in the loop that if 'i' is equal to **defaultVal** then that instance becomes active and 'lastActiveNotch' is equal to 'count', which is essentially the list index that the notch is in.

After the loop is complete then the temporary list and 'lastActiveNotch' are both returned.

ACTIVENOTCHDETECTION

```
def activeNotchDetection(pos, slider, lastActiveNotch):

    posX = int(pos[0])
    posY = int(pos[1])

    lower = None
    larger = None

    #Need to store index of last active notch, so when mouse moves out of range, the notch remains active
    for i in range(len(slider)):

        hitboxStart = (slider[i].position[0] - (slider[i].jump / 2) , slider[i].position[1] - 23)
        hitboxEnd = (hitboxStart[0] + slider[i].jump , hitboxStart[1] + 46)

        if posX > hitboxStart[0] and posY > hitboxStart[1] and posX < hitboxEnd[0] and posY < hitboxEnd[1]:
            slider[lastActiveNotch].active = False #Sets lastActiveNotch to False
            screen.blit(slider[i].icon, slider[i].iconPos)
            lastActiveNotch = i
            slider[i].active = True

        else: #If no hitbox is detected, then set the lastActiveNotch to True
            slider[i].active = False
            slider[lastActiveNotch].active = True
            screen.blit(slider[lastActiveNotch].icon, slider[lastActiveNotch].iconPos)

    for i in range(len(slider)):
        if slider[i].active:
            return slider[i].value, lastActiveNotch
```

This is another function for the sliders but this time it is to change which notch is currently active. It takes 3 parameters which are 'pos', 'slider', and 'lastActiveNotch' with 'pos' being the position of the mouse and 'slider' being the list that one of the three slider's notches are stored to.

The function starts by splitting the mouse position into two variables, one for the x coordinate and one for the y coordinate, called 'posX' and 'posY'.

A 'for' loop which will repeat for the length of the list is then used to cycle through the entire list. The first thing that happens is that a boundary is made around the notch which is equal to the size of the image used. It works by assigning a tuple to 'hitboxStart'. To work out the x coordinate, the notch's x position is subtracted by half of the jump to the next notch, and the y coordinate is the y position of the notch subtracted by 23. For the end of the hitbox 'hitboxEnd', the x coordinate of 'hitboxStart' is added to the jump to the next notch to get the x value, and the y position of the 'hitboxStart' is added to 46 to get the y coordinate. This is also stored as a tuple.

Next is an 'if' statement which uses the mouse position and hitbox to see if the mouse is in the boundary of the slider button. If so, it sets the active attribute of the last active slider to false and the current one to true. It also changes 'lastActiveNotch' to the index of the current active notch. The slider button image is also displayed at the position of the current active notch.

If this statement isn't true then it sets the current notch's active status to false, and the last active notch to true. The image is then displayed to the screen.

After all of this a final 'for' loop is then, which traverses the list, and if the notch is active, then it returns the notch's number value and 'lastActiveNotch'.

[INSERT GIF OF SLIDER SLIDING AROUND]

CREATEGRID

```
def creategrid(mineDensity, height, width):

    global Gheight #Stands for Grid height and Grid width
    global Gwidth

    Gheight = height #int(input('height: ')) #MAX 28
    Gwidth = width #int(input('width: ')) #MAX 35
    chance = mineDensity #12 #Percentage chance of a tile being a mine

    temp = []
    areaWidth = 890 #35
    areaHeight = 700 #28
    centre = (445, 350)
    size = 25

    if Gheight > 28 or Gwidth > 35: #If inputted grid size is too big
        print('too many tiles')
        0/0

    startY = 400 - ((Gheight / 2) * size)
    startX = 495 - ((Gwidth / 2) * size)

    y = startY
    x = startX

    for i in range(Gheight):

        temp = []

        for j in range(Gwidth): #Creates tiles and whether theyre a mine or not, and gives

            if random.randint(1, 100) <= chance: #changed from (1, chance) == 1
                temp.append(Tile(True, -1, size, (x, y))) #If tile is a mine. set no. to -1
                temp[j].nDisplay = pygame.image.load(GridImages['M']) #adds mine image to
                temp[j].nDisplay = pygame.transform.scale(temp[j].nDisplay, (size, size))
            else:
                temp.append(Tile(False, 0, size, (x, y))) #If tile is not a mine
                #no need to add an empty now as it'll get changed in minedistance() anyway

            x += size

        objects.append(temp)

        y += size
        x = startX

    minedistance() #calculates mine numbers
```

Here is the procedure to create the 'objects' list. It functions the exact same way as prototype 2 however, as 'objects' is now the only tile list, the 'for' loop to create the class instances have been changed. They now include some lines to add the mine image to the instance, saving it to the attribute 'nDisplay', and to scale it as well. This only happens if the tile is a mine. If it isn't, then no image is added to it yet as that is done in 'minedistance()' which is called at the end of this procedure. It also has a new attribute **mineDensity** which is gotten from the sliders and is used for figuring which tiles are going to be mines.

CREATEIMAGEGRID

As the list that this procedure made isn't being used anymore, this procedure has been removed.

MINEDISTANCE


```
if objects[i][j].number == 0:
    objects[i][j].nDisplay = pygame.image.load(GridImages['E']) #Need this for firstGo, so numbers
    objects[i][j].nDisplay = pygame.transform.scale(objects[i][j].nDisplay, objects[i][j].size)
else:
    objects[i][j].nDisplay = pygame.image.load(GridImages[objects[i][j].number])
    objects[i][j].nDisplay = pygame.transform.scale(objects[i][j].nDisplay, objects[i][j].size)
```

As this procedure is quite large in size, and works no different than in the last version, the screenshot above only shows the changes to it.

This 'if' statement is part of the 'for' loop and is after the number of mines around the current tile in the traversal has been calculated. It works by using that number and if it is 0, then an empty image is saved to 'nDisplay'. If it is not 0, then the image corresponding to the number is saved instead.

This piece of code replaces the second set of 'for' loops after this set, making the procedure smaller and therefore quicker to run.

CHECKWIN

This function has remained the exact same from prototype 2, and therefore will not be described.

CLEAREMPTY

```
while len(temp) != 0:
    node = temp[0]
    y = node[0]
    x = node[1]
    objects[y][x].cleared = True
    if objects[y][x].flagged == True or objects[y][x].questioned == True:
        objects[y][x].Remove()
    objects[y][x].Remove()
    del temp[0]
```

This is the end of the procedure, which is used to remove the flags and questions from empty tiles that have been cleared. Instead of the images being changed to empty ones and resized like prototype 2, this the 'Remove()' methods are used instead to delete the corresponding values from the top of the stack, so they are not displayed on the screen.

FIRSTGO

This procedure is the exact same as it is in prototype 2, so it will not be described.

DISPLAYGRID

```
def displayGrid():  
    for i in range(len(objects)):  
        for j in range(len(objects[i])):  
            if objects[i][j].ImgOrder[-1] == 'F':  
                screen.blit(objects[i][j].Display, objects[i][j].position)  
                screen.blit(objects[i][j].F, objects[i][j].position)  
  
            elif objects[i][j].ImgOrder[-1] == 'Q':  
                screen.blit(objects[i][j].Display, objects[i][j].position)  
                screen.blit(objects[i][j].Q, objects[i][j].position)  
  
            elif objects[i][j].ImgOrder[-1] == 'C':  
                screen.blit(objects[i][j].Display, objects[i][j].position)  
  
            elif objects[i][j].ImgOrder[-1] == 'N':  
                if objects[i][j].number == 0 and objects[i][j].mine == False: #]  
                    pass  
                else:  
                    screen.blit(objects[i][j].nDisplay, objects[i][j].position)
```

This procedure is used to find out which image is the right one to display for each tile. It starts with a set of 'for' loops to traverse 'objects' and inside the loop is a set of 'if' statements. It sees if the top of the display stack for the current tile is an 'F', and if so will display the covered and flag images. The same happens for 'Q', but instead it's the covered and question images. If it's a 'C' it is only the covered image and an 'N' gets checked to see if its an empty tile, and if so, no image is displayed. If its not empty, then the corresponding image for that number or mine is displayed.

Skipping the empty tiles means that the program doesn't waste its time and performance on displaying transparent images which are the same as not displaying the image altogether.

RESETGRID

```
def resetGrid(mineDensity):  
  
    objects = []  
    creategrid(mineDensity, gridHeight, gridWidth)
```

This is to reset the grid in preparation for another game. there are only 2 lines of code here. One that removes all of the data from 'objects' and one that calls 'creategrid()' to make the new grid.

ACTADD

```
def actAdd(winLose=None):  
  
    if activeScreens[-1] == 'menu':      #  
        activeScreens.append('playing')  
  
    elif activeScreens[-1] == 'playing':  
        activeScreens.append('gameEnd')  
  
    else:  
        pass
```

This procedure is for handling the 'activeScreens' stack. It contains an 'if' statement. If the top of the stack is 'menu' then 'playing' is added to the top of the stack. If its 'playing' then 'gameEnd' is added. If its anything else then nothing happens

ACTREMOVE

```
def actRemove():  
  
    if activeScreens[-1] != 'menu':  
        del activeScreens[-1]
```

This is for removing the top item from the 'activeScreens' stack. If the top is not 'menu' then the top is deleted from the stack.

PLAYFORFEIT

```
def playForfeit():  
  
    #Button coords 1075x 675y  
    #Button size 250x 100y  
  
    if activeScreens[-1] == 'menu':  
        actAdd()  
        return True #Restarts game (creates grid)  
  
    if activeScreens[-1] == 'playing':  
        actRemove()  
        return False  
  
    if activeScreens[-1] == 'gameEnd':  
        actRemove()  
        actRemove()  
        return True
```

This is a function for handling the changing of screens in the program. This runs when the button is pressed. If the stack is currently on 'menu' then 'actAdd()' is called. If its 'playing', then 'actRemove()' is called. And if its 'gameEnd', then 2 lots of 'actRemove' are called to return the player to the menu.

BUTTONSTATE

```
def buttonState(): #Refreshes the button state every frame  
    if activeScreens[-1] == 'menu':  
        startButton.fill((0, 150, 0))  
        screen.blit(playTxt, (1138,697))    #1075 675  
        #text on button go here  
  
    elif activeScreens[-1] == 'playing':  
        startButton.fill((150, 0, 0))  
        screen.blit(forfeitTxt, (1095, 697))  
  
    elif activeScreens[-1] == 'gameEnd':  
        startButton.fill('Black')  
        screen.blit(menuTxt, (1138,697))
```

This is a procedure for displaying the buttons onto the screen. It also uses 'activeScreens'. The button is green and has the text 'PLAY', if the menu is currently at the top. The button is red and has the text 'FORFEIT' if the game is currently in progress, and the game is over, the button is black and has the text 'MENU'.

HISCORES

Here is the procedure which handles the new high-score system. The high scores are saved to an external file so they remain even if the program is closed. It takes 4 parameters:

- **time**
The time in minutes of the game
- **density**
The density setting of the game
- **height**
The height setting of the game
- **width**
The width setting of the game

```
def hiScores(time, density, height, width):  
  
    scoreFile = open('Data\Hi-Scores.txt', 'r')  
  
    scores = []  
    mineNo = 0  
  
    for i in range(5):  
        temp = scoreFile.readline()  
        scores.append(temp[0:-1])  
  
    scoreFile.close()  
  
    newScore = (height * width)  
  
    #Time multiplier#  
    if time < 1:  
        newScore = newScore * 5  
    elif time < 3:  
        newScore = newScore * 4  
    elif time < 5:  
        newScore = newScore * 3  
    elif time < 7:  
        newScore = newScore * 2  
    elif time < 10:  
        newScore = newScore * 1  
    elif time > 10:  
        newScore = newScore * 0.5  
  
    #Density Multiplier#  
    newScore = newScore * (density / 10)  
  
    #Mine Score#  
    for i in range(len(objects)):  
        for j in range(len(objects[i])):  
            if objects[i][j].mine:  
                mineNo += 1  
  
    newScore += mineNo  
    newScore = round(newScore)
```

This is the start of the procedure. It begins by opening the file in the 'Data' folder called 'Hi-Scores.txt', and this is opened in read mode, and is saved to the label 'scoreFile'. The 'scores' list is then wiped and a variable called 'mineNo' is defined. A 'for' loop is run and reads a line of the file, saving to a temporary file, before appending a slice of it to 'scores'. It takes a slice of 'temp' to remove the '\n' at the end of the each line. This runs 5 times as there are only 5 high scores in the file. After this has run, the file is closed. Then the variable 'newScore' is defined and the result of the multiplication of the grid height and width saved to it. This is the grid size score.

The next part is the time multiplier. This uses the time parameter. A block of 'if' statements are used:

- **If time is less than 1 minute**
Multiply 'newScore' by 5
- **If time is less than 3 minutes**
Multiply 'newScore' by 4
- **If time is less than 5 minutes**
Multiply 'newScore' by 3
- **If time is less than 7 minutes**
Multiply 'newScore' by 2
- **If time is less than 10 minutes**
Multiply 'newScore' by 1
- **If time is greater than 10 minutes**
Multiply 'newScore' by 0.5

The final score modifier is then calculated which is the mine score. A set of 'for' loops are used to traverse 'objects', adding 1 onto 'mineNo' if the current tile is a mine, and nothing if it isn't. After the list has been traversed completely. The number for 'mineNo' is added onto 'newScore' and this is then rounded to get rid of any decimal places.

```
#Wipe File#
scoreFile = open('Data\Hi-Scores.txt', 'w')
scoreFile.write('')
scoreFile.close()

#Replace Scores#
for i in range(len(scores)):

    if newScore >= int(scores[i]):
        scores.insert(i, newScore)
        del scores[-1]
        break

#Add Scores To File#
scoreFile = open('Data\Hi-Scores.txt', 'a')

for i in range(len(scores)):
    scoreFile.write(str(scores[i]))
    scoreFile.write('\n') #Goes onto next line#

scoreFile.close()
```

This is the other part of the procedure, which is for checking and changing scores. The file is opened again to 'scoreFile', but this time it is in write mode. A single line to write nothing into the file is used to delete all the data from the file. A 'for' loop is then used to compare the values in 'scores'. If the new score is higher or equal to the current score the loop is on, then it is inserted into the list at that index. If this happens then the last item in the list is deleted and the loop is broken.

Lastly the scores are saved to the file. The file is open once again to 'scoreFile' in append mode. Another 'for' loop is used to write each item in the list to the file. After each item '\n' is also added to go on to the next line. After this the file closed and the procedure has finished running.

THE GAME LOOP

BEFORE THE LOOPS STARTS

```
pos = (-1, -1)
left = False
right = False
middle = False
down = False
objects = []
scores = []
activeScreens = ['menu']
                #Stack will consist of [menu, playing, gameEnd]

#Loads Scores#
scoreFile = open('Data\Hi-Scores.txt', 'r')
for i in range(5):
    temp = scoreFile.readline()
    scores.append(temp[0:-1])
scoreFile.close()

'''
TO DO:
- Timer
- Menu screen
- Score calc & store to ext. file
- Program optimisation
- Stuff on right of screen
'''

fps = 60 #Increased from 20

WIN = False #If true on loop break, player has won. If False then player has lost
LOSE = False
SCORE = False
gameEnd = False #Lets game run one more cycle before ending, to clear last tiles
reset = False #Flag to tell program to reset grid
gameLoop = True #Runs game part of loop

density = 12
gridHeight = 15
gridWidth = 15

firstGo = True #For if the player hits a mine on the first go

fpsCount = 0 #All for working the timer
secTimer = 00
secTimer2Digit = 00 #Same as secTimer but will have a zero in front if its less than 10
minTimer = 0
```

This is part of the program before the game loops starts. The purpose of this is to define variables before the game starts. The purpose of each of these is below:

- **pos** – Stores mouse position
- **left** – Flag which is true when mouse is left clicked
- **right** – Flag which is true when mouse is right clicked
- **middle** – Flag which is true when mouse is middle clicked
- **down** - Flag which is true when the left click is held down

- **objects** – 2D list that stores all tile data
- **scores** – 1D list that stores current high scores
- **activeScreens** – Stack that shows which screen is currently active
- **fps** – The frames per second that the game plays at
- **WIN** – Flag that is true when the player wins
- **LOSE** – Flag that is true when the player loses
- **SCORE** – Flag what when its true, 'hiScores()' is called
- **gameEnd** – Flag that allows the loops to run once more, to show all the mines.
- **reset** – Flag that resets gameplay related variables when this is true
- **gameLoop** – Flag that when it's true, the gameplay loop doesn't run
- **density** – Variable that stores the mine density for the grid
- **gridHeight** – Variable that stores the height for the grid
- **gridWidth** – Variable that stores the width for the grid
- **firstGo** – Flag that if its true, the player is on their first go, and won't lose if they hit a mine
- **fpsCount** – A variable that increases by one every frame
- **secTimer** – A variable that stores the seconds passed
- **secTimer2Digit** – The same as the above but is used for values below 10
- **minTimer** – A variable that stores the minutes passed

```
creategrid(density, gridHeight, gridWidth)

denSlider, LANd = createSlider((1070, 100), 1, 50, 1, 12, 270) #####
heiSlider, LANh = createSlider((1070, 200), 5, 24, 1, 15, 274)
widSlider, LANw = createSlider((1070, 300), 5, 31, 1, 15, 270)

#image = pygame.image.load('')
#image = pygame.transform.scale(image, (100, 100)) #####

#Menu screen shapes#
separator = pygame.Surface((10, 800))
separator.fill('Dark Grey')

sliderLine = pygame.Surface((270, 6))
sliderLine.fill('Grey')

startButton = pygame.Surface((250, 100))
startButton.fill((0, 150, 0))

titleFont = pygame.font.SysFont('Courier New', 50)
titleTxt = titleFont.render('Python Minesweeper', False, (0, 0, 0))

playTxt = titleFont.render('PLAY', False, (0, 0, 0))
forfeitTxt = titleFont.render('FORFEIT', False, (255, 255, 255))
menuTxt = titleFont.render('MENU', False, (255, 255, 255))

sideFont = pygame.font.SysFont('Courier New', 20)

winTxt = titleFont.render('MINES CLEARED', False, (0, 150, 0))
loseTxt = titleFont.render('MINE HIT', False, (150, 0, 0))

leftText = sideFont.render('[Left]: Clear/Select', False, (0, 0, 0))
rightText = sideFont.render('[Right]: Flag', False, (0, 0, 0))
middleText = sideFont.render('[Middle]: Question', False, (0, 0, 0))
```

This is the rest of the code before the loop starts. This part is for creating the slider lists and defining some of the text to be displayed on the screen.

THE EVENT LOOP


```
#Left click
if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
    left = True
    down = True
    pos = pygame.mouse.get_pos()

#Middle click
if event.type == pygame.MOUSEBUTTONDOWN and event.button == 2:
    middle = True
    pos = pygame.mouse.get_pos()

#Right click
if event.type == pygame.MOUSEBUTTONDOWN and event.button == 3:
    right = True
    pos = pygame.mouse.get_pos()

#Left button is not pressed
if event.type == pygame.MOUSEBUTTONUP and event.button == 1:
    down = False
```

This is the code for the mouse button controls in the event loop. there have been some changes from prototype 2, mainly the flag 'down' is now under the left click event and an extra one where if the mouse button is not pressed, 'down' changes to false.

DISPLAY

MENU

```
##### DISPLAY #####
if activeScreens[-1] == 'menu':
    screen.blit(titleTxt, (200, 370))
    screen.blit(sliderLine, (1070, 97))
    screen.blit(sliderLine, (1070, 197))
    screen.blit(sliderLine, (1070, 297))

    screen.blit(leftText, (1000, 610))
    screen.blit(middleText, (1000, 630))
    screen.blit(rightText, (1000, 650))

    densityStr = 'Mine density: ' + str(density) + '%'
    densityText = sideFont.render(densityStr, False, (0, 0, 0))
    screen.blit(densityText, (1110, 55))

    heightStr = 'Grid height: ' + str(gridHeight)
    heightText = sideFont.render(heightStr, False, (0, 0, 0))
    screen.blit(heightText, (1120, 155))

    widthStr = 'Grid width: ' + str(gridWidth)
    widthText = sideFont.render(widthStr, False, (0, 0, 0))
    screen.blit(widthText, (1125, 255))

    for i in range(len(denSlider)):
        if denSlider[i].active:
            screen.blit(denSlider[i].icon, denSlider[i].iconPos)

    for i in range(len(heiSlider)):
        if heiSlider[i].active:
            screen.blit(heiSlider[i].icon, heiSlider[i].iconPos)

    for i in range(len(widSlider)):
        if widSlider[i].active:
            screen.blit(widSlider[i].icon, widSlider[i].iconPos)

    if down:
        pos = pygame.mouse.get_pos()
        density, LANd = activeNotchDetection(pos, denSlider, LANd)
        gridHeight, LANh = activeNotchDetection(pos, heiSlider, LANh)
        gridWidth, LANw = activeNotchDetection(pos, widSlider, LANw)

    #Hi Scores#
    scorelabel = sideFont.render('High Scores:', False, (0, 0, 0))
    score1 = sideFont.render(scores[0], False, (0, 0, 0))
    score2 = sideFont.render(scores[1], False, (0, 0, 0))
    score3 = sideFont.render(scores[2], False, (0, 0, 0))
    score4 = sideFont.render(scores[3], False, (0, 0, 0))
    score5 = sideFont.render(scores[4], False, (0, 0, 0))
```

This is the start of the display for the menu screen. If the active screen is 'menu', then all of the following runs. All of the slider lines are displayed to the screen, and the text about the controls are also displayed. The text for the sliders are also displayed and then 'for' loops are used to cycle through the notches in each slider, displaying the slider button on the ones that are active.

After is an 'if' statement which if left click is held down, the 'activeNotchDetection()' for each slider is called, to change the active notch. After this, more text is displayed for the high scores, and also continues into the next image below.

```
#Refreshes scores with newest data form file#
scores = []
scoreFile = open('Data\Hi-Scores.txt', 'r')
for i in range(5):
    temp = scoreFile.readline()
    scores.append(temp[0:-1])
scoreFile.close()

screen.blit(scorelabel, (1000, 400))
screen.blit(score1, (1000, 420))
screen.blit(score2, (1000, 440))
screen.blit(score3, (1000, 460))
screen.blit(score4, (1000, 480))
screen.blit(score5, (1000, 500))
```

PLAYING

```
elif activeScreens[-1] == 'playing':  
    #Displays Grid  
    displayGrid()  
  
    #Displays Timer  
    tString = 'Time: ' + str(minTimer) + ':' + str(secTimer2Digit)  
    timerTxt = sideFont.render(tString, False, (0, 0, 0))  
    screen.blit(timerTxt, (1020, 20))
```

This is the code to display the grid and timer. The grid is put to the screen by calling 'displayGrid()'. The timer is then displayed by creating a string by combining the minute and second timer, and then displaying this onto the screen.

GAMEEND

```
elif activeScreens[-1] == 'gameEnd':  
    displayGrid()    #Grid is still shown so you can grid at time of game end  
  
    #Displays Timer but will be paused  
    tString = 'Time: ' + str(minTimer) + ':' + str(secTimer2Digit)  
    timerTxt = sideFont.render(tString, False, (0, 0, 0))  
    screen.blit(timerTxt, (1020, 20))  
  
    if WIN:  
        screen.blit(winTxt, (1005, 100))  
    if LOSE:  
        screen.blit(loseTxt, (1083, 100))
```

This is the code to display the data on the screen that will show when the game is over. Like the above section, the timer is displayed. The grid is also displayed but the player can't interact with it. Depending on whether the player wins or loses, the win or lose text is displayed to the screen.

The button states are also refreshed after this using 'buttonState()'

PLAYING LOOP

The playing loop or as it is called in prototype 2, 'tile interaction' hasn't changed much across the 2 versions. The differences are that the methods 'Add()' and 'Remove()' are used to process tile flagging, questioning and clearing now instead of 'FQgrid' and 'imagegrid'.

END OF THE LOOP

```
WIN, SCORE = CheckWin()

#Resets mouse click status#
left = False
right = False
middle = False
reset = False

if WIN == True or LOSE == True:

    if WIN:
        #Flags all mines so the player can see all t
        for i in range(len(objects)):
            for j in range(len(objects[i])):
                if objects[i][j].mine:
                    objects[i][j].Add('F')
    if SCORE and scorePass == False:
        hiScores(minTimer, density, gridHeight, gridWidth)
        scorePass = True

    if LOSE:
        #Shows mine locations on lose
        for i in range(len(objects)):
            for j in range(len(objects[i])):
                if objects[i][j].mine:
                    objects[i][j].ImgOrder = ['N']

    gameEnd += 1
    gameLoop = False

    actAdd()

if activeScreens[-1] == 'playing': #If game is playing, shows timer
    fpsCount += 1

    if fpsCount % fps == 0:
        secTimer += 1

    if secTimer == 0:
        secTimer2Digit = '00'
    elif secTimer < 10 and secTimer != 0:
        secTimer2Digit = '0' + str(secTimer)
    else:
        secTimer2Digit = secTimer

    if secTimer == 60:
        secTimer = 0
        minTimer += 1

#Refreshes screen
pygame.display.update()
clock.tick(fps) #fps
```

This is the code after the playing loop and is the last part of the game loop. Like prototype 2 it handles the winning and losing conditions. It also now includes the 'hiScores()' procedure to handle the editing of the high scores external file. It also has the code for the timer, which works by every 'fps' amount of frames that go by, the second timer increases by one, and then the minute timer increases for every 60 seconds that pass.

At the very bottom is the screen refresh and tick speed which is the frames per second.

TESTING

TEST PLAN

Below is the defined test plan for the final version. It contains the same tests as prototype 2 with some added tests for the GUI and grid creation and reset. The test that were in prototype 2 and have the same result will not have the results evidence in the images section to save space and to make sure it is easy to understand.

Test no.	Test description	Test type	Data	Expected result
1a	Left click any covered tile	Normal	Mouse input: Left click	Tile is cleared
1b	Right click any covered tile	Normal	Mouse input: Right click	Tile is flagged
1c	Middle click any covered tile	Normal	Mouse input: Middle click	Tile is questioned
1d	All clicks work on corners	Normal	Mouse input: All	Tiles behave how they should
1e	Rapidly press all buttons	Boundary	Mouse input: Press all buttons rapidly on one tile	tile gets either flagged, questioned or cleared
1f	Press multiple buttons at the same time	Boundary/ Erroneous	Mouse input: Left and Right click	The tile either gets cleared or flagged
2a	Player can win	Normal	Flag all mines/ clear all non-mined tiles	The game loop stops and the player wins
2b	Player can lose	Normal	Hit a mine not on the first go	The game loop stops, and the player loses
2c	'firstGo' rule works	Normal	Hit a mine on the first go	The game carries on and the cleared tile changes to either a number or empty tile
2d	ClearEmpty works	Normal	Clear empty tile	All adjacent empty tiles are cleared
3a	The program loads within 3-5 seconds	Normal	Run the program	The program loads within 3-5 seconds
3b	The 'firstGo' rule takes up to 1-2 seconds to calculate and change the grid	Normal	Same as 2c	The rule takes 1-2 seconds to complete
4a	Flag a flagged tile	Normal	Mouse input: Right click	Flag is removed from tile
4b	Question a questioned tile	Normal	Mouse input: Middle click	Question is removed from tile
4c	Flag a questioned tile	Normal	Mouse input: Right click	Tile changed to flag
4d	Question a flagged tile	Normal	Mouse input:	Tile changes to question

			Middle click	
4e	Clear a flagged or questioned tile	Erroneous	Mouse input: Left click	Nothing happens
5a	Play button creates grid and changes screen to gameplay	Normal	Mouse input: Left click play button	Grid is shown on the screen
5b	Foreit button returns player to menu screen	Normal	Mouse input: Left click forfeit button	Menu screen is shown on the screen
5c	Menu button returns player to menu screen	Normal	Mouse input: Left click menu button	Menu screen is shown on the screen
5d	Spamming the buttons rapidly	Boundary	Mouse input: Left click buttons rapidly	The program doesn't crash and will show the correct screens depending on the button state
6a	Grid is created to the size and density as shown on the menu	Normal	Mouse input: Left click play button	The grid has the same size and density as the settings in the menu
6b	Grid is different each time the play button is pressed, and the grid is created	Normal	Mouse input: Left click play button and grid	The grid is different than the last
7a	Sliders let player adjust value	Normal	Mouse input: Left click and drag slider button	The slider moves as the mouse does and the value changes
7b	Value from sliders affect grid settings	Normal	Mouse input: Left click and drag slider button	The grid is the same size and density as the sliders' ones are
7c	Value of slider is saved across games	Normal	Mouse input: Left click play button	The slider value doesn't change after the player returns to the menu from playing a game
7d	Rapidly move mouse across slider	Boundary	Mouse input: Left click and drag slider button erratically	The slider shows the value the mouse is pointing to

TEST RESULTS (TABLE)

Test no.	Actual result	Action & Reason
1a	Expected result	N/A
1b	Expected result	N/A
1c	Expected result	N/A
1d	Expected result	N/A
1e	Expected result	N/A
1f	Expected result	N/A
2a	Expected result	N/A
2b	Expected result	N/A
2c	Expected result	N/A
3a	Expected result (loads in 2s)	N/A
3b	Expected result (<2s, seamless)	N/A
4a	Expected result	N/A
4b	Expected result	N/A
4c	Expected result	N/A
4d	Expected result	N/A
4e	Expected result	N/A
5a	Expected result	N/A
5b	Expected result	N/A
5c	Expected result	N/A
5c	Expected result	N/A
5d	Expected result	N/A
6a	Expected result	N/A
6b	Expected result	N/A
7a	Expected result	N/A
7b	Expected result	N/A
7c	Expected result	N/A
7d	Expected result	N/A

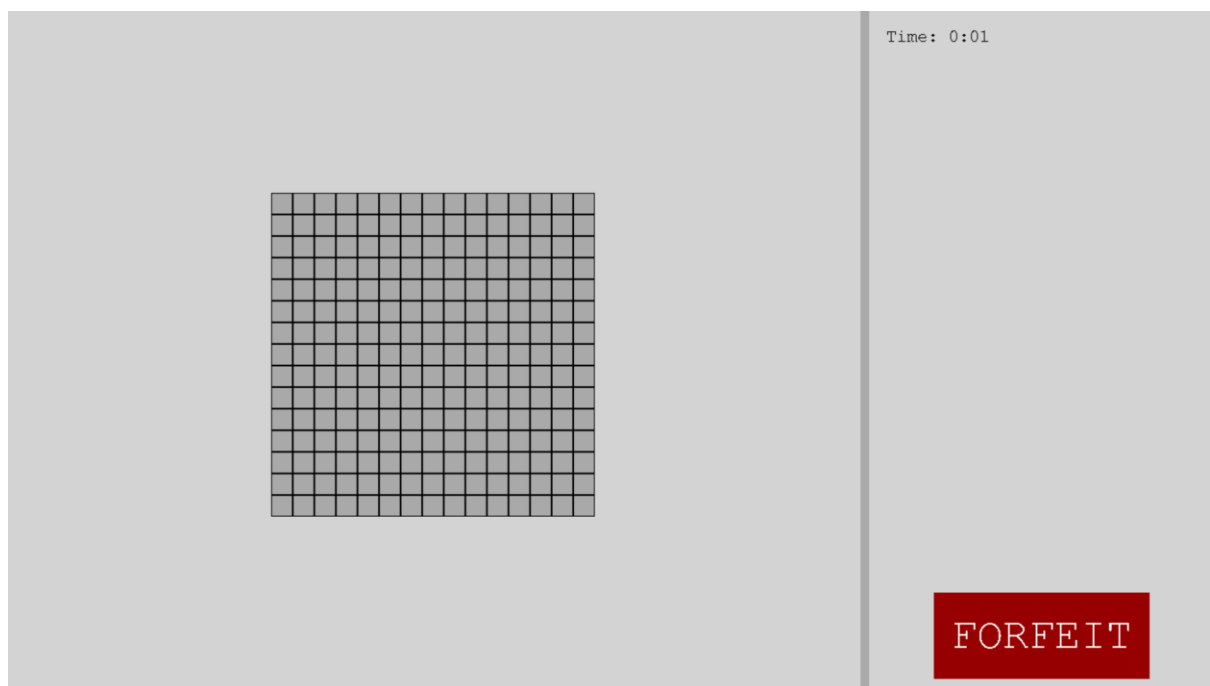
Throughout the testing of the program, I countered a bug to do with the win conditions. If I set the grid size to small and a small mine density, there is a chance that the grid will generate with no mines at all, immediately winning the game. if I then press the menu button, it will immediately go back to the gameplay screen and make a new grid without going to the menu. This means that I can get into a loop of these grid.

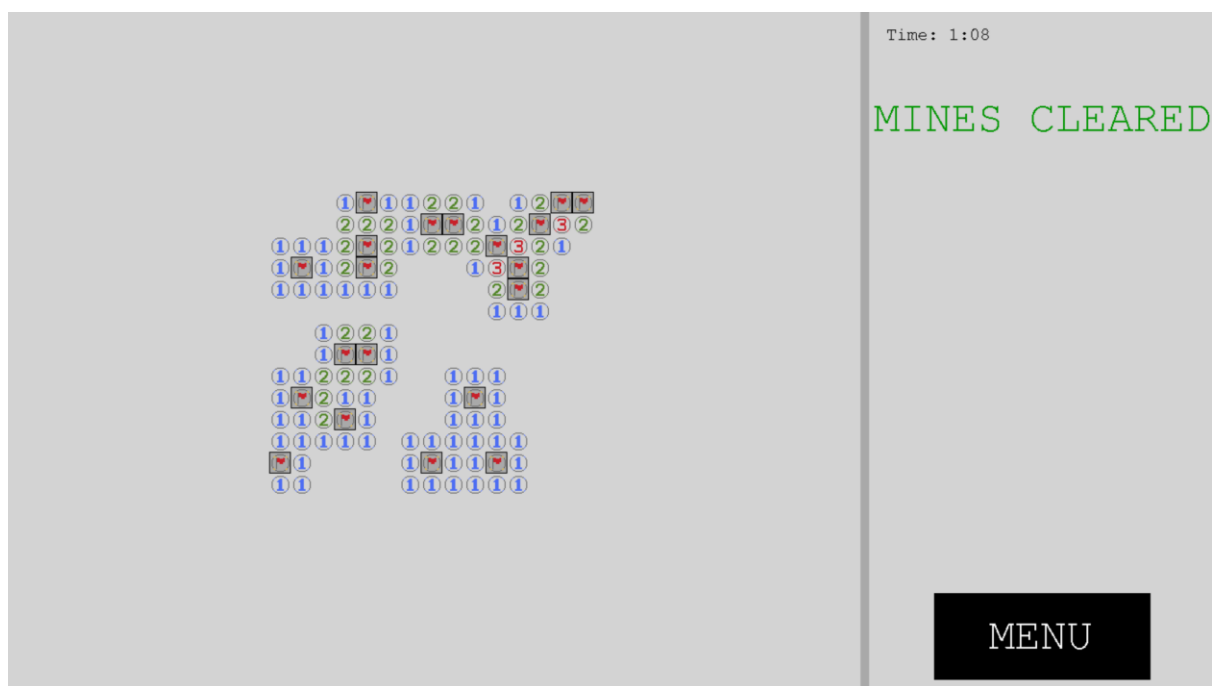
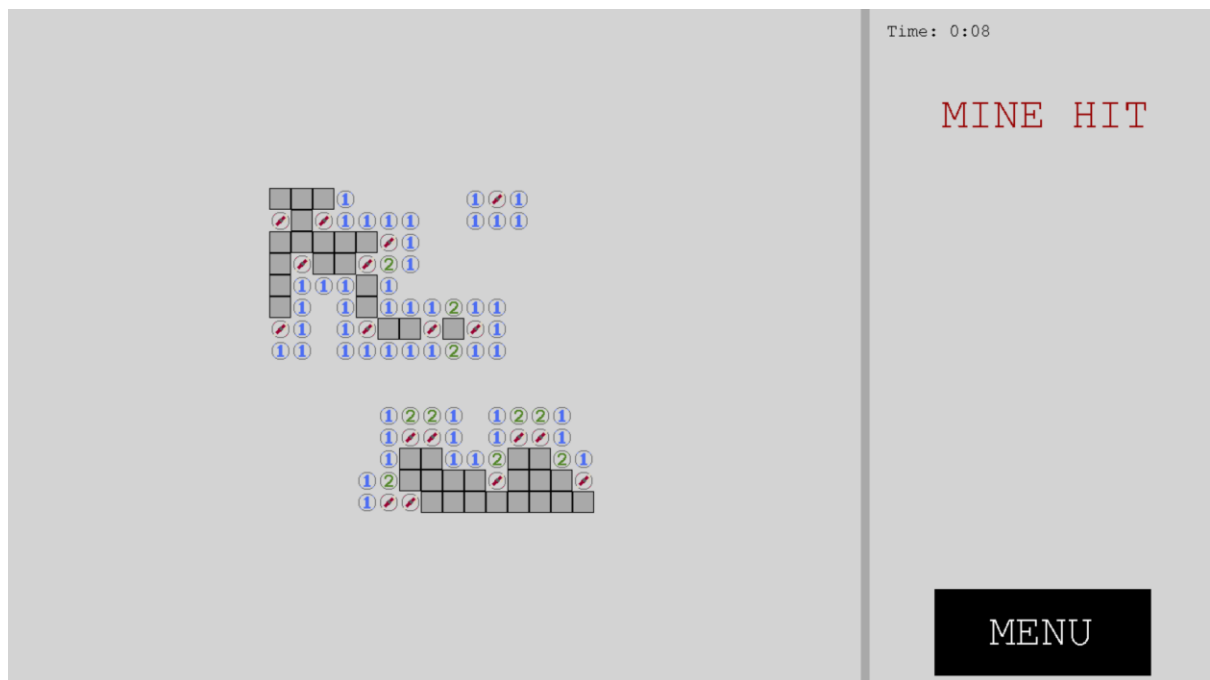
A solution to this would be to add some code into 'creategrid()' that will put force a mine into the grid, or keep calling 'creategrid()' until a grid is made which has at least one mine.

TEST RESULTS (IMAGES, IF APPLICABLE)

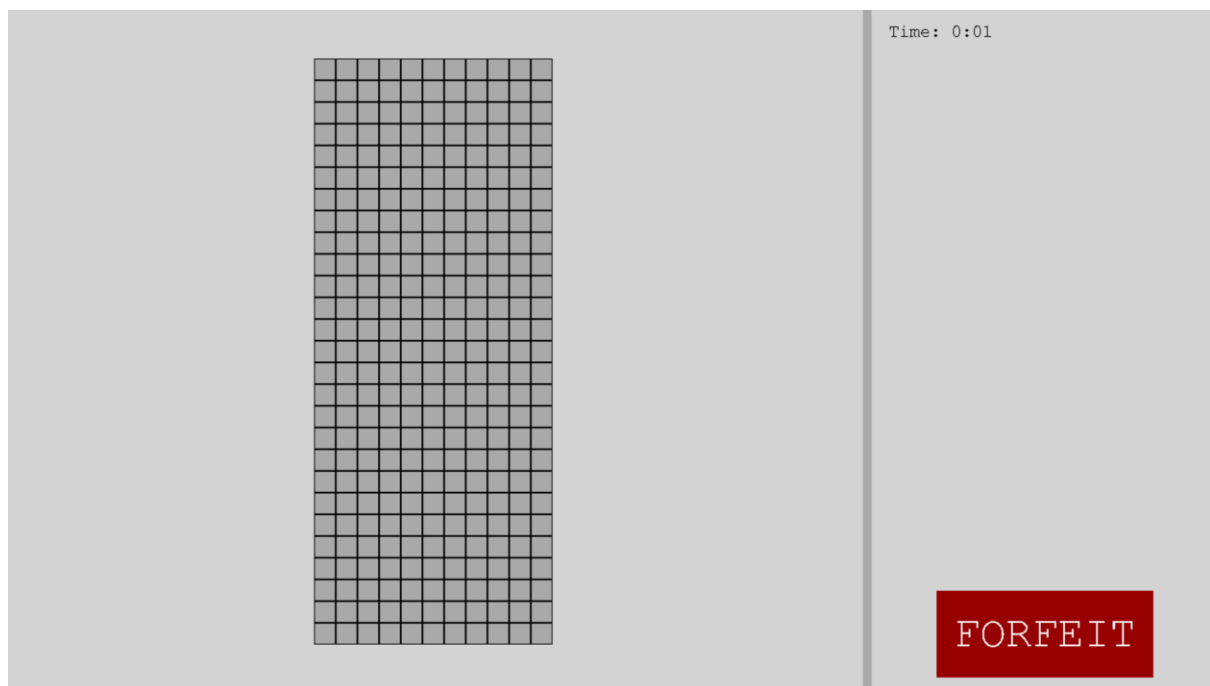
Results that are the same as the ones in prototype 2 will not be shown here.

Tests **2a 2b 2d 5a 5b 5c 5d 6a 6b**:





Tests **7a 7b 7c 7d**:





FIXED ERRORS

```
mineCount = 0

for i in range(len(objects)):
    for j in range(len(objects[i])):
        if objects[i][j].mine:
            mineCount += 1

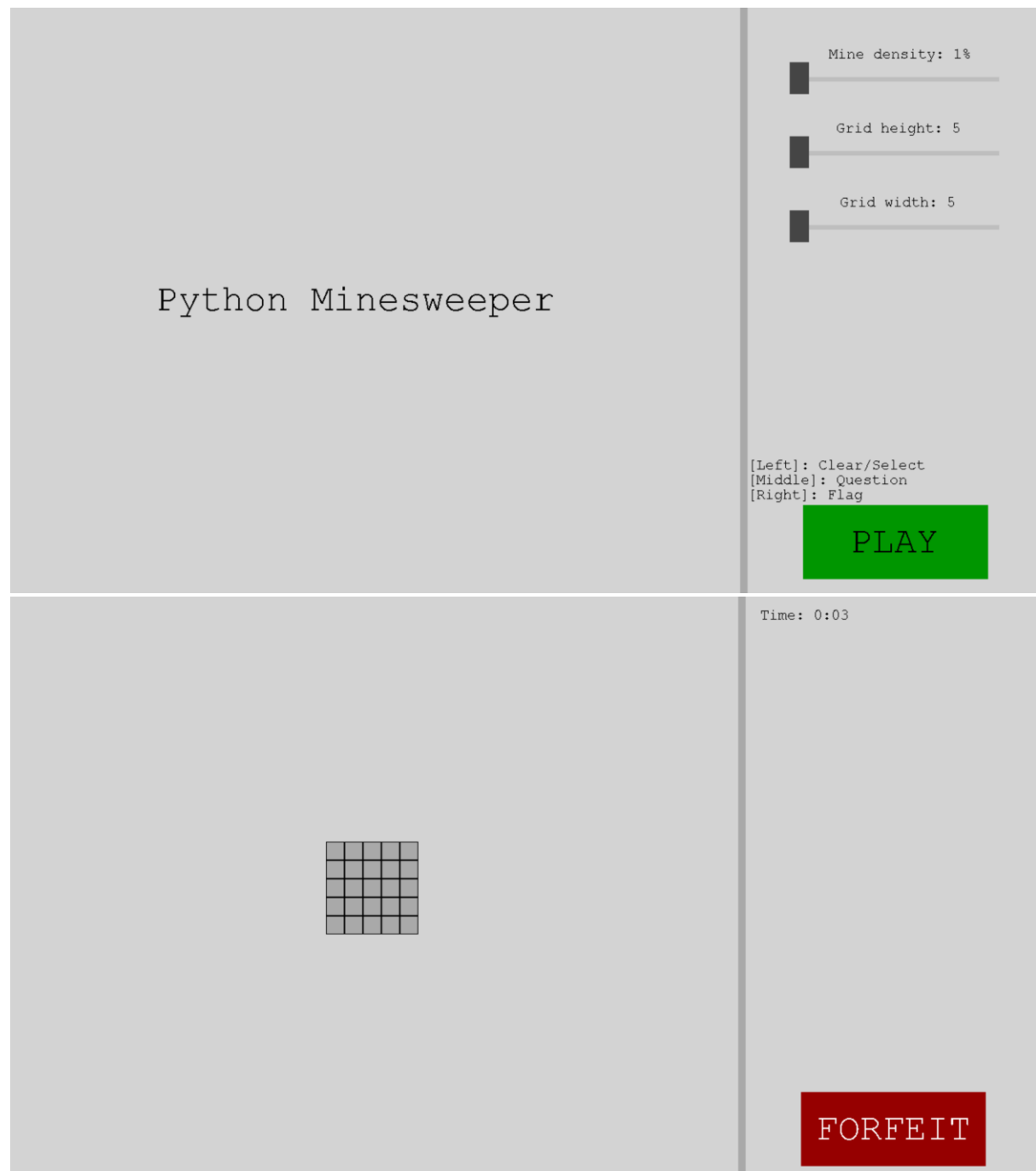
if mineCount == 0:
    objects[0][0] = Tile(True, -1, size, (startX, startY))
    objects[0][0].nDisplay = pygame.image.load(GridImages['M'])
    objects[0][0].nDisplay = pygame.transform.scale(objects[0][0].nDisplay, (size, size))
```

Here is the code I have written to fix the lose mine density bug. It defines a variable which will store the number of mines in the grid. A set of 'for' loops are then run to traverse the list. If the current tile in the traversal is a mine, then 1 is added to the counter.

After this has run, an 'if' statement checks if the counter is zero. If so, then it sets the top left tile to a mine and saves and scales the mine image to that tile. If it isn't zero, then it runs as normal.

This block of code runs after the original loops and just before 'minedistance()' is called. This is located in 'creategrid()'.

FIXED ERRORS (IMAGES)

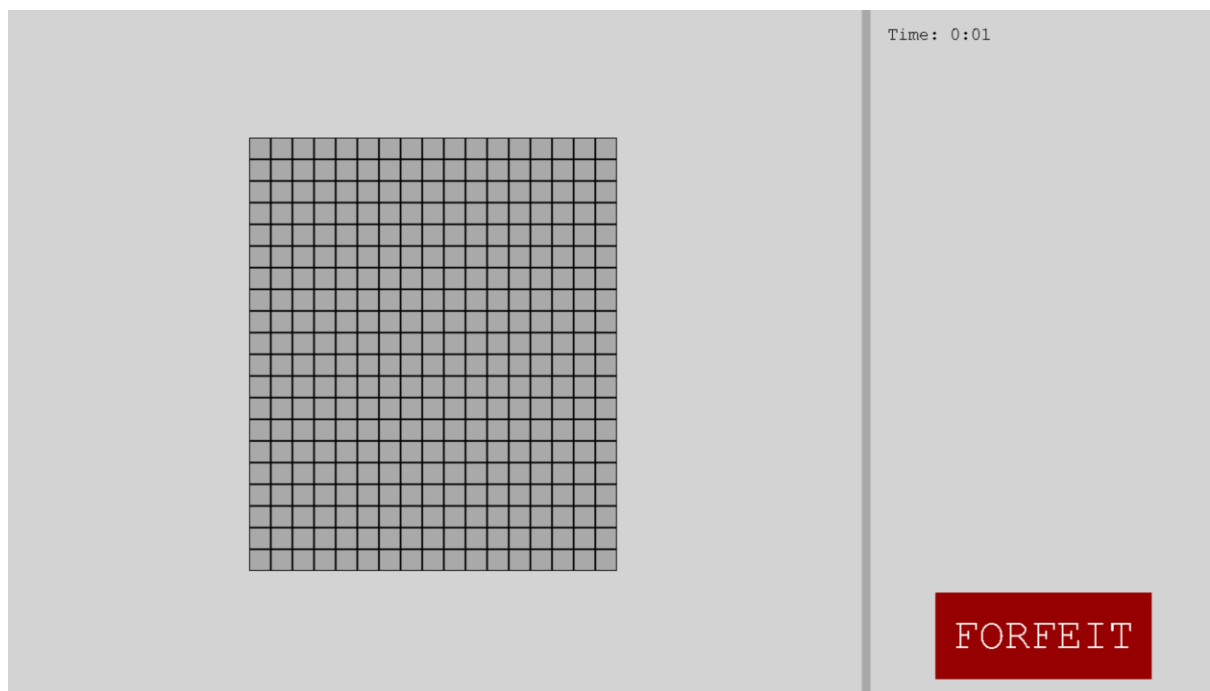
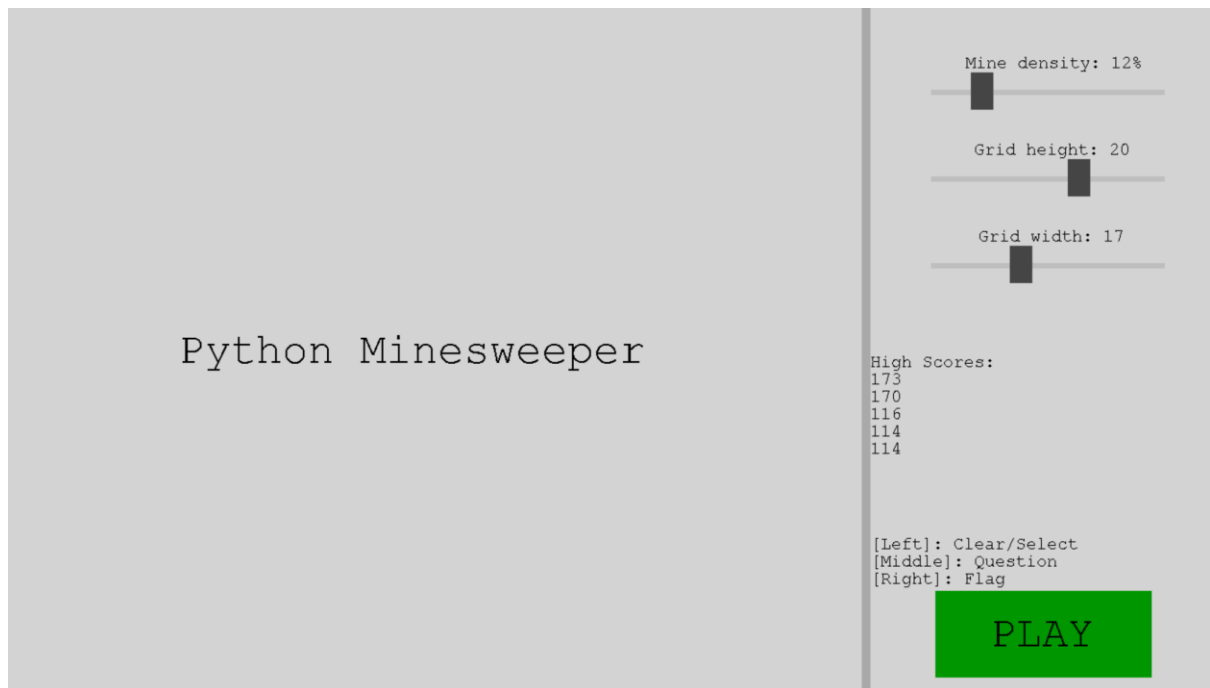


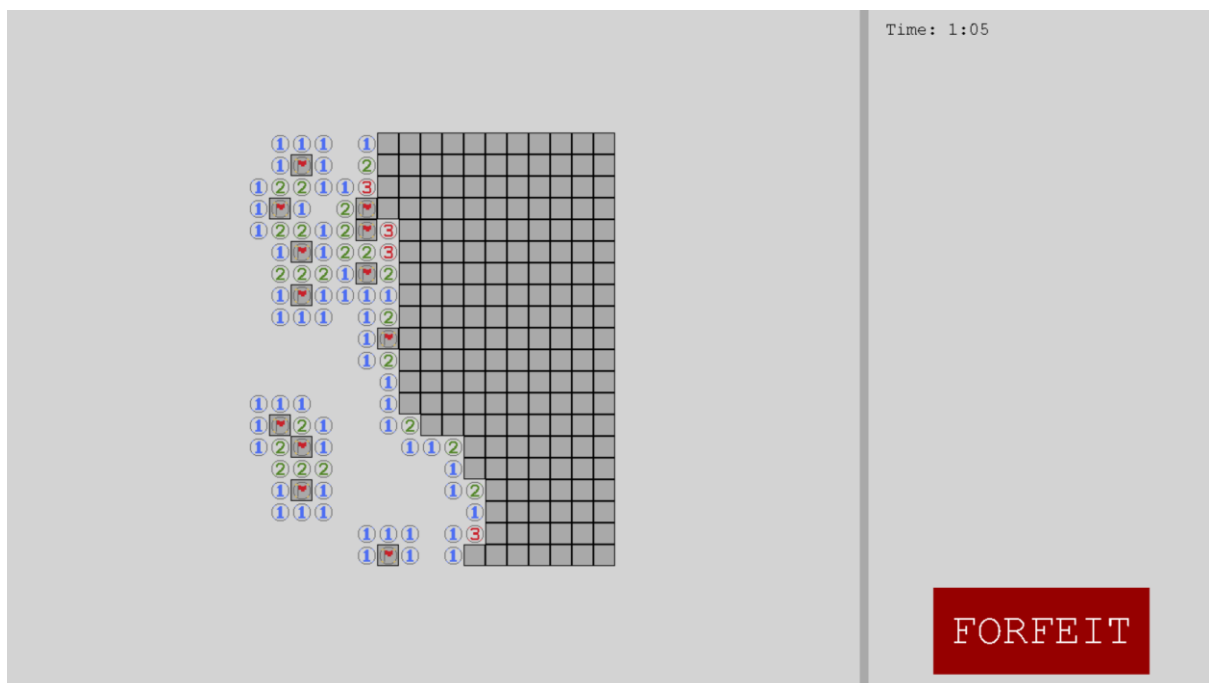
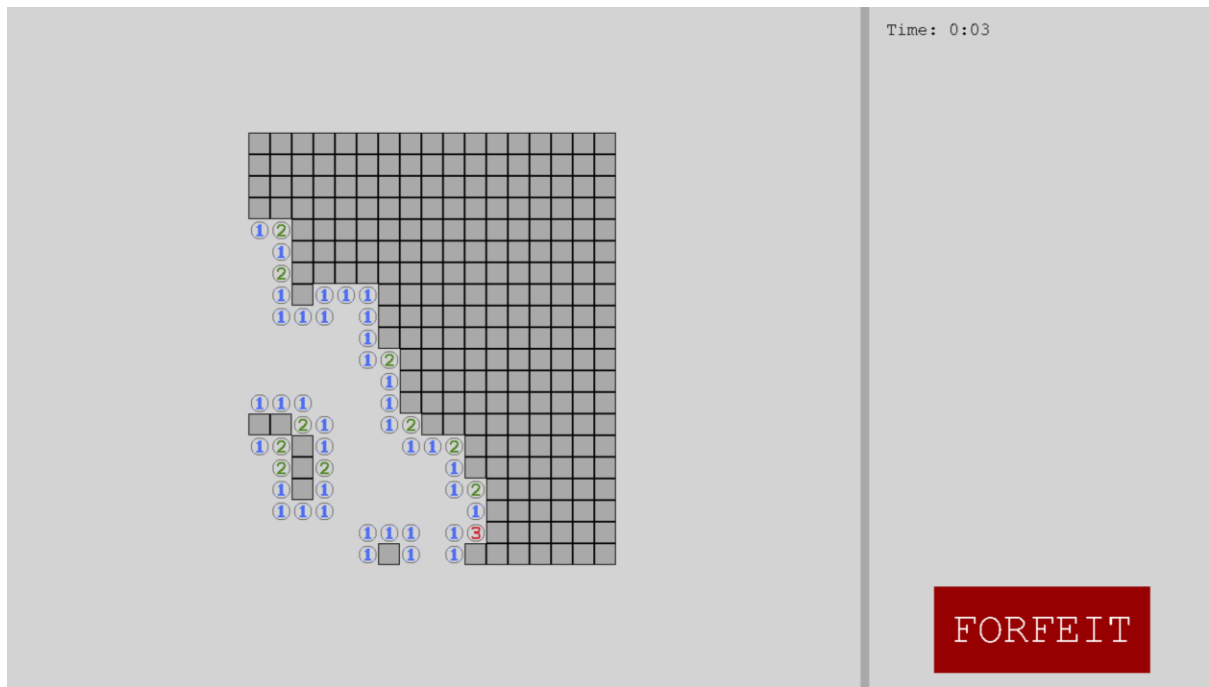


As you can see above, there wouldn't have been a mine here to begin with, so a mine is forced onto the grid in the top left position.

FINAL PROGRAM IMAGES

Below are a series of images showing a game of minesweeper being played and a new high score being achieved, saved, and displayed on the menu screen.





Time: 2:40

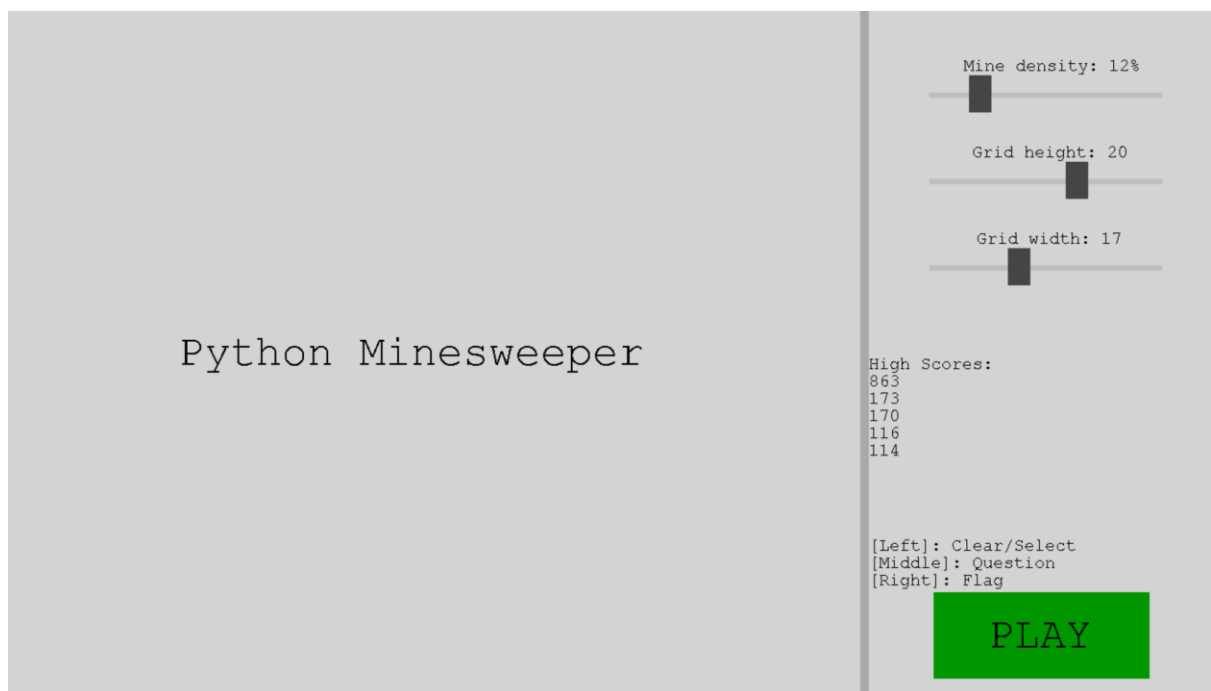
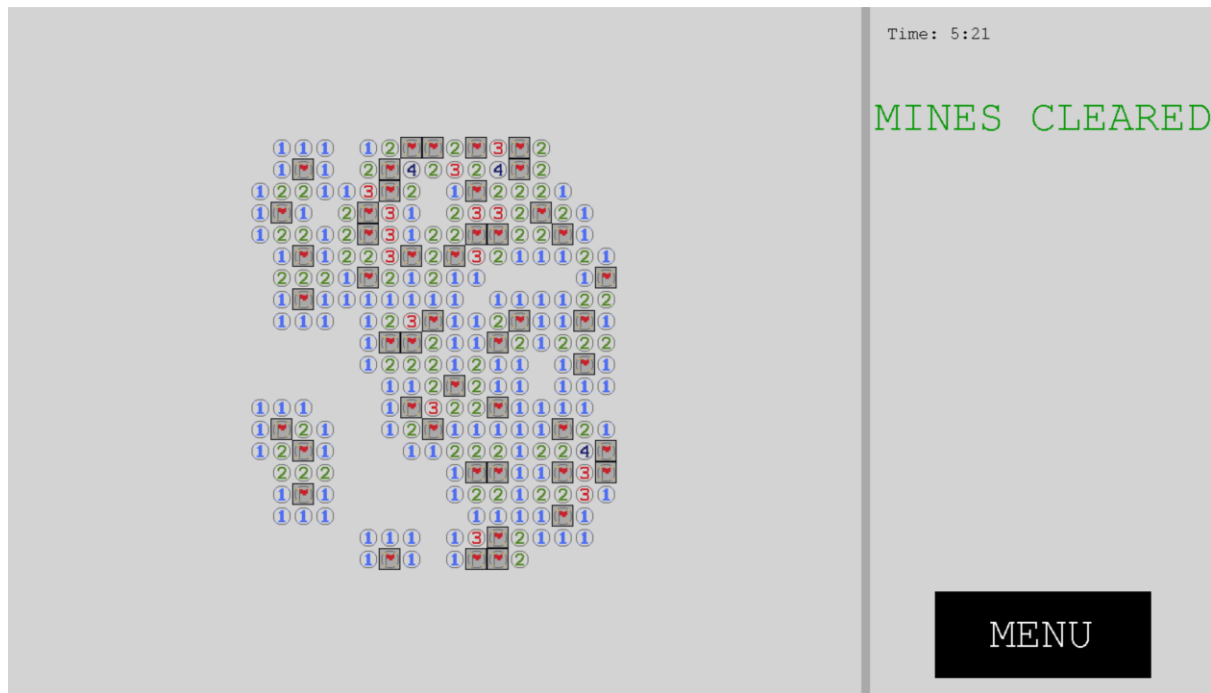


FORFEIT

Time: 4:17



FORFEIT



CONCLUSION

Here is a copy of the success criteria from the Analysis Section.

Requirement	Criteria	Justification
Mine-Distance calculations	The numbers next to each mine are calculated quickly (within 2 seconds) and correctly.	In order for the game to be playable.

Quick startup	The program starts up within 5 seconds.	To ensure the game is quick and efficient.
Quick processing	The program processes user inputs and changes the grid where fit within 1 second.	To ensure the game is quick and efficient.
Win and Lose conditions	Win: The program ends if all mines have been flagged or there are only mined tiles left. Lose: The program ends if a mined tile is hit, except for the first go, in which the mine will be relocated, and distances recalculated.	In order for the game to be playable.
Grid creation	The grid is successfully created to the desired height and width which is chosen by the user, within a set range.	In order for the game to be playable.
Mine locations	Using the grid and a random number generator, the grid is successfully filled with mines using the random number generator to change the likelihood that a mine is put there.	In order for the game to be playable.
Display grid	The grid is displayed correctly and is in proportion with the screen. The entire grid fits in the screen and isn't being cut off.	In order for the game to be playable.
High scores	High scores are saved onto an external text file. The score and 3 letter initials are written onto the file. Only the top 5 scores are written.	Extra features to make the program more unique and interesting.

In conclusion, the final version of my program is fully functional and meets the requirements of the success criteria. Errors outlined in prototype 2 have been fixed and I have achieved what I planned to do at the start of this section.

PROGRAM EVALUATION

SUCCESS CRITERIA

Test program against success criteria, what was achieved, partially or not at all.

Below is a copy of success criteria for the program from the analysis section and have numbered each one. These criteria are the parts of the program which are the most essential for it to run well and reliably.

No.	Requirement	Criteria	Justification
1	Mine-Distance calculations	The numbers next to each mine are calculated quickly (within 2 seconds) and correctly.	In order for the game to be playable.
2	Quick startup	The program starts up within 5 seconds.	To ensure the game is quick and efficient.
3	Quick processing	The program processes user inputs and changes the grid where fit within 1 second.	To ensure the game is quick and efficient.
4	Win and Lose conditions	Win: The program ends if all mines have been flagged or there are only mined tiles left. Lose: The program ends if a mined tile is hit, except for the first go, in which the mine will be relocated, and distances recalculated.	In order for the game to be playable.
5	Grid creation	The grid is successfully created to the desired height and width which is chosen by the user, within a set range.	In order for the game to be playable.
6	Mine locations	Using the grid and a random number generator, the grid is successfully filled with mines using the random number generator to change the likelihood that a mine is put there.	In order for the game to be playable.
7	Display grid	The grid is displayed correctly and is in proportion with the screen. The entire grid fits in the screen and isn't being cut off.	In order for the game to be playable.
8	High scores	High scores are saved onto an external text file. The score and 3 letter initials are written onto the file. Only the top 5 scores are written.	Extra features to make the program more unique and interesting.

No.	Requirement	Result	Pass/Fail
1	Mine-Distance calculations	The numbers are calculated correctly and images assigned without error.	Pass
2	Quick startup	The program starts up within 2 seconds.	Pass
3	Quick processing	The processing is almost seamless.	Pass
4	Win and Lose conditions	Win: The game is won then all mines are flagged, or all non-mine tiles are removed. Lose: The game is lost when the player hits a mine.	Pass
5	Grid creation	The grid is created to the height and width the user inputted.	Pass
6	Mine locations	The mines are assigned randomly to the rough percentage that the user had inputted.	Pass
7	Display grid	The grid is displayed using a 'for' loop and the coordinates stored in the class instance for each for each tile.	Pass
8	High scores	High scores are calculated upon game win and are stored in the high score file if it is a higher value than at least one of the existing ones in it.	Pass

Evidence of these tests are in the testing sections in the code development sections, as they have been used as a baseline set of tests for each prototype, with extra tests added on which are prototype specific.

USER TESTING

To test the program, two players, both in the 17-18 age range, played the game and provided feedback with their opinions and ideas.

USABILITY

Both players commented on the ease of using the menu, and that it wasn't complicated or confusing to use. They also mentioned that the instructions for the mouse controls on the menu screen were all they needed to understand how to work the game.

This will show that the usability of the game is good, and nothing needs to be added to make it easier for the player to understand how to use the game.

In general, avoiding the use of a keyboard for all inputs in the game has allowed for a simple and easy way to control the game by only allowing three inputs of the left, right and middle clicks. Programming the inputs this way has also created a 'built-in' validation without needing to program specific validation functions. This is because the mouse clicks and coordinates of the cursor serve as their own range validation when checking if it is in a hitbox.

APPEARANCE

Both players were positive about the appearance of the GUI and program as a whole. Player one stated that the appearance was easy to look at and doesn't hurt the eyes. And player two stated that the appearance had a straightforward layout and made it easy to understand what everything does.

This will show that the simple appearance is a good one to go for as there isn't too much going on that could irritate the player or hurt their eyes.

In general, the use of a simple menu layout and plain colour scheme makes it easy to understand and not complicated for the user. This ties in with the usability of the game, as it makes it as easy as possible for the player to use the game. The only thing the player needs to know is an understanding of how to play Minesweeper.

PLAYING EXPERIENCE

The game itself was smooth playing for both players. But player one had some issues with moving the slider to the end, due to the hitboxes not being able to handle the moving quickly and perhaps the size of the hitboxes being too small. Player two had no issues with this.

Player one also encountered an error where they hit an empty tile, but the adjacent tiles didn't clear and only that tile was cleared, creating a hole with no numbers showing. I'm unsure of the cause of this, but it might be to do with clearing a tile very quickly after pressing 'play'.

This shows that the actual playing part of the game run well, but there are some parts that need to be refined to make the experience more positive overall.

ERRORS/LIMITATIONS

Both players encountered small errors/ bugs which didn't impact the overall experience but are still things that should be changed. As mentioned before, player one encountered an error which can cause an empty tile to show without any adjacent tiles being cleared. Whereas player two discovered that they can use as many flags as they want, so they would be able to win the game by flagging every tile until all mines are flagged. Another limitation would be the sliders not being reliable when the player drags it quickly, this is down to how the hitboxes are programmed into the slider and could be fixed by enlarging the hitbox for each notch.

This shows that although they are minor errors, there is still work to be done to remove errors and exploits that the player can use.

In general, there aren't many errors throughout the program, being found by the players during testing. The two errors/ limitations are the only ones that I know of. These two are only minor errors, with the slider hitbox not really being an issue altogether, just something that should be tweaked. The error with the empty tiles not clearing is more of a problem, as it can impact the gameplay. Still however, the tile next to the cleared one can be pressed and all of the adjacent tiles will clear away, which is what they should have done before. I believe it may be caused by the player clearing an empty tile almost immediately after the grid has been created, therefore not giving the program enough time to process this, but I'm not exactly sure and I haven't been able to recreate it since (I was only able to create it once, which was before user testing began), which means that it is quite rare to happen. The error can be resolved by the player by just clearing the tile next to it, clearing all the empty tiles like normal, but it is still an error that will impact the gameplay and the player's experience. An alternative way to fix it is for the player to just re-tap the tile, which may mean its an issue with the images, where the images think the tile has been cleared, but the rest of the program doesn't, so `clearEmpty()` isn't called.

In terms of program maintenance, I believe that other programmers would be able to understand how my program works. I have commented in a way where it shows what the important pieces of code do but doesn't mention how it works. This might mean that any potential programmer would need to study how the code works, but they should ultimately be able to understand and add to the program.

CHANGES THEY WANT TO SEE

Both players have things that they would want to. Player one would like to see the flag counter being implemented to help with seeing how many flags they can use. And player two would like and an explosion animation when the player this a mine to make it more exciting.

OVERALL OPINION

Overall, both players liked the program and thought that it was an accurate representation and version of Minesweeper.

REFLECTION/ NEXT STEPS

The program does not contain all of the features I would have like it to have, and if I had more time, I would try to program them in.

If I had a short amount of time, like a few weeks, then I would focus on adding a flag counter which is show and the limit the user to how many flags they can use. I would also improve the design of the menu screen and overall GUI, and perhaps add a dark mode, where the light grey is swapped for black or dark grey and the tile image colours are whites and greys. The focus on these would be to improve the player experience and make it easier to use and understand, by giving the player all of the information they need in a non-complex and easy to look at way.

If I had a longer amount of time to make changes, then I would try to implement a hexagonal version of Minesweeper as an additional option the player could select when in the menu. I would also consider rewriting

the program in Lua using the LOVE2D game engine, which will allow me to edit the properties of the window easier. The layout of the program would also make it easier to read and edit, by programming in predefined 'load()', 'update()' and 'draw()' functions.

I would also fix the errors that were found in user testing and anymore that I would find whilst doing these improvements. Refining the slider is also something I would do, improving its reliability by making the hitboxes for each notch taller, which means the player can be less accurate with the mouse when dragging it, and the correct notch will still be selected.

OVERALL CONCLUSION

Overall, the program does what I wanted it to do in the design section, with a customisable grid and high scores. The program works with a couple of errors present but doesn't crash the program or affect the player in any major way.

The program has also differed from the original idea. For example, the GUI is a different design from the one seen in the design section, where what was going to input by keyboard is now input by the mouse through the use of the three sliders. All of the possible inputs the user can make are three mouse buttons, which serves as its own form of input validation as they're only able to choose from a set of numbers defined in the program. This means there is no way to crash the program through bad inputs.